



Hausarbeit

Beispieltitel
für einen Bericht

Autor: Vorname Nachname
Matrikel-Nr.: xxxxxxxx
Studiengang: Maschinenbau

Erstprüfer: Prof. Dr. Elmar Wings
Abgabedatum: 21. Juni 2026

Inhaltsverzeichnis

Inhaltsverzeichnis	3
Abbildungsverzeichnis	9
Liste des Listings	11
I. Einleitung	15
1. Einführung	17
2. Aufgabenstellung	19
3. Stand der Technik	21
4. Einbettung des Projektes	23
5. Besondere Herausforderung	25
5.1. Lösung der Herausforderung	25
6. Inhalte der folgenden Kapitel	27
II. Domain Knowledge - Application	29
7. Applikationen	31
III. Domain Knowledge - Hardware	33
8. Arduino Nano 33 BLE Sense	35
8.1. Allgemeine Beschreibung	35
8.2. Mikrocontroller und Leistungsdaten	36
8.3. Betriebsspannung und Logikpegel	36
8.4. Digitale Ein- und Ausgänge	36
8.5. PWM und Servo-Ansteuerung	37
8.6. Zeitmessung und Ablaufsteuerung	37
8.7. USB, Programmierung und serielle Kommunikation	38
8.8. Integrierte Funktionen und Sensorik	38
8.9. Further Readings	39
9. Ultraschallsensor HC-SR04	41
9.1. Allgemeines	41
9.2. Ultraschallsensor HC-SR04	41
9.3. Spezifikation	41
9.4. Bibliothek	42
9.4.1. Beschreibung	42
9.4.2. Installation	42

9.4.3. Funktionen	42
9.5. Beispiel	44
9.5.1. Handbuch	44
9.5.2. Funktion vom Beispiel	45
9.5.3. Code	45
9.6. Kalibrierung	46
9.7. Einfacher Code ohne newPing Bibliothek	46
9.8. Einfache Anwendung	47
9.9. Tests	48
9.9.1. Einfacher Funktionstest	48
9.9.2. Alle Funktionen testen	48
9.10. Weiterführende Literatur	49
10. SG90 Servomotor	51
10.1. Allgemeines	51
10.2. Servomotor TowerPro SG90	51
10.3. Spezifikation	51
10.4. Bibliothek	52
10.4.1. Beschreibung	52
10.4.2. Installation	52
10.4.3. Funktionen	53
10.5. Beispiel	53
10.5.1. Handbuch	53
10.5.2. An einem Beispiel	53
10.5.3. Code	53
10.5.4. Dateien	54
10.6. Kalibrierung	54
10.7. Einfacher Code (ohne Bibliothek)	55
10.8. Einfache Anwendung	56
10.9. Tests	56
10.9.1. Einfacher Funktionstest	56
10.9.2. Alle Funktionen testen	56
10.10. Weiterführende Literatur	56
IV. Domain Knowledge - Tools	57
11. Mikrocontroller-Programmierung mit der Arduino IDE	59
11.1. Allgemeines zur Arduino IDE	59
11.2. Das Steuerungsprogramm des Ultraschallradars	59
11.2.1. Quellcode der Arduino-Steuerung	60
12. Arduino IDE 2.3.6	63
12.1. Arduino IDE Beschreibung	63
12.2. Installation der Arduino IDE	64
12.2.1. Download oder Arduino IDE	64
12.2.2. Anforderungen Betriebssystem	65
12.2.3. Installation auf Windows	65
12.2.4. Installation (MacOS)	68
12.3. Übersicht der Eigenschaften	69
12.3.1. Sketchbook	70
12.3.2. Boards Manager	71
12.3.3. Library Manager	71

12.4. Integration und Verwendung einer benutzerdefinierten Bibliothek in der Arduino IDE	72
12.4.1. Erstellen der Bibliotheksdateien	72
12.4.2. Installation und Integration der Arduino IDE	72
12.4.3. Verwendung symbolischer Verknüpfungen zur Bibliotheksintegration	73
12.4.4. Windows-Tool mklink zum Erstellen symbolischer Verknüpfungen	73
12.4.5. Überprüfen der Verfügbarkeit der Bibliothek in der Arduino IDE	74
12.4.6. Serieller Monitor	75
12.4.7. Serieller Plotter	75
12.5. Beispiele	76
12.6. Debuggen	76
12.7. Autovervollständigung	77
12.8. Bibliotheken und Modularität	78
13. Processing 4.3	79
13.1. Processing Beschreibung	79
13.2. Installation von Processing	80
13.2.1. Download von Processing	80
13.2.2. Anforderungen Betriebssystem	81
13.2.3. Installation auf Windows	81
13.2.4. Installation auf macOS	81
13.3. Übersicht der Eigenschaften und Contribution Manager	82
13.3.1. Die serielle Schnittstelle (Serial-Bibliothek)	83
V. Developmenmt	85
14. Flow Chart Development XY	87
VI. Monitoring	89
15. Monitoring XY	91
VII. Verification - Evaluation - Conclusion	93
16. Evaluation/Verification XY	95
17. To DoX Y	97
18. Conclusion XY	99
VIII. Anhang	101
19. Bill Of Material	103
Stichwortverzeichnis Keywords	105
20. SBOM	107
21. Package Example	109
21.1. Introduction	109
21.2. Description	109

21.3. Installation	109
21.4. Example - Manual	109
21.5. Example	109
21.6. Example - Code	109
21.7. Example - Files	109
21.8. Further Reading	109
IX. Hints - Examples for LaTeX - Read, Use and Remove	111
22. Hinweise	113
22.1. Allgemeine mathematische Beschreibung Bézier-Kurve	114
23. Verrundung mit einem Kreisbogen	117
23.1. Gleichungen	117
23.2. Bewertung	119
23.3. Beispiele	121
24. Maple-Dateien	125
24.1. Geometrien mit Maple	125
24.2. Verwendete Geometrieelemente	125
24.3. Aufbau der Datenstruktur	126
24.4. Struktur eines Moduls	126
24.4.1. Genereller Aufbau eines Moduls	128
24.4.2. Sicherung eines Moduls	130
24.4.3. Verwendung eines Moduls	130
24.4.4. Erstellung eines Maple-Moduls	130
24.5. Funktionen der Module	131
24.5.1. Modul <code>MPoint</code>	131
24.5.2. Modul <code>MLine</code>	132
24.5.3. Modul <code>MArc</code>	133
24.5.4. Modul <code>MBezier</code>	133
24.5.5. Modul <code>MPolygon</code>	134
24.5.6. Modul <code>MGeoList</code>	134
24.5.7. Modul <code>MHermiteProblem</code>	135
24.5.8. Modul <code>MHermiteProblemSym</code>	135
24.5.9. Modul <code>MConstant</code>	136
24.5.10. Modul <code>MGeneralMath</code>	136
24.5.11. Modul <code>Biarc</code>	137
24.5.12. Modul <code>MBiarc</code>	137
24.6. Programmablaufplan	138
24.6.1. Gesamauf	138
24.6.2. Verrundung der Kurve	138
25. Representation of Python Programs	141
26. Representation of Programs written for Arduino Boards	143
27. Erstes Kapitel	145
28. CAGD	147
A. Zeichnungen mit tikz	149
B. Kriterien für eine gutes $\text{\LaTeX}$-Projekt	151

Abbildungsverzeichnis

8.1. Arduino Nano 33 BLE Sense – Pinbelegung	35
9.1. Schematische Darstellung des Ultraschallsensors HC-SR04	42
9.2. Schaltplan Ultraschallsensor HC-SR04	43
10.1. Schematische Darstellung des Servomotors TowerPro SG90	52
12.1. Arduino IDE 2.3.6	65
12.2. Arduino IDE Downloadoptionen	66
12.3. Lizenzbestätigung Installation	67
12.4. Wahl Zielverzeichnis	67
12.5. Startfenster Arduino-IDE	68
12.6. ArduinoIDE macOS	69
12.7. Überblick	70
12.8. Sketchbook	70
12.9. Boards Manager	71
12.10 Library Manager	72
12.11 Eingabeaufforderung mit Administrator Rechten	73
12.12 Eingabeaufforderung	74
12.13 Einbinden der Nano33BLESenseLED Bibliothek	75
12.14 Serieller Monitor	75
12.15 Beispiele	76
12.16 Beispiel Debuggen	77
12.17 Autovervollständigung	77
13.1. Processing Downloadverzeichnis	81
13.2. Startfenster der Processing-IDE	82
13.3. Processing Contribution Manager	83
22.1. Bernstein-Polynom vom Grad 3	115
22.2. Bézier-Kurve zum Beispiel 22.1	116
23.1. Glättung einer Ecke mit Hilfe eines Kreisbogens - Dreieck	117
23.2. Winkeländerung bei einer Verrundung mit einem Kreisbogen	119
23.3. Krümmungsverlauf bei der Verrundung mit einem Kreisbogen	119
23.4. Maximaler Bereich für die Verrundung mit einem Kreisbogen - $L(\varepsilon = \text{const}, \alpha)$	120
23.5. Abweichung bei Vorgabe des Abstands L bei der Verrundung mit einem Kreisbogen	120
24.1. Programmablaufplan „Eckenverrundung“	139
24.2. Programmablaufplan „Auswahl der Verrundungsstrategien“	140

Liste des Listings

9.1. Beispiel Code	45
9.2. Beispiel Code ohne Bibliothekseinbindung	46
9.3. Einparkhilfe Beispielcode	47
10.1. Beispiel Code für den TowerPro SG90	53
10.2. Code ohne Bibilothek für den TowerPro SG90	55
25.1. „Hello World“ in Python – Variant 1	142
26.1. „Hello World“ in Python – Variant 1	144

Abkürzungen

CNC Computerized Numerical Control

SPS Speicherprogrammierbare Steuerung

Teil I.

Einleitung

1. Einführung

Die Umgebungserfassung und die präzise Detektion von Hindernissen sind fundamentale Bestandteile moderner Automatisierungs-, Automobil- und Robotiksysteme. Ein klassisches, aber nach wie vor anschauliches Anwendungsbeispiel hierfür ist das Radar-Prinzip beziehungsweise das hier adaptierte Prinzip des Sonars (Sound Navigation and Ranging).

Im Rahmen dieses Projekts wird ein voll funktionsfähiges Ultraschallradar konzipiert, mechanisch konstruiert und informationstechnisch implementiert. Ziel der Arbeit ist es, das komplexe Zusammenspiel von Mikrocontroller-Hardware, Sensorik, Aktuatorik und einer grafischen Benutzeroberfläche an einem praxisnahen, mechatronischen System zu demonstrieren.

Als zentrale Steuereinheit des Systems dient ein Arduino Nano. Die mechanische Scan-Bewegung wird durch einen 180°-Servomotor vom Typ SG90 realisiert, auf dessen Rotationsachse ein HC-SR-04 Ultraschallsensor montiert ist. Um eine stabile Plattform für diese Komponenten zu gewährleisten, wurde im Zuge des Projekts ein maßpassendes Gehäuse im 3D-Druck-Verfahren gefertigt.

Auf der Software-Seite gliedert sich das Projekt in zwei Hauptbereiche: Die Programmierung des Mikrocontrollers (Embedded C/C++) über die Arduino IDE sowie die Entwicklung einer Benutzeroberfläche für den Computer. Letzteres wird mithilfe der auf Java basierenden Umgebung Processing umgesetzt. Diese Software empfängt die seriellen Sensordaten des Arduinos (Winkel und Distanz) kontinuierlich und übersetzt sie in eine dynamische, visuell ansprechende Radar-Anzeige, welche erkannte Objekte und Systemzustände visualisiert.

Der vorliegende Bericht dokumentiert den gesamten Entwicklungsprozess dieses Systems. Er erläutert zunächst die verwendeten Hardwarekomponenten, die mechanische Konstruktion sowie den elektronischen Schaltungsaufbau. Anschließend wird detailliert auf die Programmierung und Implementierung der Steuerungs- sowie Visualisierungssoftware eingegangen.

2. Aufgabenstellung

Im Rahmen der Veranstaltung Automatisierungstechnik soll eine Applikation entwickelt werden, die die Funktionsfähigkeit des Arduino Nano 33 BLE Sense Lite sowie ausgewählter Sensoren und Aktoren demonstriert. Die vorliegende Arbeit befasst sich mit der Realisierung eines Systems zur Umgebungserfassung mittels eines Ultraschall-Abstandssensors, der auf einem Servomotor montiert ist und von einem Arduino Nano 33 BLE Sense Lite angesteuert wird. Ziel ist die Umsetzung eines radarartigen Scans der Umgebung und die Visualisierung der gemessenen Distanzen in Form einer geeigneten Darstellung auf dem PC.

Konkret umfasst die Aufgabenstellung die Integration eines Ultraschall-Abstandssensors (HC-SR-04) und eines Servomotors (Tower Pro SG90) in die vorhandene Hardwareplattform. Der Servomotor soll den Sensor in einem definierten Winkelbereich schwenken, während der Mikrocontroller in vorgegebenen Winkelinkrementen Entfernungswerte aufnimmt und diesen Messdaten einen eindeutigen Winkel zuordnet. Auf Basis dieser Messwerte ist eine Auswertung zu konzipieren, die eine übersichtliche Darstellung der Umgebung ermöglicht.

Neben der reinen Funktionserfüllung werden weitere Anforderungen an die Applikation gestellt. Dazu gehören eine strukturierte und modular aufgebaute Firmware, die Verwendung geeigneter Bibliotheken zur Ansteuerung von Sensor und Aktor. Außerdem sind Verfahren zur Visualisierung der Live-Daten auf einem PC zu entwickeln und zu implementieren, sodass die resultierende Umgebungskarte in Echtzeit nachvollzogen werden kann.

Die Aufgabenstellung umfasst weiterhin die Erstellung einer Entwickler- und Service-Dokumentation. Diese beinhaltet neben der Projektbeschreibung eine detaillierte Beschreibung der Hardware des Arduino Nano 33 BLE Sense Lite und der eingesetzten Komponenten, die Spezifikation und Durchführung von Tests, eine Darstellung der Softwarearchitektur sowie Angaben zu Bezugsquellen und Stückliste (Bill of Materials).

3. Stand der Technik

Ultraschall-Abstandssensoren werden seit vielen Jahren in der Automatisierungstechnik, Robotik und Fahrzeugtechnik zur berührungslosen Distanzmessung eingesetzt.[Leu] Sie arbeiten in der Regel nach dem Laufzeitmessprinzip, bei dem ein kurzer Ultraschallimpuls ausgesendet und die Zeit bis zum Empfang des reflektierten Echos gemessen wird. Aus der Schalllaufzeit lässt sich unter Annahme einer bekannten Schallgeschwindigkeit der Abstand zu einem Hindernis bestimmen. Im Niedrigkostenbereich haben sich dafür Sensormodule wie der HC-SR-04 etabliert, die eine einfache Anbindung an Mikrocontroller über Trigger- und Echo-Signale erlauben.

Für die mechanische Ausrichtung von Sensoren und leichten Lasten werden in zahlreichen Anwendungen kompakte RC Servomotoren eingesetzt. Diese enthalten neben einem Gleichstrommotor ein Getriebe sowie eine integrierte Positionsregelung, die über ein Pulsweitenmodulationssignal (PWM) angesteuert wird. Miniaturservos wie der Tower Pro SG90 sind vor allem in ferngesteuerten Modellen wie Flugzeugen und Helikoptern verbreitet, wo sie etwa Ruderflächen oder andere bewegliche Komponenten ansteuern.[kts:2026] In technischen Lehr- und Demonstrationsaufbauten werden derartige Servos zudem häufig genutzt, um Sensoren in einem begrenzten Winkelbereich zu positionieren.

Die Kombination aus Ultraschall-Abstandssensor und Servomotor ermöglicht die Realisierung von Systemen zur zweidimensionalen Umgebungserfassung. Durch das sequenzielle Abtasten verschiedener Winkelpositionen kann ein Scan der Umgebung erzeugt werden. In frei verfügbaren Projekten und Tutorials wird dieses Prinzip typischerweise mithilfe eines Ultraschallmoduls des Typs HC-SR-04, eines SG90-Servos und eines Arduino Uno oder Arduino Nano umgesetzt. (vgl. [pro15]) Die gemessenen Distanzen werden dabei entweder auf einem seriell angebundenen PC visualisiert oder direkt auf einem Display dargestellt. Derartige Beispiele dienen häufig als Einführung in die Sensordatenerfassung und -visualisierung mit einfachen Mikrocontrollersystemen.

4. Einbettung des Projektes

Das in diesem Projekt realisierte System knüpft an den beschriebenen Stand der Technik an, indem es die etablierte Kombination aus Ultraschall-Abstandssensor HC-04 und Tower Pro SG90-Servomotor zur Umgebungserfassung verwendet. Der Ultraschallsensor ist auf dem Servomotor montiert und wird in einem definierten Winkelbereich geschwenkt, sodass in diskreten Winkelpositionen Entfernungswerte aufgenommen und zu einem Umgebungsbild zusammengeführt werden.

Viele veröffentlichte Beispielprojekte setzen noch auf Arduino-Boards mit 8Bit-Mikrocontrollern.[Rit21] In der vorliegenden Applikation kommt dagegen der Arduino Nano 33 BLE Sense als Steuerplattform zum Einsatz. Diese Hardwarebasis ermöglicht eine strukturierte Implementierung der Mess- und Auswertalgorithmen sowie die Erweiterung des klassischen Ultraschall-Radar-Konzepts um weitergehende Funktionen, etwa eine flexible Visualisierung der Messdaten auf einem PC.

Durch diese Ausgestaltung ordnet sich das Projekt in die Gruppe kostengünstiger Ultraschall-Radar-Systeme ein, entwickelt diese jedoch durch den Einsatz moderner Hardware und eine systematische Softwarestruktur weiter.[Rit21] Es dient damit sowohl als Demonstrator für die Sensordatenerfassung mit dem Arduino Nano 33 BLE Sense als auch als Grundlage für weiterführende Anwendungen in der Automatisierungstechnik.

5. Besondere Herausforderung

Im Rahmen der Entwicklung der Applikation traten mehrere besondere Herausforderungen auf, die sowohl die mechanische Konstruktion als auch die Softwaregestaltung betrafen. Ein zentrales Ziel war der möglichst kompakte Aufbau des Systems. Dabei musste das Gehäuse so dimensioniert und gestaltet werden, dass alle Komponenten, insbesondere der Ultraschallsensor, der Servomotor und das ArduinoBoard, platzsparend integriert werden konnten, ohne die Funktionalität oder Zugänglichkeit zu beeinträchtigen. Eine zusätzliche Herausforderung bestand in der Unterbringung der Jumper-Kabel im Inneren des Gehäuses, um mechanische Kollisionen, Kabelbrüche sowie Beeinflussungen der Sensorik durch bewegte Leitungen zu vermeiden.

Auf der Softwareseite ergaben sich weitere Anforderungen aus der gewünschten Plattformunabhängigkeit der Benutzerschnittstelle. Es sollte eine Anwendung bereitgestellt werden, die sich direkt von einem USB Stick starten lässt und möglichst ohne zusätzliche Installation auf unterschiedlichen Betriebssystemen lauffähig ist. Dies erforderte eine genaue Auswahl der eingesetzten Entwicklungsumgebung, um Abhängigkeiten zu reduzieren und eine robuste Ausführung zu ermöglichen.

Im Verlauf der Entwicklung zeigte sich zudem, dass die Visualisierung der Messdaten auf der Benutzeroberfläche zunächst zu grob und unübersichtlich war. Einzelne Objekte wurden zu groß dargestellt, sodass die eigentlichen Objektdimensionen anhand des Ultraschallradars nur eingeschränkt erkennbar waren.

5.1. Lösung der Herausforderung

6. Inhalte der folgenden Kapitel

Teil II.

Domain Knowledge - Application

7. Applikationen

Teil III.

Domain Knowledge - Hardware

8. Arduino Nano 33 BLE Sense

8.1. Allgemeine Beschreibung

Der Arduino Nano 33 BLE Sense ist ein kompaktes Mikrocontrollerboard aus der Arduino-Nano-Serie. Es eignet sich für eingebettete Anwendungen, bei denen Sensoren ausgelesen, Aktoren angesteuert und Messdaten verarbeitet werden müssen. Durch seine kleine Bauform kann das Board gut in Versuchsaufbauten, Prototypen und mobile Systeme integriert werden.

Wie andere Arduino-Boards dient der Nano 33 BLE Sense als programmierbare Steuereinheit. Er kann digitale und analoge Signale verarbeiten, zeitabhängige Abläufe steuern und über Schnittstellen mit externen Bauteilen kommunizieren. Dadurch kann er in vielen mechatronischen Anwendungen als Bindeglied zwischen Software, Sensorik und Aktorik eingesetzt werden.

Obwohl der Arduino Nano 33 BLE Sense mehrere integrierte Sensoren besitzt, muss ein Projekt diese nicht zwingend verwenden. Das Board kann auch ausschließlich zur Ansteuerung externer Komponenten eingesetzt werden. In diesem Fall stehen vor allem die Rechenleistung des Mikrocontrollers, die digitalen Ein- und Ausgänge, die Zeitmessung, die PWM-Ausgabe und die serielle Kommunikation im Vordergrund.

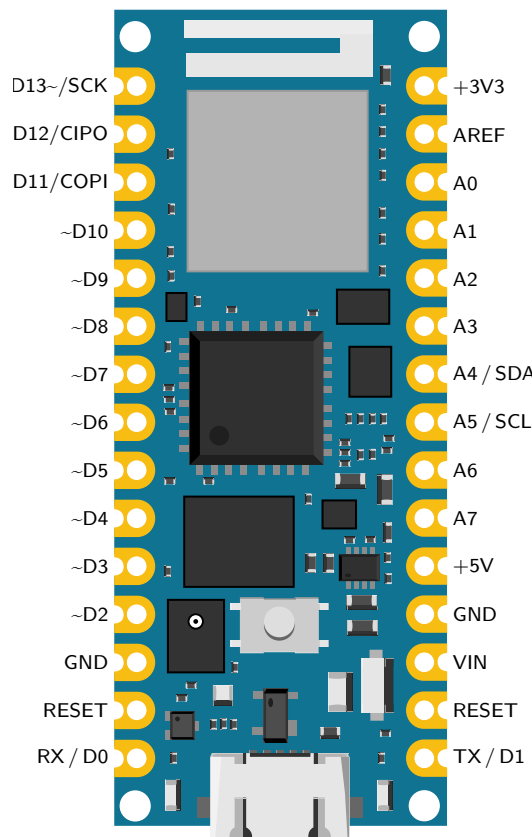


Abbildung 8.1.: Arduino Nano 33 BLE Sense – Pinbelegung

8.2. Mikrocontroller und Leistungsdaten

Der Arduino Nano 33 BLE Sense basiert auf dem Nordic nRF52840 Mikrocontroller. Dieser enthält einen Arm Cortex-M4F-Prozessor mit einer Taktfrequenz von 64 MHz. Laut Arduino-Datenblatt besitzt das Board 1 MB Flash-Speicher und 256 KB SRAM [Ard26a]. Damit verfügt das Board über ausreichend Leistung für typische Steuerungs-, Mess- und Auswertungsaufgaben.

Der Flash-Speicher dient zur Speicherung des Programmcodes. Der SRAM wird während der Programmausführung für Variablen, Messwerte und Zwischenergebnisse verwendet. Für Anwendungen wie das Einlesen eines Sensors, das Erzeugen eines Steuersignals oder das Berechnen einfacher physikalischer Grössen ist die vorhandene Rechenleistung in der Regel ausreichend.

Eigenschaft	Wert/ Bedeutung
Mikrocontroller	Nordic nRF52840
Prozessorkern	Arm Cortex-M4F
Taktfrequenz	64 MHz
Flash-Speicher	1 MB für Programmcodes
SRAM	256 KB für Variablen und Zwischenspeicher
Betriebsspannung	3,3 V Logikspannung
Digitale Ein- und Ausgänge	Anschluss externer Sensoren und Aktoren
USB	Programmierung und serielle Kommunikation

Tabelle 8.1.: Wichtige Kenndaten des Arduino Nano 33 BLE Sense

8.3. Betriebsspannung und Logikpegel

Ein wichtiger Punkt bei der Verwendung des Arduino Nano 33 BLE Sense ist die Betriebsspannung der Ein- und Ausgänge. Das Board arbeitet mit einer Logikspannung von 3,3 V und ist laut Arduino-Datenblatt nicht 5-V-tolerant [Ard26a]. Das bedeutet, dass an den digitalen und analogen Eingangspins maximal 3,3 V anliegen dürfen. Höhere Spannungen, wie sie beispielsweise bei vielen 5-V-Sensoren oder älteren Arduino-Modulen auftreten, können den Mikrocontroller beschädigen[Bri].

Dies ist insbesondere bei der Kombination mit externen Sensoren und Aktoren relevant. Viele elektronische Module werden mit 5 V betrieben und geben ihre Ausgangssignale ebenfalls mit einem Pegel von 5 V aus. Vor dem Anschluss solcher Komponenten muss daher geprüft werden, ob deren Signalpegel mit dem Arduino Nano 33 BLE Sense kompatibel sind.

Falls ein Ausgangssignal eines externen Moduls oberhalb von 3,3 V liegt, ist eine Pegelanpassung erforderlich. Diese kann beispielsweise mit einem Spannungsteiler oder einem Pegelwandler umgesetzt werden. Ein Spannungsteiler reduziert die Eingangsspannung mithilfe zweier Widerstände auf einen für den Mikrocontroller zulässigen Wert. Ein Pegelwandler eignet sich insbesondere dann, wenn Signale bidirektional übertragen werden oder eine robustere Anpassung zwischen unterschiedlichen Logikspannungen erforderlich ist[Jim].

Die Berücksichtigung der Logikpegel ist somit sowohl für die Funktionsfähigkeit der Schaltung als auch für den Schutz des Mikrocontrollers von Bedeutung.

8.4. Digitale Ein- und Ausgänge

Die digitalen Anschlüsse des Arduino Nano 33 BLE Sense können je nach Anwendung als Eingang oder Ausgang verwendet werden. Wird ein Pin als Ausgang konfiguriert, kann der Mikrocontroller darüber ein digitales Signal ausgeben, das entweder

den Zustand High oder Low besitzt. Bei einer Konfiguration als Eingang kann der Mikrocontroller den anliegenden Signalzustand erfassen und auswerten.[Arda]

Auf diese Weise lassen sich verschiedene externe Komponenten mit dem Arduino verbinden. Digitale Eingänge können beispielsweise zum Einlesen von Tastern oder Sensorsignalen genutzt werden. Digitale Ausgänge eignen sich hingegen zur Ansteuerung von LEDs, Treiberschaltungen oder anderen elektronischen Baugruppen[Arda]. Neben dem einfachen Erfassen von High- und Low-Zuständen kann der Mikrocontroller auch zeitabhängige digitale Signale auswerten. Dabei wird nicht nur erkannt, ob ein Signal anliegt, sondern auch, wie lange ein bestimmter Signalzustand andauert. Diese Funktion ist besonders bei Sensoren relevant, die ihre Messinformation über die Dauer eines Impulses ausgeben[Arda].

Ein Beispiel dafür ist der Ultraschallsensor HC-SR04. Bei diesem Sensor wird die Messung durch einen kurzen Trigger-Impuls gestartet. Laut Datenblatt reicht dafür ein Impuls mit einer Dauer von 10 μ s aus. Anschließend gibt der Sensor am Echo-Ausgang ein Signal aus, dessen Impulsdauer von der Entfernung zum reflektierenden Objekt abhängt.

8.5. PWM und Servo-Ansteuerung

Neben der Verarbeitung einfacher digitaler Signale kann der Arduino Nano 33 BLE Sense auch pulswidenmodulierte Signale ausgeben. Bei der Pulsweitenmodulation wird ein digitales Signal in regelmäßigen Zeitabständen ein- und ausgeschaltet. Dabei ist nicht nur die Periodendauer des Signals entscheidend, sondern vor allem die Dauer des High-Zustands innerhalb einer Periode. Dieses Verhältnis wird als Tastgrad bezeichnet und kann zur Ansteuerung verschiedener elektronischer Komponenten genutzt werden[Ardc].

Ein typisches Anwendungsbeispiel für pulswidenmodulierte Signale ist die Ansteuerung von Servomotoren. Der Tower Pro SG90 ist ein kompakter Modellbau-Servomotor, dessen Winkelposition über eine Signalleitung vorgegeben wird. Dazu erhält der Servo ein periodisches Steuersignal mit einer Periodendauer von etwa 20 ms. Die gewünschte Position wird über die Breite des High-Impulses bestimmt. Ein kurzer Impuls von ungefähr 1 ms führt zu einer Endposition, ein Impuls von etwa 1,5 ms zur Mittelstellung und ein Impuls von ungefähr 2 ms zur gegenüberliegenden Endposition.

Der Arduino kann diese Steuersignale über einen geeigneten digitalen Ausgang erzeugen und den Servomotor dadurch auf unterschiedliche Winkelpositionen einstellen. Dadurch lassen sich einfache Bewegungsabläufe realisieren, beispielsweise das Schwenken eines Sensors oder das Positionieren kleiner mechanischer Bauteile. Da ein Servomotor beim Anlaufen oder unter Last deutlich mehr Strom benötigt als ein Mikrocontroller-Pin bereitstellen kann, darf die Versorgung des Motors nicht direkt über einen digitalen Ausgang erfolgen. Der Ausgangspin dient nur zur Signalübertragung, während die Stromversorgung des Servos separat ausreichend dimensioniert werden muss[Ardd].

8.6. Zeitmessung und Ablaufsteuerung

Eine wichtige Aufgabe von Mikrocontrollern ist die zeitliche Steuerung von Abläufen. Der Arduino kann Signale zu definierten Zeitpunkten ausgeben, kurze Impulse erzeugen und die Dauer externer Signale messen. Dadurch lassen sich Sensorabfragen, Bewegungsabläufe und Berechnungen in einer festgelegten Reihenfolge ausführen.

Bei Anwendungen mit Sensoren und Aktoren ist diese zeitliche Reihenfolge besonders wichtig. Wird beispielsweise ein Sensor mechanisch bewegt, sollte die Messung erst erfolgen, nachdem die gewünschte Position erreicht wurde. Andernfalls können die Messwerte ungenau oder schwer interpretierbar sein. Ebenso benötigen manche Sensoren einen Mindestabstand zwischen zwei Messungen, damit sich aufeinanderfolgende

Messvorgänge nicht gegenseitig beeinflussen.

Ein allgemeiner Ablauf für eine kombinierte Sensor-Aktor-Anwendung kann daher aus mehreren Schritten bestehen:

1. Aktor auf eine gewünschte Position stellen.
2. Kurze Wartezeit einhalten, bis die Bewegung abgeschlossen ist.
3. Sensorabfrage auslösen.
4. Eingangssignal messen.
5. Messwert berechnen oder auswerten.
6. Ergebnis speichern oder über eine Schnittstelle ausgeben.

8.7. USB, Programmierung und serielle Kommunikation

Der Arduino Nano 33 BLE Sense wird über eine USB-Verbindung mit dem Computer verbunden. Diese Verbindung dient einerseits zur Spannungsversorgung des Boards und andererseits zur Übertragung des Programms auf den Mikrocontroller. In der Arduino IDE wird dazu das passende Board sowie der zugehörige Anschluss ausgewählt. Anschließend kann der Quellcode kompiliert und auf das Board geladen werden [Ardb]. Neben der Programmübertragung kann die USB-Verbindung auch für die serielle Kommunikation zwischen Mikrocontroller und Computer genutzt werden. Dafür stellt Arduino die serielle Schnittstelle zur Verfügung. Über Befehle wie `Serial.begin()` und `Serial.println()` können Daten vom Mikrocontroller an den Computer gesendet und im seriellen Monitor angezeigt werden [Ardb].

Die serielle Ausgabe ist insbesondere während der Entwicklung und Inbetriebnahme eines Versuchsaufbaus hilfreich. Über den seriellen Monitor können beispielsweise Rohdaten, berechnete Messwerte oder Statusmeldungen ausgegeben werden. Dadurch lässt sich überprüfen, ob angeschlossene Sensoren plausible Werte liefern, ob Steuersignale korrekt erzeugt werden und ob die Verarbeitung der Messdaten im Programm wie vorgesehen abläuft.

8.8. Integrierte Funktionen und Sensorik

Der Arduino Nano 33 BLE Sense verfügt neben den allgemeinen Mikrocontrollerfunktionen über mehrere integrierte Sensoren sowie eine Bluetooth-Low-Energy-Schnittstelle. Zur Sensorik des Boards gehören eine Inertialmesseinheit zur Erfassung von Beschleunigung, Winkelgeschwindigkeit und Magnetfeld, ein Umgebungslicht-, Farb-, Gesten- und Näherungssensor, ein barometrischer Drucksensor, ein Temperatur- und Feuchtigkeitssensor sowie ein digitales Mikrofon [Ard26a].

Diese integrierten Komponenten erweitern die Einsatzmöglichkeiten des Boards deutlich. Sie ermöglichen beispielsweise Anwendungen in den Bereichen Bewegungserkennung, Umgebungsüberwachung, Gestensteuerung, Audioverarbeitung und drahtlose Datenübertragung. Welche Sensoren oder Schnittstellen tatsächlich genutzt werden, hängt jedoch vom jeweiligen Aufbau und der Zielsetzung der Anwendung ab.

Im Rahmen dieses Projekts werden die Onboard-Sensoren des Arduino Nano 33 BLE Sense nicht verwendet. Stattdessen dient das Board als zentrale Steuereinheit für externe Komponenten. Dazu zählen der Ultraschallsensor HC-SR04 zur Entfernungsmessung und der Servomotor Tower Pro SG90 zur Ausrichtung des Sensors. Im Vordergrund stehen daher nicht die integrierte Sensorik des Boards, sondern die digitalen Ein- und Ausgänge, die Zeitmessung, die Signalverarbeitung sowie die Ansteuerung des Servomotors.

8.9. Further Readings

9. Ultraschallsensor HC-SR04

Einleitung

Dieses Kapitel befasst sich mit der berührungslosen Abstandsmessung mithilfe von Sensoren. Zunächst wird ein allgemeiner Überblick über Ultraschallsensoren zur Distanzmessung gegeben. Anschließend wird der in diesem Projekt verwendete Ultraschallsensor HC-SR04 detailliert beschrieben, einschließlich seiner technischen Spezifikationen und der softwareseitigen Implementierung unter Verwendung ausgewählter C++ Bibliotheken.

9.1. Allgemeines

Sensoren zur Abstandsmessung sind wichtige Komponenten in der modernen Automatisierungstechnik, Robotik und Fahrzeugtechnik [Fra16]. Sie ermöglichen es Systemen, eine Umgebung räumlich zu erfassen. Ultraschallsensoren arbeiten nach dem Echo-Lot-Prinzip (Schalllaufzeitmessung). Sie senden Schallwellen aus und messen die Zeit, bis das Echo zurückkehrt [Ele20]. Sie sind unabhängig von der Farbe oder optischen Transparenz eines Objekts, haben jedoch ihre Grenze bei schallabsorbierenden Materialien (z. B. Schaumstoff) [Fra16].

9.2. Ultraschallsensor HC-SR04

In diesem Projekt kommt der Ultraschallsensor **HC-SR04** zum Einsatz. Er wandelt die gemessene Entfernung in ein PWM-Signal um, welches am Ausgang zur Verfügung steht [KT-12]. Das Auslösen eines Messzyklus geschieht durch eine fallende Flanke am TRIG-Eingang für mindestens 10 μ s. Das Modul sendet darauf nach ca. 250 μ s ein 40 kHz Burst-Signal für die Dauer von 200 μ s aus. Danach geht der ECHO-Ausgang sofort auf HIGH-Pegel und das Modul wartet auf den Empfang des Echos. Wird dieses detektiert, fällt der Ausgang auf LOW-Pegel. Wird kein Echo detektiert, verweilt der Ausgang für insgesamt 200 ms auf HIGH-Pegel und zeigt so eine erfolglose Messung an. Ein vollständiges Messintervall hat eine Dauer von 20 ms, sodass maximal 50 Messungen pro Sekunde durchgeführt werden können [KT-12].

9.3. Spezifikation

Die elektrischen und physikalischen Parameter entstammen den offiziellen Spezifikationen von KT-elektronic [KT-12].

- Betriebsspannung (VCC: +5 V DC $\pm 10\%$)
- Stromaufnahme: < 2 mA
- Messbare Distanz: 2 cm bis ca. 300 cm
- Messauflösung: 3 mm
- Messintervall: 20 ms (maximal 50 Messungen pro Sekunde)

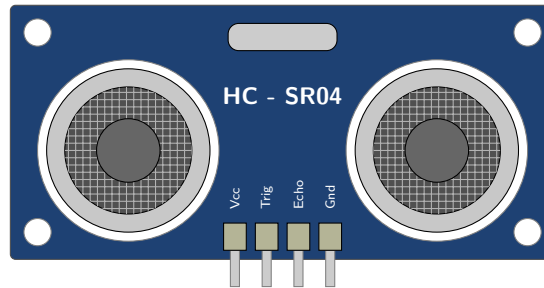


Abbildung 9.1.: Schematische Darstellung des Ultraschallsensors HC-SR04

- Messwinkel: Bis zu 45° bei kurzen Distanzen (< 1 m), Sendekegel von 15° bei maximaler Distanz
- Abmessungen (L x B x T): 45 x 21 x 18 mm

Schaltplan Das Modul besitzt vier Pins (siehe Abbildung 9.1). Der VCC-Pin ist mit der 5V-Spannungsversorgung des Mikrocontrollers verbunden, der GND-Pin (Pin 4) mit dem GND-Pin des Mikrocontrollers. Der TRIG-Eingang (Pin 2) und der ECHO-Ausgang (Pin 3) arbeiten mit TTL-Pegel (siehe Abbildung 9.2). Hinweis bei 3.3V-Mikrocontrollern: Da der ECHO-Pin ein 5V-Signal liefert, muss dieser zwingend durch einen Spannungsteiler an den 3.3V-Eingang des Mikrocontrollers angepasst werden [KT-12].

9.4. Bibliothek

9.4.1. Beschreibung

Für die Ansteuerung des Sensors wird die externe C++ Bibliothek **NewPing** von Tim Eckel verwendet [Eck23].

9.4.2. Installation

Die Bibliothek kann direkt über den internen Bibliotheksverwalter der Arduino IDE installiert werden [Eck23].

- **Data types:** Messzeiten in Mikrosekunden und Distanzen in Zentimetern werden als `unsigned int` deklariert, um Speicherplatz zu sparen und negative Distanzen auszuschließen.
- **Data structure:** Der Sensor wird als Objekt der Klasse `NewPing` instanziiert.
- **Data quantity:** Das Softwaredesign ist ressourcenschonend konzipiert. Es können bis zu 15 Ultraschallsensoren gleichzeitig und asynchron über ein einziges Board betrieben werden, bei einer Abtastrate von bis zu 30 Pings pro Sekunde je Sensor.
- **Data quality:** Die Bibliothek bietet integrierte digitale Filter. Die Funktion `pingmedian` verarbeitet mehrere Messwerte zu einem Median, was Rauschen und fehlerhafte Echos effektiv aus dem Datensatz entfernt.

9.4.3. Funktionen

Die wichtigsten Befehle der NewPing-Bibliothek umfassen [Eck23]:

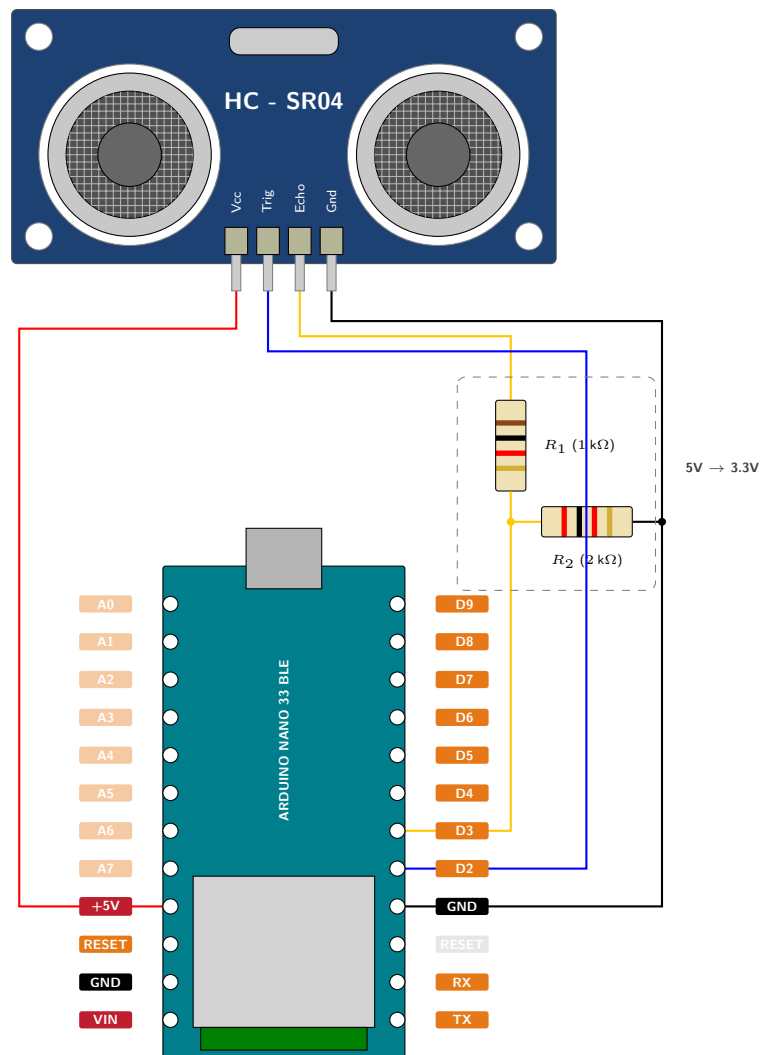


Abbildung 9.2.: Schaltplan Ultraschallsensor HC-SR04

- `NewPing sonar`(`TRIGGER_PIN`, `ECHO_PIN`, `MAX_DISTANCE` 9.1): Konstruktor zur Erstellung des Sensor-Objekts.
 - **TRIGGER_PIN (Der Auslöser):** Dieser Anschluss entspricht Pin 2 auf dem HC-SR04 Modul und arbeitet mit einem regulären TTL-Pegel [KT-12]. Um einen neuen Messzyklus auszulösen, muss an diesem Triggereingang ein Signal mit einer fallenden Flanke für mindestens 10 μ s anliegen [KT-12]. Als direkte Reaktion auf dieses Signal sendet das Modul selbstständig ein 40 kHz Burst-Signal aus [KT-12]. In dem vorliegenden Code-Beispiel wird dieser Anschluss dem digitalen Pin 12 des Arduino zugewiesen.
 - **ECHO_PIN (Der Empfänger):** Dieser Ausgang liegt auf Pin 3 des Sensormoduls und gibt das eigentliche Messergebnis ebenfalls als TTL-Pegel aus [KT-12]. Unmittelbar nach dem Aussenden des Ultraschall-Bursts springt dieser Pin auf einen HIGH-Pegel und wartet auf den Empfang des zurückkehrenden Echos [KT-12]. Sobald das reflektierte Signal detektiert wird, fällt der Ausgang sofort auf einen LOW-Pegel ab [KT-12]. Wird gar kein Echo erkannt (weil der Raum leer ist), verweilt der Pin für exakt 200 ms auf dem HIGH-Pegel, um dem Mikrocontroller die erfolglose Messung zu signalisieren [KT-12]. In der Software ist dieser Pin dem digitalen Pin 11 zugewiesen.
 - **MAX_DISTANCE (Die maximale Reichweite):** Rein physikalisch besitzt der HC-SR04 eine messbare Distanz von 2 cm bis zu ca. 300 cm [KT-12]. Soll die maximale Distanz von 3 Metern erreicht werden, muss der Sensor sehr exakt auf das Objekt zielen, und es dürfen keine störenden Gegenstände im Sendekegel von 15° vorhanden sein [KT-12]. Im Programmcode ist `MAX_DISTANCE` eine festgelegte Konstante (hier auf 200 cm gesetzt), die dem `NewPing`-Objekt mitteilt, bis zu welcher Entfernung der Sensor überhaupt auf Objekte reagieren soll [Eck23].
- `sonar.ping()`: Sendet einen Ping und gibt die gemessene Laufzeit in Mikrosekunden (μ s) zurück.
- `sonar.ping_cm()`: Führt eine Messung durch und gibt die in Zentimeter (cm) umgerechnete Distanz zurück.
- `sonar.ping_median(int iterations)`: Führt die angegebene Anzahl an Pings durch, verwirft Ausreißer und gibt die mediane Laufzeit zurück.

9.5. Beispiel

In diesem Abschnitt wird ein einfaches, eigenständiges Codebeispiel beschrieben, welches die Funktionsweise und Integration der Bibliothek in C++ verdeutlicht.

9.5.1. Handbuch

1. Verbinden Sie VCC und GND des HC-SR04 mit 5V und GND des Boards [KT-12].
2. Verbinden Sie den TRIG-Pin mit Pin 12 und den ECHO-Pin mit Pin 11.
3. Öffnen Sie die Arduino IDE, kompilieren Sie den Code und laden Sie ihn hoch.
4. Öffnen Sie den Seriellen Monitor (Baudrate 115200). Hier werden nun die Messergebnisse ausgegeben[Ele20].

9.5.2. Funktion vom Beispiel

Das Skript bindet die `NewPing.h` ein. Über `#define` werden Pins und der maximale Messbereich (200 cm) definiert. In der Hauptschleife (`loop()`) stellt `delay(50)` sicher, dass zwischen zwei Pings gewartet wird, um Echosüberlappungen zu verhindern [Eck23]. Anschließend wird `ping_median(5)` aufgerufen. Ist ein Objekt vorhanden, muss die gemessene Zeit in eine Längeneinheit umgewandelt werden. Hierfür kommt der bibliotheksinterne Befehl `US_ROUNDTRIP_CM` zum Einsatz. Dies ist ein vordefinierter Teilerfaktor, mit dem Wert 57. Er basiert auf der physikalischen Schallgeschwindigkeit in der Luft. Da der Schall für einen Zentimeter Wegstrecke etwa 29 Mikrosekunden benötigt und das Signal den Weg doppelt zurücklegen muss (Hin- und Rückweg), entspricht 1 cm Objektabstand exakt einer Signallaufzeit von ca. 57 bis 58 Mikrosekunden. Durch die Division `uS / US_ROUNDTRIP_CM` wird die gemessene Zeit somit effizient und direkt in den korrekten Zentimeter-Abstand umgerechnet [Eck23].

9.5.3. Code

Listing 9.1.: Beispiel Code

```

1000 \begin{verbatim}
// File: Beispielcode.ino
1002 // Standalone script for filtered distance measurement with HC-SR04.
// Uses NewPing library to get a median value from multiple measurements
// to reduce noise.
1004 // Author: Wilko Hinrichs
// Date: 2026-04-27
1006
#include <NewPing.h>
1008
// Digital pin for trigger signal
1010 #define TRIGGER_PIN 12
1012
// Digital pin for echo signal
#define ECHO_PIN 11
1014
// Maximum distance for the sensor (in cm)
1016 #define MAX_DISTANCE 200
1018
// Initialize NewPing object
NewPing sonar(TRIGGER_PIN, ECHO_PIN, MAX_DISTANCE);
1020
void setup() {
1022   Serial.begin(115200);
   Serial.println("System gestartet. Beginne Messung...");
1024 }
1026
void loop() {
// Wait between pings (min. 29ms recommended)
1028   delay(50);
1030
// Get median of 5 measurements (in microseconds)
unsigned int uS = sonar.ping_median(5);
1032
   Serial.print("Gefilterte Distanz: ");
1034
   if (uS == 0) {
1036     Serial.println("Außerhalb der Reichweite");
   } else {
// Convert microseconds to centimeters
1038     Serial.print(uS / US_ROUNDTRIP_CM);
1040     Serial.println(" cm");
   }
1042 }
\end{verbatim}

```

../Code/Ultraschallsensor/Beispielcode/Beispielcode.ino

9.6. Kalibrierung

Die Ultraschallmessung beruht auf der Ausbreitungsgeschwindigkeit von Schall in der Luft. Um das System zu überprüfen wird ein Objekt in einem exakt gemessenem Abstand vor dem Sensor aufgestellt und mit der Ausgabe im seriellen Monitor verglichen. Die Standardberechnung der Bibliothek geht von einer konstanten Schallgeschwindigkeit von $c \approx 343 \text{ m/s}$ bei 20°C aus [Fra16]. Da die Schallgeschwindigkeit temperaturabhängig ist, ändert sie sich um ca. $0,6 \text{ m/s}$ pro Grad Celsius [Fra16]. Für hochpräzise Anwendungen muss das System kalibriert werden (z.B. über einen DHT22-Sensor). Die dynamische Berechnung lautet:

$$\text{Distanz} = \frac{\text{Laufzeit}}{2} \times (331,3 + 0,606 \times \text{Temperatur in } ^\circ\text{C})$$

9.7. Einfacher Code ohne newPing Bibliothek

Der folgende Code demonstriert eine Abstandsmessung ohne Einbindung der NewPing-Bibliothek und nutzt `pulseIn()`.

Listing 9.2.: Beispiel Code ohne Bibliothekseinbindung

```

1000 \begin{verbatim}
// File: Beispielcode2.ino
1002 // Standalone script for filtered distance measurement with HC-SR04.
// Uses NewPing library to get a median value from multiple measurements
// to reduce noise.
1004 // Author: Wilko Hinrichs
// Date: 2026-04-27
1006
const int TRIG_PIN = 12; // Pin for trigger signal
1008 const int ECHO_PIN = 11; // Pin for echo signal
1010
void setup() {
  pinMode(TRIG_PIN, OUTPUT);
1012  pinMode(ECHO_PIN, INPUT);
  Serial.begin(115200);
1014 }
1016
void loop() {
  digitalWrite(TRIG_PIN, LOW);
1018  delayMicroseconds(2);

  // 10 microseconds HIGH pulse (ends with falling edge)
  digitalWrite(TRIG_PIN, HIGH);
1022  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
1024

  // pulseIn measures the length of the ECHO pulse
1026  long duration = pulseIn(ECHO_PIN, HIGH);

  // Calculate distance (travel time in us / 58)
1028  float distance = duration / 58.0;
1030
  Serial.print("Gemessene Distanz: ");
1032  Serial.print(distance);
  Serial.println(" cm");

```

```

1034 // Keep interval (> 20ms)
1036 delay(50);
1038 }
\end{verbatim}

```

../Code/Ultraschallsensor/Beispielcode2/Beispielcode2.ino

9.8. Einfache Anwendung

Eine anschauliche Anwendung für den HC-SR04 ist eine automatisierte Einparkhilfe. Fährt ein Fahrzeug heran, liest der Sensor die Distanz. Unterschreitet der distance-Wert 50 cm, schaltet das System eine Warn-LED ein. Fällt der Wert auf unter 10 cm, wird ein Buzzer getriggert [Eck23].

Listing 9.3.: Einparkhilfe Beispielcode

```

1000 \begin{verbatim}
// File: EinparkhilfeCode.ino
1002 // Application example: Automated parking assistant.
// System shows distance via LED and warns acoustically on critical
// approach.
1004 // Author: Wilko Hinrichs
// Date: 2026-04-28
1006 #include <NewPing.h>
1008 // Pin configuration
1010 #define TRIGGER_PIN 12
1012 #define ECHO_PIN 11 // Maximum scan range in cm
1014 #define MAX_DISTANCE 200 // Yellow warning LED
1016 #define LED_PIN 3 // Acoustic buzzer
1018 #define BUZZER_PIN 4
1020 // Initialize sensor
NewPing sonar(TRIGGER_PIN, ECHO_PIN, MAX_DISTANCE);
1022 void setup() {
1024 Serial.begin(115200);
1026 pinMode(LED_PIN, OUTPUT);
1028 pinMode(BUZZER_PIN, OUTPUT);
1030 }
1032 void loop() {
1034 // Keep measurement interval
delay(50);
1036 // Measure distance in cm
1038 unsigned int distance = sonar.ping_cm();
1040 Serial.print("Abstand: ");
1042 Serial.print(distance);
1044 Serial.println(" cm");
1046 // Logic check based on distance thresholds
1048 if (distance > 0 && distance < 10) {
1050 // Critical area: LED and buzzer active
1052 digitalWrite(LED_PIN, HIGH);
1054 digitalWrite(BUZZER_PIN, HIGH);
1056 }
1058 else if (distance >= 10 && distance < 50) {
1060 // Warning area: Only LED active
1062 digitalWrite(LED_PIN, HIGH);
1064 digitalWrite(BUZZER_PIN, LOW);
1066 }
1068 }
1070 \end{verbatim}

```

```

1046   }
1048   else {
1049       // Safe area: All off
1050       digitalWrite(LED_PIN, LOW);
1051       digitalWrite(BUZZER_PIN, LOW);
1052   }
}
\end{verbatim}

```

../Code/Ultraschallsensor/EinparkhilfeCode/EinparkhilfeCode.ino

9.9. Tests

9.9.1. Einfacher Funktionstest

Für die Überprüfung der grundlegenden Schaltung und Sensorik wird das Standalone-Skript aus Abschnitt 9.1 (Gefilterte Distanzmessung mit der NewPing-Bibliothek) auf den Mikrocontroller geladen.

Ein harter, planer Gegenstand (z. B. ein Buch) wird in exakt 15 cm Entfernung parallel zur Sensorfront platziert. Anschließend wird der Serielle Monitor der Arduino IDE mit einer Baudrate von 115200 geöffnet. Der ausgegebene Wert muss durch den Aufruf der Funktion `sonar.ping_median(5)` sehr stabil bei 15 cm liegen. Die Schwankung darf dabei die gerätespezifische Auflösung des Moduls von 3 mm [KT-12] kaum überschreiten. Das Skript bestätigt somit, dass das Sende- und Empfangsmodul korrekt arbeiten und die serielle Datenübertragung fehlerfrei funktioniert.

9.9.2. Alle Funktionen testen

Ein vollständiger Systemtest kann wie folgt durchgeführt werden. Hierfür können die Ausgaben des Seriellen Monitors aus dem vorherigen Testcode 9.1 oder das Verhalten der LEDs aus dem Parkassistenten-Code 9.3 beobachtet werden:

- **Minimaldistanz:** Das Objekt wird auf unter 2 cm an den Sensor herangeführt. Da der Sensor nach dem Senden des 200 µs langen Bursts eine kurze Umschaltzeit benötigt [KT-12], kann er Echos im absoluten Nahbereich physikalisch nicht mehr auflösen. Der Monitor sollte hier durch die Bibliothek korrekterweise Außerhalb der Reichweite (Rückgabewert 0) ausgeben [Eck23].
- **Maximaldistanz:** Der Sensor wird in einen leeren, großen Raum gerichtet, sodass das nächste Hindernis mehr als 300 cm entfernt ist. Laut Spezifikation verweilt der ECHO-Pin bei einem fehlenden Echo für 200 ms auf einem HIGH-Pegel [KT-12]. Der Test belegt, dass die NewPing-Bibliothek diesen Hardware-Timeout sicher abfängt, ohne dass sich der Mikrocontroller in einer blockierenden Endlosschleife aufhängt [Eck23].
- **Winkeltest (Abstrahlkegel):** Ein Objekt wird in 50 cm Entfernung positioniert und langsam seitlich aus dem Zentrum bewegt. Überschreitet der Winkel 15° zur Mittelachse, wird die Fläche vom Objekt für die Ultraschallwellen zu klein [KT-12]. Die Messwerte brechen ab, was die seitlichen Grenzen des Erfassungsbereichs bestätigt.
- **Dynamischer Reaktionstest:** Mit dem Code des automatisierten Parkassistenten aus Abschnitt 9.3 wird die Reaktionsgeschwindigkeit der Gesamtanlage getestet. Ein Objekt wird zügig auf den Sensor zubewegt. Die gelbe Warn-LED muss exakt beim Unterschreiten der 50-cm-Marke aufleuchten, und der Buzzer muss ohne spürbare Software-Verzögerung bei exakt 10 cm triggern.

9.10. Weiterführende Literatur

SparkFun Electronics / ElecFreaks (2019): HC-SR04 Ultrasonic Sensor Datasheet, Verfügbar unter: <https://www.sparkfun.com/datasheets/Sensors/Ultrasonic/HCSR04.pdf> [abgerufen am 28.04.2026]

- Diese technische Spezifikation bildet die fundamentale Grundlage für die Ansteuerung. Sie enthält die exakten Timing-Diagramme für das Trigger- und Echo-Signal sowie Angaben zum Öffnungswinkel des Schallkegels, die für die Gehäuseplanung wichtig sind.

Eckel, Tim (2023): Arduino NewPing Library Documentation, GitHub/Bitbucket Repository, Verfügbar unter: <https://bitbucket.org/teckel12/arduino-newping/wiki/Home> [abgerufen am 28.04.2026]

- Eine umfassende Dokumentation zur verwendeten Programmbibliothek NewPing Library von Tim Eckel. Es werden die Vorteile gegenüber herkömmlichen Messmethoden erläutert und die Implementierung von Filtern wie der `ping_median()`-Funktion zur Rauschunterdrückung detailliert beschrieben.

Ekbert Hering und Gert Schönfelder, Sensoren in Wissenschaft und Technik Funktionsweise und Einsatzgebiete, Springer Vieweg, 2023, ISBN: 9783658394905 [HS23]

- Eine umfassende und aktuelle Einführung in die Grundlagen und Anwendungen von Sensoren in Technik, Biologie und Medizin. Das Buch behandelt neben physikalischen Grundlagen auch Themen wie Schnittstellen, Sicherheit und Signalverarbeitung bei Ultraschallwandlern.

10. SG90 Servomotor

Einleitung

Dieses Kapitel beschreibt den mechanischen Aktor des Systems, den Micro-Servomotor TowerPro SG90. Es werden die technischen Spezifikationen, die hardwareseitige Einbindung sowie die softwareseitige Ansteuerung über die Standard-Bibliotheken detailliert erläutert.

10.1. Allgemeines

Ein Servomotor ist ein geschlossenes mechatronisches System (Closed-Loop), das für präzise Winkelpositionierungen eingesetzt wird. Im Gegensatz zu kontinuierlich drehenden Gleichstrommotoren besteht ein Standard-Servo aus einem DC-Motor, einem Untersetzungsgetriebe, einem Potentiometer zur Positionserfassung und einer Steuerelektronik. Der Mikrocontroller sendet ein Pulsweitenmodulations-Signal (*PWM*), welches die Steuerelektronik auswertet und den Motor exakt auf den gewünschten Winkel dreht und dort aktiv hält. Servomotoren findet man vor allem im RC-Modellbau (z.B. Lenkung, Ruder, Klappen), in Robotergelenken sowie in Schwenkmechanismen für Kameras oder Sensoren, bei denen wiederholgenaue und stabile Positionen erforderlich sind.[Mik20]

10.2. Servomotor TowerPro SG90

Der TowerPro SG90 ist einer der weltweit am häufigsten verwendeten Micro-Servomotoren für kleine Projekte. Er zeichnet sich durch sein extrem geringes Gewicht, seine kompakte Bauform und sein Nylon-Zahnradgetriebe aus. Der Aktor verfügt über ein dreipoliges Anschlusskabel: Braun für die Masse (*GND*), Rot für die Versorgungsspannung (*VCC*) und Orange für das PWM-Steuersignal (*Signal*). Der TowerPro SG90 hat einen mechanischen Rotationsspielraum von 180 Grad (0° bis 180°). Gesteuert wird er über ein 50-Hz-PWM-Signal (20 ms Periodendauer). Eine Pulsweite von 1 ms entspricht theoretisch 0°, 1,5 ms entsprechen 90° (Mittelstellung) und 2 ms entsprechen 180°.[Pat; Com17]

10.3. Spezifikation

- **Quellen referenzieren:** Die physikalischen und elektrischen Kennwerte beziehen sich auf technische Angaben zum SG90 von Elektronik-Kompendium und Components101.[Pat; Com17]
- **Extrahierte Informationen:**
 - Betriebsspannung: 4,8 V (ca. 5 V)
 - Stellmoment (Stall Torque): 1,8 kg/cm (bei 4,8 V)
 - Stellgeschwindigkeit (ohne Last): 0,1 Sekunden / 60 Grad (bei 4,8 V)
 - Gewicht: 9 Gramm
 - Getriebetyp: Kunststoff (Nylon)

- Steuersignal: PWM (50 Hz / 20 ms Periode)
- Dimensionen: ca. 22,2 x 11,8 x 31 mm

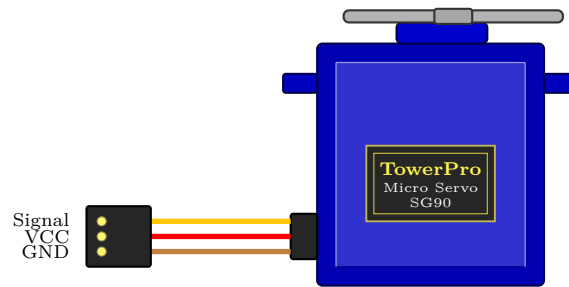


Abbildung 10.1.: Schematische Darstellung des Servomotors TowerPro SG90

- **Schaltplan:** Das rote Kabel (*VCC*) des SG90 wird mit dem 5V-Ausgang (*VUSB*) des Systems verbunden, um ausreichend Leistung für den Motor bereitzustellen. Das braune Kabel (*GND*) wird mit der gemeinsamen Masse verbunden. Das orange Kabel (*Signal*) wird an einen digitalen PWM-fähigen Pin (z.B. Pin 9) des Mikrocontrollers angeschlossen.[Pat]

Hinweis: Der Arduino Nano 33 BLE Sense arbeitet mit 3,3V-I/O-Pegeln und ist nicht 5V-tolerant.[Ard26a] Viele Servomotoren können jedoch ein 3,3V-PWM-Steuersignal verarbeiten, sofern der Servo selbst mit einer geeigneten Versorgungsspannung betrieben wird und eine gemeinsame Masse vorhanden ist.[Dr]

10.4. Bibliothek

10.4.1. Beschreibung

Für die Ansteuerung des SG90 wird in der Arduino-Umgebung die integrierte *Servo*-Bibliothek genutzt[Lib25a]. Dadurch entfällt für den Programmierer die aufwändige Erzeugung des exakten 50-Hz-PWM-Signals, und der Motor kann bequem über die direkte Vorgabe eines Winkelwertes von 0 bis 180 Grad angesteuert werden.[Lib25a]

10.4.2. Installation

- **Installation:** Die Bibliothek kann in der Arduino IDE über den Bibliotheksverwalter eingebunden werden. Im Programm wird sie anschließend mit `#include <Servo.h>` importiert.[Mik20]
- **Data types:** Die Zielwinkel werden als `int` (Integer/Ganzzahlen) übergeben. Für präzisere Steuerungen können Pulsweiten in Mikrosekunden ebenfalls als `int` übergeben werden (vgl. [Lib25b]).
- **Data structure:** Jeder Servomotor wird im Code als Objekt der Klasse *Servo* deklariert (z.B. `meinServo;`).
- **Data quantity:** Auf Standard Arduino-Boards unterstützt die Bibliothek bis zu 12 Servos gleichzeitig; auf speziellen Architekturen (wie der Mbed-OS Architektur des Nano 33 BLE) kann dies variieren, reicht jedoch für mehrere Aktoren vollkommen aus.[Doc26]
- **Data quality:** Die Bibliothek arbeitet timer- und interruptbasiert. Die Servosignale werden im Hintergrund mit dem zuletzt über `write()` gesetzten Wert erzeugt, wodurch keine manuelle PWM-Erzeugung mit blockierenden Delay-Schleifen notwendig ist.[Lib25b]

10.4.3. Funktionen

Die wichtigsten Methoden der Bibliothek sind:

- `attach(pin)`: Verbindet das Servo-Objekt mit einem bestimmten digitalen Pin.[Ard26b]
- `write(angle)`: Dreht den Servo auf den entsprechenden Winkel (0° bis 180°).
- `writeMicroseconds(uS)`: Ermöglicht die Ansteuerung über Pulsweiten in Mikrosekunden.[Ard26d]
- `read()`: Gibt den zuletzt gesetzten Servo-Winkel zurück.[Ard26c]

10.5. Beispiel

Dieser Abschnitt zeigt ein elementares Beispiel „Sweep“, bei dem der Servo kontinuierlich von 0 auf 180 Grad und wieder zurück schwenkt.

10.5.1. Handbuch

Schließen Sie das rote Kabel an 5V, das braune an GND und das orange Signalkabel an Pin 9 des Arduinos an. Kompilieren Sie das Programm und laden Sie es auf das Board. Der Arm des Servomotors sollte nun anfangen, langsam und gleichmässig hin und her zu wischen.

10.5.2. An einem Beispiel

Das Skript in 10.1 bindet zunächst die Bibliothek `Servo.h` ein und erstellt mit `radarServo` ein Servo-Objekt. Zusätzlich wird die Variable `pos` zur Speicherung der aktuellen Winkelposition verwendet. In der `setup()`-Routine wird der Motor über `radarServo.attach(9)` an den digitalen Pin 9 gebunden.

In der `loop()`-Schleife sorgen zwei `for`-Schleifen für die kontinuierliche Schwenkbewegung des Servomotors. Die erste Schleife erhöht den Winkel schrittweise von 0° auf 180°. Dabei wird mit `radarServo.write(pos)` jeweils eine absolute Zielposition angefahren. Nach jeder Winkeländerung folgt mit `delay(15)` eine kurze Pause von 15 ms, damit der Motor die physikalische Position erreichen kann. Die zweite Schleife führt anschließend die Rückwärtsbewegung von 180° zurück auf 0° aus. Dadurch entsteht eine fortlaufende Hin- und Herbewegung des Servomotors.

10.5.3. Code

Listing 10.1.: Beispiel Code für den TowerPro SG90

```

1000 /**
1001  * @file    Servomotor.ino
1002  * @author  Simon Müller
1003  * @date    2026-05-01
1004  * @version 1.0
1005  *
1006  * @brief   Servo-based radar scanning system.
1007  *
1008  * This sketch controls a TowerPro SG90 (or similar) servo to perform
1009  * a continuous back-and-forth sweep from 0 deg to 180 deg and back.
1010  * All position commands are absolute angles
1011  * (0180 deg range).
1012  */
1013
1014 #include <Servo.h>

```

```

1016 Servo radarServo;    ///< Servo object used for radar scanning
1017 int pos = 0;         ///< Current servo position in degrees (0180,
1018                       absolute)
1019
1020 /**
1021  * @brief Initializes the servo on startup.
1022  *
1023  * Attaches the servo object to digital pin 9 and prepares it
1024  * for subsequent position commands. The servo will interpret
1025  * all following write() calls as absolute target angles in
1026  * the range 0°180°.
1027  */
1028 void setup() {
1029     radarServo.attach(9); // Attach servo to pin 9
1030 }
1031
1032 /**
1033  * @brief Main loop for continuous servo motion.
1034  *
1035  * The servo is moved in two phases using absolute angle commands:
1036  * - Phase 1: Sweep from 0° to 180° in 1° steps
1037  * - Phase 2: Sweep from 180° back to 0° in 1° steps
1038  *
1039  * Each call to radarServo::write(pos) sets an absolute target
1040  * angle; it does NOT move the servo by a relative offset.
1041  * With a 15 ms delay between steps, one complete cycle
1042  * takes approximately 5.4 seconds.
1043  */
1044 void loop() {
1045     // Phase 1: sweep from 0° to 180° (absolute positions)
1046     for (pos = 0; pos <= 180; pos += 1) {
1047         radarServo.write(pos); ///< Set servo to absolute angle 'pos'
1048         delay(15);             ///< Wait 15 ms to reach the position
1049     }
1050
1051     // Phase 2: sweep from 180° back to 0° (absolute positions)
1052     for (pos = 180; pos >= 0; pos -= 1) {
1053         radarServo.write(pos); ///< Set servo to absolute angle 'pos'
1054         delay(15);             ///< Wait 15 ms
1055     }
1056 }

```

../Code/Servomotor/Servomotor.ino

10.5.4. Dateien

Für das Ausführen des Codes wird ausschließlich die Standard-Headerdatei `Servo.h` benötigt, welche in der Arduino IDE vorinstalliert ist.

10.6. Kalibrierung

Bei Servomotoren können die praktisch nutzbaren Endpositionen von den theoretischen PWM-Werten abweichen. Wird der Motor mit zu kleinen oder zu großen Pulsweiten an seine mechanischen Grenzen gefahren, kann dies zu Brummen, Vibrationen oder mechanischer Belastung führen[Ard26d]. Zur Kalibrierung bietet die `attach()`-Funktion optionale Parameter für die minimalen und maximalen Mikrosekunden: `radarServo.attach(9, 500, 2400)`. Diese Werte können durch iteratives Ausprobieren mit `writeMicroseconds()` an den individuellen Motor angepasst werden, bis der gewünschte Winkelbereich ohne mechanisches Blockieren erreicht ist.

10.7. Einfacher Code (ohne Bibliothek)

Der Code in 10.2 erzeugt das PWM-Signal für den Servo manuell ohne die Verwendung der Servo-Bibliothek. In der `setup()`-Routine wird Pin 9 mit `pinMode(9, OUTPUT)` als Ausgang konfiguriert. Die `loop()`-Funktion generiert anschließend ein 50-Hz-Signal. Dazu wird der Pin mit `digitalWrite(9, HIGH)` für 1500 µs auf HIGH gesetzt, was ungefähr einer Servo-Position von 90° entspricht. Danach wird der Pin mit `digitalWrite(9, LOW)` für 18500 µs auf LOW gesetzt, sodass eine vollständige Periodendauer von 20 ms entsteht.

Diese Methode zeigt vereinfacht, wie die Ansteuerung eines Servomotors über ein PWM-Signal funktioniert. Da der Ablauf jedoch durch die verwendeten Verzögerungen blockierend ist, eignet sich diese Variante nur eingeschränkt für komplexere Anwendungen. In solchen Fällen sollte die Servo-Bibliothek verwendet werden, da diese mit Timern arbeitet und die Ansteuerung komfortabler übernimmt.

Listing 10.2.: Code ohne Bibliothek für den TowerPro SG90

```

1000 /**
1001  * @file ServoManual.ino
1002  * @brief Manual servo control without library
1003  *
1004  * @details Demonstrates servo motor control through direct PWM
1005  *          generation
1006  * using digitalWrite() and delayMicroseconds(). The servo is positioned
1007  * at approximately 90° (center position). This method shows how the
1008  * Servo library works at the hardware level.
1009  *
1010  * @author Simon Müller
1011  * @date 01.05.2026
1012  * @version 1.0
1013  *
1014  * @section hardware Hardware
1015  * – Arduino Board (e.g. Uno, Nano)
1016  * – Servo motor connected to Pin 9
1017  * – Power supply: 5V for servo (VCC and GND)
1018  *
1019  * @section notes Notes
1020  * This implementation uses blocking delays and is for educational
1021  * purposes.
1022  * For production code, use the Arduino Servo library instead.
1023  */
1024
1025 int servoPin = 9; ///< Pin number for servo control signal
1026
1027 /**
1028  * @brief Initialization at program startup
1029  *
1030  * @details Configures the servo pin as OUTPUT to enable
1031  * sending PWM signals to the servo motor.
1032  *
1033  * @return void
1034  */
1035 void setup() {
1036     pinMode(servoPin, OUTPUT); // Set servo pin as output
1037 }
1038
1039 /**
1040  * @brief Main loop for continuous PWM generation
1041  *
1042  * @details Manually generates a 50 Hz PWM signal (20 ms period).
1043  * The HIGH pulse of 1.5 ms corresponds to a servo position of approx.
1044  * 90°.
1045  *
1046  * PWM Timing:
1047  * – HIGH phase: 1500 µs (1.5 ms) 90° position (center)

```

```

1046 * - LOW phase: 18500  $\mu$ s (18.5 ms) fills up to 20 ms total period
1047 * - Frequency: 50 Hz (1000000  $\mu$ s / 20000  $\mu$ s = 50 Hz)
1048 *
1049 * @note This method blocks program execution. For more complex
1050 * applications, the Servo library should be used.
1051 *
1052 * @return void
1053 */
1054 void loop() {
1055     // Move to approx. 90 degrees (1.5ms HIGH pulse)
1056     digitalWrite(servoPin, HIGH);    ///< Starts the HIGH pulse
1057     delayMicroseconds(1500);          ///< Pulse width: 1.5 ms 90°
1058     position
1059     digitalWrite(servoPin, LOW);      ///< Ends the HIGH pulse
1060
1061     // Fill the remaining 20ms period (50 Hz)
1062     delayMicroseconds(18500);         ///< LOW phase: 18.5 ms until next
1063     period
1064 }

```

../Code/Servomotor/ServomotorohneBib.ino

10.8. Einfache Anwendung

Eine klassische Anwendung für den SG90 ist die Basis eines **Ultraschall-Radarsystems**. Der Ultraschallsensor HC-SR04 wird auf der Achse des Servos montiert. Der Servo tastet systematisch in 1-Grad-Schritten die Umgebung vor sich ab (0° bis 180°). Bei jedem Schritt wird eine Entfernungsmessung durchgeführt. Diese Kombination aus Aktor-Winkel und Sensor-Distanz ermöglicht es der Software (z. B. in Processing), eine zweidimensionale „Radarkante“ der Umgebung auf dem Bildschirm zu zeichnen.[die25]

10.9. Tests

10.9.1. Einfacher Funktionstest

Mit dem „Sweep“-Code aus Abschnitt 10.2 lässt sich prüfen, ob der Motor korrekt verkabelt ist. Der Motor sollte möglichst ruckelfrei und gleichmässig schwenken. Ein stotternder Motor oder ein Neustart des Arduinos während der Bewegung kann auf eine unzureichende Stromversorgung, beispielsweise einen VCC-Abfall, hinweisen.

10.9.2. Alle Funktionen testen

Ein vollständiger Systemtest umfasst:

1. **Grenzttest:** Anfahren von 0° und 180°. Der Motor darf in diesen Endlagen nicht surren oder ungewöhnlich warm werden.[Ard26d]
2. **Geschwindigkeitstest:** Befehle `write(0)` und sofort danach `write(180)` ohne Delay. Messung, ob der Motor die physikalische Herstellerangabe von 0,3 Sekunden für diesen 180°-Weg einhalten kann.[Com17]
3. **Präzisionstest:** Verwendung von `writeMicroseconds(1500)` zur exakten Zentrierung, gefolgt von der Überprüfung der Winkelgenauigkeit mittels eines Geodreiecks auf dem Rotorkopf.[Ard26d]

10.10. Weiterführende Literatur

Teil IV.

Domain Knowledge - Tools

11. Mikrocontroller-Programmierung mit der Arduino IDE

11.1. Allgemeines zur Arduino IDE

Die Arduino Integrated Development Environment (IDE) ist eine plattformunabhängige Open-Source-Entwicklungsumgebung, die speziell für die Programmierung von Mikrocontrollern entworfen wurde. Sie basiert im Kern auf der Programmiersprache C/C++, verbirgt jedoch viele komplexe hardwarenahe Konfigurationen hinter einer stark vereinfachten Syntax und einer umfangreichen Standardbibliothek.

Ein klassisches Arduino-Programm (auch „Sketch“ genannt) ist strukturell immer in zwei wesentliche Hauptfunktionen unterteilt: Die `setup()`-Funktion, welche genau einmal beim Systemstart ausgeführt wird und für die Initialisierung von Pins und Schnittstellen zuständig ist, sowie die `loop()`-Funktion, welche im Anschluss als Endlosschleife die eigentliche Arbeitslogik des Mikrocontrollers abarbeitet. Die fertigen Programme werden direkt aus der IDE heraus kompiliert und über eine serielle USB-Verbindung auf den Flash-Speicher des Mikrocontrollers übertragen.

11.2. Das Steuerungsprogramm des Ultraschallradars

Der Quellcode für den Arduino Nano 33 BLE übernimmt die direkte Hardwaresteuerung der Sensorik, der Aktuatorik sowie der visuellen Statusanzeigen (LEDs). Bevor das System in den regulären Betriebsmodus wechselt, durchläuft es in der `setup()`-Phase einen sogenannten Power-On Self-Test (POST). Hierbei teilt der Mikrocontroller der verbundenen Processing-Software zunächst den Status `S:TESTING` mit. Daraufhin werden die LEDs auf Funktion geprüft, der Servomotor absolviert eine Testdrehung und der Ultraschallsensor führt eine initiale Referenzmessung durch.

Nur wenn diese Testmessung valide Werte (zwischen 1 cm und 400 cm) liefert, wird das System freigegeben, die grüne Status-LED aktiviert und das Signal `S:OK` an den Computer gesendet. Tritt ein Fehler auf, beispielsweise durch einen Verbindungsabbruch zum Sensor (Timeout), leuchtet die rote LED auf, das System blockiert aus Sicherheitsgründen den weiteren Betrieb und meldet `S:ERR_SENSOR`.

Im laufenden und fehlerfreien Betrieb (`loop`) iteriert das System kontinuierlich durch die Positionen des Servomotors (0° bis 180° und zurück). Nach jedem Positionsschritt pausiert das System kurz, um die mechanische Trägheit des Motors auszugleichen. Anschließend feuert der HC-SR04-Sensor einen 10 µs langen Ultraschall-Impuls ab und misst die Zeit bis zum Eintreffen des Echos. Aus dieser Laufzeit errechnet der Mikrocontroller unter Einbeziehung der Schallgeschwindigkeit die genaue Distanz.

Abschließend formatiert der Arduino diese Wertepaare in das speziell für dieses Projekt definierte Kommunikationsprotokoll (`D:Winkel,Distanz`) und übermittelt sie in Echtzeit an die serielle Schnittstelle zur weiteren grafischen Auswertung.

11.2.1. Quellcode der Arduino-Steuerung

Listing 11.1: Arduino Steuerungscode mit Doxygen-Dokumentation

```

1  /**
2   * @file UltraschallradarArduinoIDeMitDoxygen.ino
3   * @brief Firmware für ein automatisches 120-Grad-Ultraschallradar.
4   * @details Diese Software steuert ein Ultraschallradar, bestehend aus einem
5   * Arduino Nano V3, einem HC-SR04 Ultraschallsensor und einem SG90 Positions-Servomotor.
6   * Beim Systemstart wird eine Zentrierung auf 90° durchgeführt und ein visueller
7   * Power-On Self-Test (POST) über zwei Status-LEDs signalisiert. Im laufenden Betrieb
8   * wird ein kontinuierlicher Schwenk von 30° bis 150° ausgeführt, die gemessene
9   * Distanz ermittelt und über das serielle Protokoll an das Processing-Frontend gesendet.
10  * * @author Wilko Hinrichs
11  * @date 2026-06-08
12  * @version 1.1
13  */
14
15  #include <Servo.h>
16
17  /* --- PIN-DEFINITIONEN --- */
18  const int PIN_LED_GREEN = 2;    ///< GPIO-Pin für die grüne Status-LED (System betriebsbereit)
19  const int PIN_LED_RED = 3;      ///< GPIO-Pin für die rote Status-LED (Fehler / Selbsttest)
20  const int PIN_TRIG = 4;         ///< Digitaler Ausgangs-Pin zur Initiierung des Ultraschall-Impulses (Trig)
21  const int PIN_ECHO = 5;         ///< Digitaler Eingangs-Pin zur Messung der Schall-Laufzeit (Echo)
22  const int PIN_SERVO = 9;        ///< PWM-Ausgangs-Pin zur Winkelsteuerung des SG90 Servomotors
23
24  /* --- GLOBALE OBJEKTE --- */
25  Servo radarServo;               ///< Servo-Instanz der offiziellen Arduino-Bibliothek zur Achsensteuerung.
26
27  /**
28   * @brief Initialisiert die Hardware-Peripherie und führt den Boot-Selbsttest aus.
29   * @details Konfiguriert die seriellen Kommunikationsparameter auf 9600 Baud, definiert
30   * die Pin-Modi (INPUT/OUTPUT) und leitet den Selbsttest ein, bei dem zunächst beide LEDs leuchten. Der Se
31   * erfolgreiche Testmessung im gültigen physikalischen Bereich (0 - 400 cm) registriert,
32   * schaltet das System die rote LED aus und startet den Normalbetrieb. Andernfalls blockiert
33   * das System im Fehlermodus.
34   */
35  void setup() {
36      // Serielle Schnittstelle mit stabiler Standard-Baudrate initialisieren
37      Serial.begin(9600);
38
39      // Definition der Signalrichtungen
40      pinMode(PIN_LED_GREEN, OUTPUT);
41      pinMode(PIN_LED_RED, OUTPUT);
42      pinMode(PIN_TRIG, OUTPUT);
43      pinMode(PIN_ECHO, INPUT);
44
45      // SELBSTTEST-BEGINN: Beide LEDs signalisieren den Testzustand
46      Serial.println("S:TESTING");
47      digitalWrite(PIN_LED_RED, HIGH);
48      digitalWrite(PIN_LED_GREEN, HIGH);
49
50      // ZENTRIERUNG: Servo fährt in die exakte Mitte (90 Grad) für die Gehäusemontage
51      radarServo.attach(PIN_SERVO);
52      radarServo.write(90);
53      delay(1500); // Ausreichend Zeit, um den Sensorkopf mechanisch auszurichten
54
55      // Ultraschall-Testmessung triggern (10 Mikrosekunden High-Impuls)
56      digitalWrite(PIN_TRIG, LOW);
57      delayMicroseconds(2);
58      digitalWrite(PIN_TRIG, HIGH);
59      delayMicroseconds(10);
60      digitalWrite(PIN_TRIG, LOW);
61
62      // Messung der Echo-Rücklaufzeit mit einem Timeout von 30ms (ca. 5 Meter Reichweite)
63      long duration = pulseIn(PIN_ECHO, HIGH, 30000);
64      float distance = duration * 0.0343 / 2.0;

```



```

65
66 // Validierung des Sensorsignals
67 if (distance > 0 && distance < 400) {
68     // SELBSTTEST ERFOLGREICH: Rot aus, Grün bleibt an -> Normalbetrieb
69     digitalWrite(PIN_LED_RED, LOW);
70     digitalWrite(PIN_LED_GREEN, HIGH);
71     Serial.println("S:OK");
72 } else {
73     // SELBSTTEST FEHLGESCHLAGEN: Grün aus, Rot bleibt an -> System sperren
74     digitalWrite(PIN_LED_GREEN, LOW);
75     digitalWrite(PIN_LED_RED, HIGH);
76     Serial.println("S:ERR_SENSOR");
77     while(1); // Endlosschleife verhindert die Ausführung der loop() bei Defekt
78 }
79 }
80
81 /**
82  * @brief Kontinuierliche Hauptschleife für die periodische Schwenkbewegung.
83  * @details Realisiert die Schwenkbewegung des Radars symmetrisch um die Mittelachse (90°).
84  * Der gesamte Scanbereich beträgt exakt 120 Grad und wird in drei Phasen durchlaufen:
85  * 1. Schwenk von der Mitte (90°) nach ganz links zur Grenze (30°).
86  * 2. Kompletter Schwenk von ganz links (30°) nach ganz rechts zur Grenze (150°).
87  * 3. Rückschwenk von ganz rechts (150°) zurück in die Ausgangsmitte (90°).
88  */
89 void loop() {
90     // Phase 1: Von der Mitte nach links (90° bis 30°)
91     for (int angle = 90; angle >= 30; angle--) {
92         scanPosition(angle);
93     }
94
95     // Phase 2: Von links komplett nach rechts (30° bis 150°)
96     for (int angle = 30; angle <= 150; angle++) {
97         scanPosition(angle);
98     }
99
100    // Phase 3: Von rechts wieder zurück in die Mitte (150° bis 90°)
101    for (int angle = 150; angle >= 90; angle--) {
102        scanPosition(angle);
103    }
104 }
105
106 /**
107  * @brief Führt einen spezifischen Winkel an, misst die Distanz und überträgt das Datenprotokoll.
108  * @details Diese Funktion kapselt den gesamten Messvorgang für ein Gradsegment:
109  * 1. Der Servo erhält den neuen Stellbefehl. Ein Delay von 35ms sichert der Mechanik
110  * des SG90 ausreichend Zeit zu, die Position vibrationsfrei zu erreichen.
111  * 2. Der HC-SR04 sendet einen Ultraschall-Burst aus.
112  * 3. Über die Schallgeschwindigkeit in Luft ( $\frac{343 \text{ m/s}}{2}$ ) wird aus der Laufzeit
113  * die Distanz in cm berechnet ( $\text{Weg} = \frac{\text{Zeit}}{2} \times 0.0343$ ).
114  * 4. Die Daten werden im standardisierten Stringformat 'D:[Winkel],[Distanz]' über die
115  * serielle Schnittstelle (UART) an Processing übertragen.
116  * * @param angle Der präzise anzufahrende Zielwinkel in Grad (30° bis 150°).
117  */
118 void scanPosition(int angle) {
119     radarServo.write(angle);
120     delay(35); // Einschwingzeit für die Servomechanik (wichtig gegen Bildzittern)
121
122     // Ultraschall-Messzyklus starten
123     digitalWrite(PIN_TRIG, LOW);
124     delayMicroseconds(2);
125     digitalWrite(PIN_TRIG, HIGH);
126     delayMicroseconds(10);
127     digitalWrite(PIN_TRIG, LOW);
128
129     // Pulsdauer am Echo-Pin messen
130     long duration = pulseIn(PIN_ECHO, HIGH, 30000);
131     int distance = duration * 0.0343 / 2.0;
132
133     // Protokollausgabe an das Processing-Frontend (z.B. "D:45,18")

```

```
134   Serial.print("D:");  
135   Serial.print(angle);  
136   Serial.print(",");  
137   Serial.println(distance);  
138 }
```

12. Arduino IDE 2.3.6

Dieses Kapitel liefert eine detaillierte Einführung in die Arduino IDE, einschließlich ihrer zentralen Funktionen, verfügbaren Schnittstellen und grundlegenden Arbeitsweise. Es erläutert die einzelnen Bestandteile der Entwicklungsumgebung, beschreibt den Installationsprozess anhand klar strukturierter Schritte und bietet eine Anleitung zur Ersteinrichtung sowie zur grundlegenden Konfiguration der Programmierungsumgebung.

12.1. Arduino IDE Beschreibung

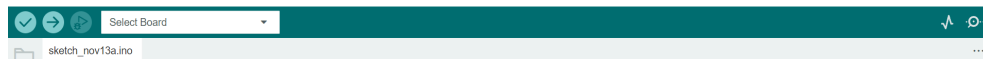
Die Arduino IDE ist die offizielle, Open-Source Entwicklungsumgebung zur Programmierung und Verwaltung von Arduino Mikrocontrollern. Sie ermöglicht das Schreiben, Kompilieren und Übertragen von Quellcode auf eine Vielzahl unterstützter Boards. Als plattformübergreifende Anwendung ist sie mit allen gängigen Betriebssystemen wie Windows, Linux und macOS kompatibel. Ursprünglich auf der Java-Plattform basierend, bietet die IDE Unterstützung für zahlreiche Arduino-Boards sowie eine auf C und C++ basierende Programmiersprache. Durch ihre intuitive Benutzeroberfläche richtet sie sich sowohl an Einsteiger als auch an erfahrene Entwickler im Bereich der Mikrocontroller-Programmierung [FAD18].

Die Mikrocontroller auf Arduino-Boards werden mithilfe von Programmen gesteuert, die in Form von Quellcode verfasst und über die Entwicklungsumgebung verarbeitet werden. In der Arduino IDE erstellte Programme werden als sogenannte Sketche bezeichnet. Dabei handelt es sich um Skripte, die den Funktionsablauf des Mikrocontrollers definieren etwa das Einlesen von Sensorwerten, die Ansteuerung von Aktoren oder die Kommunikation mit anderen Geräten. Nach dem Schreiben wird der Sketch von der IDE kompiliert und in eine maschinenlesbare Hex-Datei übersetzt. Diese Datei wird im Anschluss über die serielle Schnittstelle auf den Mikrocontroller hochgeladen, wo sie dauerhaft gespeichert und beim nächsten Start ausgeführt wird [FAD18].

Die Arduino-IDE setzt sich im Wesentlichen aus zwei funktionalen Kernkomponenten zusammen: dem Editor und dem Compiler. Der integrierte Editor dient als zentrale Arbeitsfläche zum Schreiben und Bearbeiten von Quellcode, während der Compiler für die Übersetzung des Codes in maschinenlesbare Anweisungen sowie dessen Übertragung auf das angeschlossene Arduino-Modul verantwortlich ist [FAD18].

In der Version 2.3.6 der Arduino IDE spielt die Menüleiste am oberen Rand der Benutzeroberfläche eine zentrale Rolle. Sie bietet direkten Zugriff auf essentielle Funktionen wie das Hochladen von Sketchen auf das Board, das Kompilieren zur Überprüfung auf Fehler sowie das Speichern von Projekten. Unter dem Menüpunkt Datei lassen sich neue Sketche anlegen, bestehende Projekte öffnen oder speichern. Ein weiterer bedeutender Abschnitt ist Beispiele dort finden sich eine Vielzahl vorgefertigter Sketche, die typische Anwendungsszenarien demonstrieren, darunter klassische Programme wie Blink zur LED-Steuerung oder Fade zur Helligkeitssteuerung mittels Pulsweitenmodulation. Diese Beispiele erleichtern insbesondere Einsteigern den Zugang zur Mikrocontroller-Programmierung und dienen oft als Ausgangspunkt für eigene Entwicklungen [FAD18].

Die fünf Schaltflächen am oberen Bildschirmrand sind wie folgt:



-  Das Häkchen-Symbol wird verwendet, um den geschriebenen Code zu **Überprüfen**, nachdem der Code eingegeben wurde, wird der Code auf Syntaxfehler geprüft. Dieser Schritt stellt sicher, dass der Code für die Verwendung mit der Hardware bereit ist.
-  Das Symbol **Hochladen** in der Arduino IDE kompiliert den Code und lädt ihn auf den angeschlossenen Arduino hoch.
-  Das Symbol **Start Debugging** in der Arduino IDE startet eine Debugging Sitzung. Damit kann der Code analysiert werden, indem Haltepunkte gesetzt werden. Beim Debuggen wird die Ausführung schrittweise durchlaufen und Variablen auf Kompatibilität und Funktion geprüft, ohne das ganze Programm in einem Stück zu starten.
-  Das Symbol **Serial Plotter** in der Arduino IDE öffnet ein Tool, das Echtzeitdaten, die vom Board über die serielle Verbindung gesendet werden, grafisch darstellt. Dadurch wird die Analyse der Daten erleichtert.
-  Das Symbol **Serial Monitor** in der Arduino IDE öffnet ein Terminal, um textbasierte Daten über die serielle Verbindung anzuzeigen und zu senden. Dadurch wird eine Echtzeitkommunikation mit dem angeschlossenen Board ermöglicht.

12.2. Installation der Arduino IDE

12.2.1. Download oder Arduino IDE

Der Arduino Nano 33 BLE Sense wird über die Arduino Integrated Development Environment (IDE) programmiert, die als Standardumgebung für sämtliche Arduino-Boards etabliert ist. Sie zählt zu den am weitesten verbreiteten Entwicklungsplattformen im Bereich der Mikrocontroller-Programmierung. Die IDE ist sowohl als Offline-Variante zur lokalen Installation als auch als Online-Editor verfügbar. Als Open-Source-Software ist sie darauf ausgelegt, den Programmierprozess möglichst zugänglich zu gestalten vom Schreiben des Codes bis hin zum Hochladen auf das jeweilige Board.

Für alle gängigen Betriebssysteme einschließlich macOS, Linux und Windows stehen passende Versionen zur Verfügung. Die Installation ist unkompliziert und wird durch die breite Unterstützung der Arduino-Community zusätzlich erleichtert. Neben der Desktop-Version bietet Arduino auch eine cloudbasierte Entwicklungsumgebung an: den Online-Arduino-Editor. Diese moderne Webanwendung ermöglicht das plattformunabhängige Programmieren, Speichern und Verwalten von Sketchen direkt im Browser. Sie enthält alle gängigen Bibliotheken und bietet sofortige Unterstützung für aktuelle Arduino-Boards, einschließlich des Nano 33 BLE Sense.

Zugriff auf die verschiedenen Softwarepakete erhalten Sie über die offizielle Website der Arduino-Entwickler: [Arduino-Software](https://www.arduino.cc/en/software). Dort finden sich stets die neuesten Versionen der IDE sowie weiterführende Informationen. Eine Übersicht der verfügbaren Downloadoptionen ist in Abbildung 12.1 dargestellt.

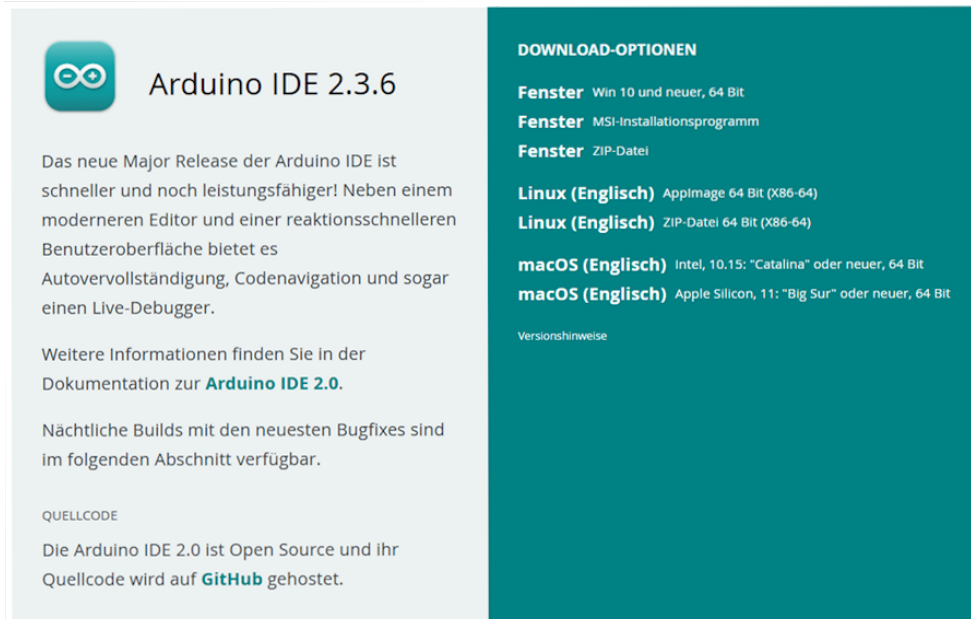


Abbildung 12.1.: Arduino IDE 2.3.6

12.2.2. Anforderungen Betriebssystem

Im Folgenden sind die Systemanforderungen und Varianten der verfügbaren Downloadoptionen für die Arduino-IDE aufgeführt. Diese sind jeweils auf unterschiedliche Betriebssysteme und Nutzerbedürfnisse zugeschnitten. Um einen reibungslosen Betrieb sowie den Zugriff auf alle aktuellen Funktionen und Sicherheitsupdates zu gewährleisten, sollte stets die zur eigenen Plattform passende Version der IDE ausgewählt werden.

- Windows Win 10 und neuer, 64 Bit
- Linux 64 Bit
- macOS Intel, Version 10.15: Catalina oder neuer, 64 Bit
- macOS Apple Silicon, 11: Big Sur oder neuer, 64 Bit

12.2.3. Installation auf Windows

Die Installation der Arduino IDE auf einem Windows-System wird im folgenden Abschnitt Schritt für Schritt erläutert. Im Anschluss daran folgt eine Einführung in zentrale Funktionen der Umgebung, darunter der Board-Manager, der Bibliotheks-Manager, das Sketchbook sowie weitere essentielle Werkzeuge der Benutzeroberfläche.

Um die Arduino IDE auf einem Windows-Computer zu installieren, laden Sie zunächst die entsprechende Installationsdatei von der offiziellen Website unter [Arduino-Software](#) herunter. Starten Sie anschließend die heruntergeladene Datei und folgen Sie den angezeigten Installationsanweisungen. Diese beinhalten in der Regel die Auswahl des Installationsverzeichnis sowie optionaler Komponenten wie Verknüpfungen. Sobald

der Vorgang abgeschlossen ist, steht die Entwicklungsumgebung zur sofortigen Nutzung bereit.



Abbildung 12.2.: Arduino IDE Downloadoptionen

Nachdem der Download der Arduino IDE abgeschlossen wurde, erscheint im nächsten Schritt das Fenster mit den Lizenzbedingungen. Diese sollten aufmerksam durchgelesen werden, bevor mit der Installation fortgefahren wird. Um die Installation zu starten, müssen alle Lizenzbedingungen akzeptiert werden. Dies geschieht durch einen Klick auf die Schaltfläche **Annehmen**, wie in Abbildung 12.3 dargestellt.

Im Anschluss wird der Benutzer aufgefordert, ein Zielverzeichnis für die Installation auszuwählen. Standardmäßig ist ein Pfad vorgegeben, dieser kann jedoch bei Bedarf individuell angepasst werden. Nachdem das gewünschte Verzeichnis ausgewählt wurde, wird der Installationsvorgang durch einen Klick auf **Installieren** gestartet, siehe Abbildung 12.4.

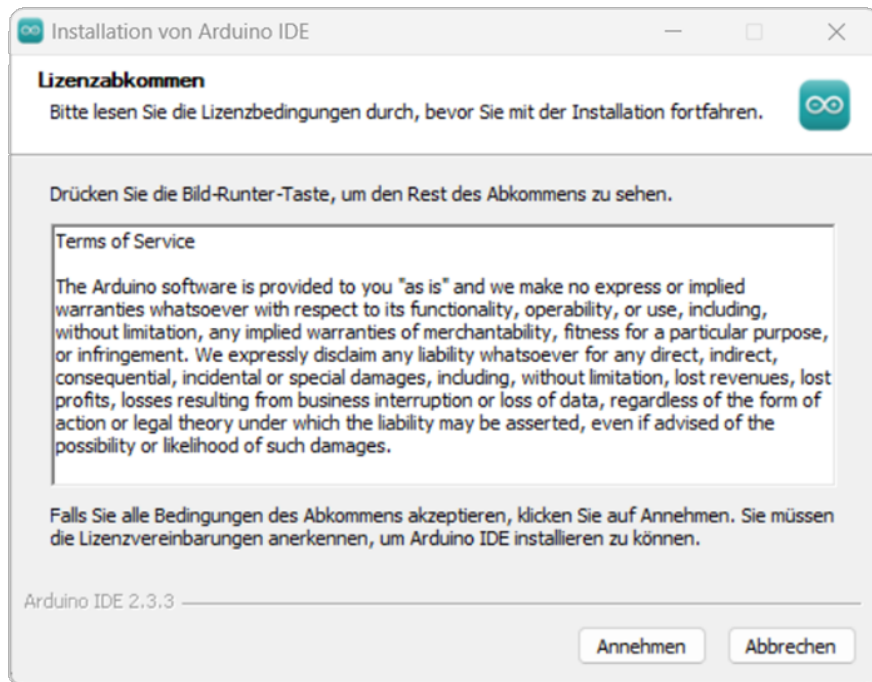


Abbildung 12.3.: Lizenzbestätigung Installation

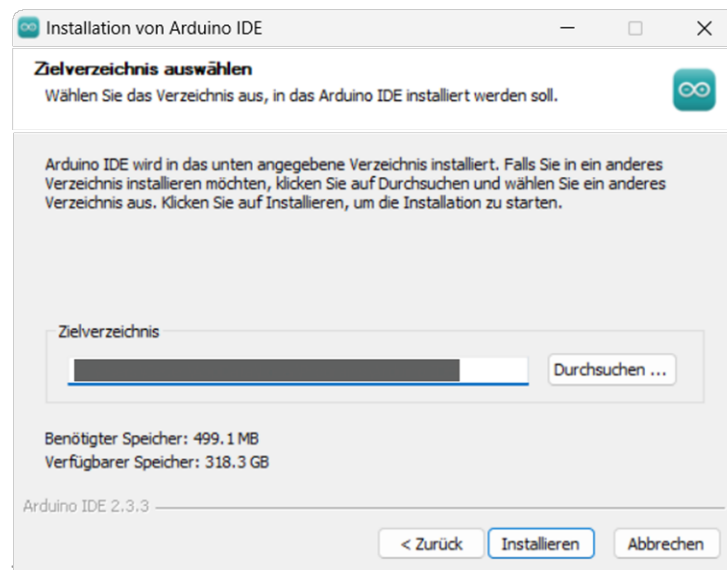


Abbildung 12.4.: Wahl Zielverzeichnis

Sobald die Installation der Arduino IDE erfolgreich abgeschlossen ist, wird die Anwendung automatisch gestartet. Beim ersten Öffnen der Arduino IDE wird ein Standard-Sketch angezeigt, der als Beispiel dient. Dieser ist im Startfenster sichtbar, wie in Abbildung 12.5 dargestellt.

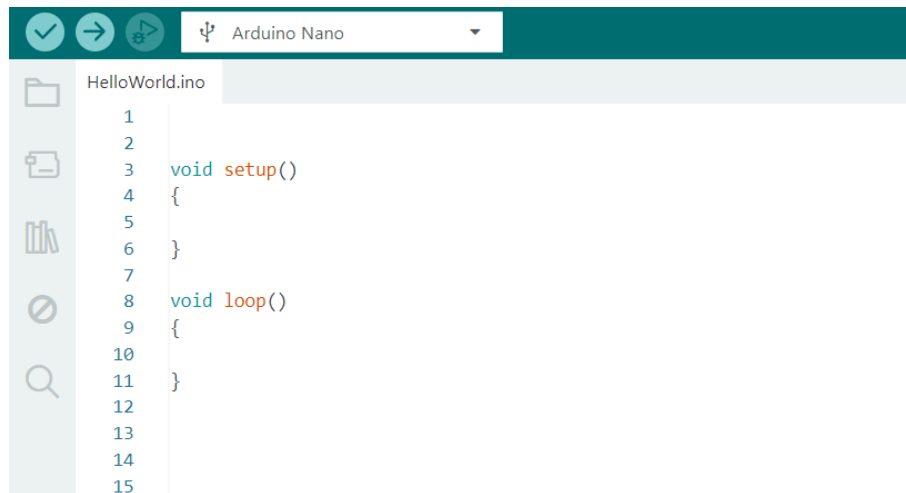


Abbildung 12.5.: Startfenster Arduino-IDE

Aus der oben dargestellten Abbildung 12.5 geht hervor, dass der grundlegende Arduino-Sketch aus zwei Hauptteilen besteht. Der erste Teil ist die Funktion `void setup()`, die keinen Rückgabewert liefert und in der typischerweise die Initialisierungen vorgenommen werden. Hier werden beispielsweise Einstellungen wie die Konfiguration der Ausgangs-LED, die Festlegung von Pin-Modi oder die Initialisierung von Variablen durchgeführt.

Der zweite Teil des Sketches ist die Funktion `void loop()`, in der der Hauptcode geschrieben wird. Funktionen, die wiederholt ausgeführt werden sollen, werden hier definiert und laufen fortlaufend in der Schleife. Diese Funktionen sind ebenfalls zwischen geschweiften Klammern `{}` eingefasst. Jede Funktion in Arduino hat einen Rückgabewert in diesem Fall handelt es sich um den Rückgabebetyp `void`, was bedeutet, dass die Funktion keinen Wert zurückgibt.

12.2.4. Installation (MacOS)

Um die Arduino IDE zu installieren, beginnen Sie mit dem Herunterladen der neuesten Version von der offiziellen Arduino-Website unter [Arduino-Software](#). Wählen Sie die Version, die mit Ihrem Betriebssystem kompatibel ist. In diesem Beispiel wird die Version Arduino 2.6.3 für macOS (Sonoma 14.4.1) verwendet. Die Setup-Datei trägt den Namen Arduino-Software und hat eine Grösse von etwa 193.600 KB. Diese Datei ist im Zip-Format.

Für Nutzer, die die Arduino IDE 2.3.6 mit Safari herunterladen, wird die Datei nach Abschluss des Downloads automatisch entpackt. Andere Browser hingegen erfordern möglicherweise eine manuelle Entpackung. Die folgende Abbildung 12.6 zeigt die neueste Offline-Version der Arduino IDE 2.3.6, die ebenfalls mit allen gängigen Betriebssystemen kompatibel ist.

Da in diesem Kapitel keine entsprechende Hardware für macOS zur Verfügung steht, wird der Installationsprozess für dieses Betriebssystem nicht im Detail behandelt.

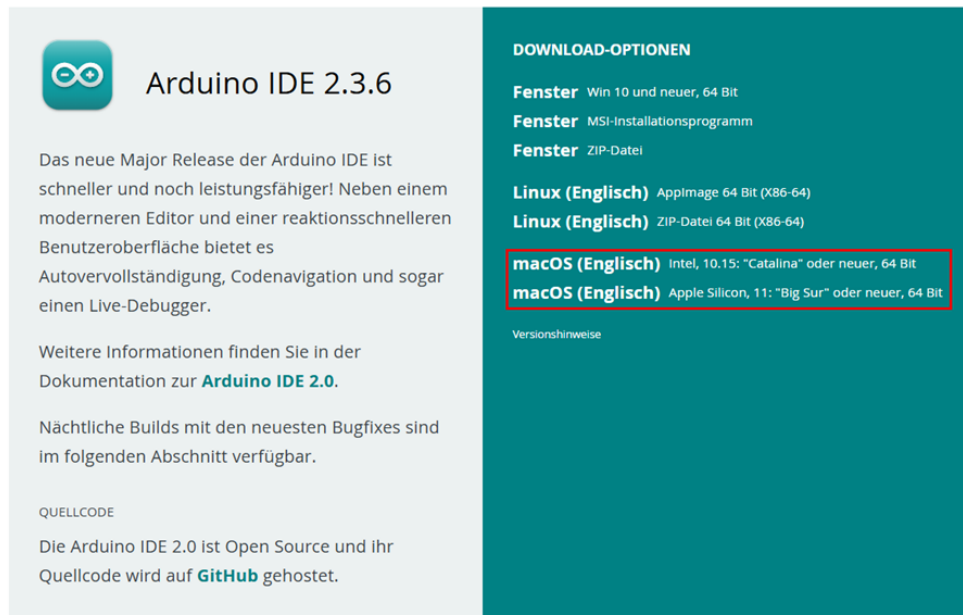


Abbildung 12.6.: ArduinoIDE macOS

12.3. Übersicht der Eigenschaften

Die Arduino IDE verfügt über eine neue Seitenleiste, die den Zugriff auf die am häufigsten verwendeten Werkzeuge erleichtert. 12.7

- **Verify / Upload** Diese Funktionen werden verwendet, um Code zu kompilieren und auf ein Arduino-Board hochzuladen.
- **Select Board and Port** Automatisch erkannte Arduino-Boards werden hier zusammen mit der Portnummer angezeigt.
- **Sketchbook** Dieser Bereich dient als zentrales Verzeichnis für alle Sketche, die lokal auf dem Computer des Benutzers gespeichert sind. Das Sketchbook bietet einen strukturierten, leicht zugänglichen Raum zum Verwalten, Organisieren und Bearbeiten von Code, der in der Arduino-Umgebung entwickelt wurde. Darüber hinaus bietet das Sketchbook eine Synchronisierungsfunktion mit der Arduino Cloud, die einen nahtlosen Zugriff auf gespeicherte Sketche über verschiedene Geräte hinweg ermöglicht. Diese Cloud-Integration erlaubt es Nutzern, Sketche aus der Online-Arduino-Umgebung abzurufen und zu bearbeiten.
- **Boards Manager** Durchsuchen Sie Arduino- und Drittanbieterpakete, die installiert werden können. Zum Beispiel erfordert die Verwendung eines MKR WiFi 1010-Boards die Installation des Arduino SAMD Boards-Pakets.
- **Library Manager** Durchsuchen Sie tausende Arduino-Bibliotheken, die von Arduino und seiner Community erstellt wurden.
- **Debugger** Programme in Echtzeit testen und debuggen.
- **Search** Nach Schlüsselwörtern im geschriebenen Code suchen.
- **Open Serial Monitor** Öffnet das Werkzeug Serial Monitor als neuen Tab in der Konsole.



Abbildung 12.7.: Überblick

12.3.1. Sketchbook

Das Sketchbook ist der Ort, an dem Arduino-Code-Dateien - sogenannte Sketche - gespeichert werden. Diese Dateien haben die Dateiendung .ino. Um eine saubere Organisation zu gewährleisten, muss jeder Sketch in einem Ordner gespeichert werden, der genau denselben Namen wie die Sketch-Datei trägt. Beispiel: Ein Sketch namens MySketch.ino muss in einem Ordner namens MySketch gespeichert werden. Diese Namenskonvention ist notwendig, damit die Arduino IDE die Sketche korrekt erkennen und verwalten kann 12.8.

In der Regel werden Sketche in einem Ordner namens Arduino gespeichert, der sich im Dokumente-Verzeichnis des Systems befindet.

Um auf das Sketchbook zuzugreifen, kann in der Arduino IDE das Ordnersymbol in der Seitenleiste ausgewählt werden. Dadurch wird das Verzeichnis geöffnet, in dem die Sketche gespeichert sind, und ein effizienter Zugriff sowie eine einfache Verwaltung der Sketche ermöglicht.

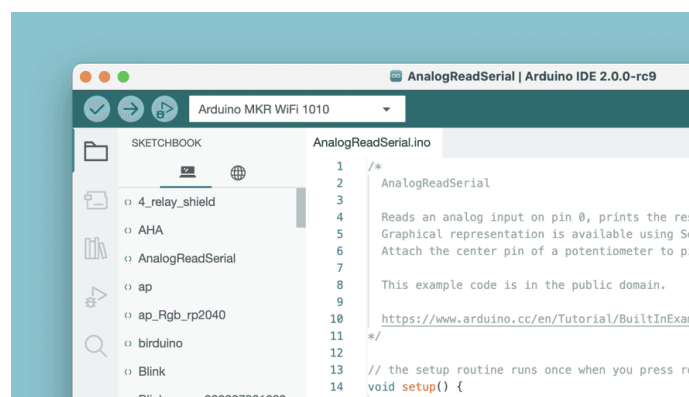


Abbildung 12.8.: Sketchbook

12.3.2. Boards Manager

Der Boards Manager kann über folgende Schritte in der Arduino IDE aufgerufen werden: **Tools** » **Board** » **Board Manager**.

Der Boards Manager ermöglicht die Installation verschiedener Board-Pakete, wie in Abbildung 12.9 dargestellt.

Ein Board-Paket enthält die notwendigen Dateien und Anweisungen, um Code für bestimmte Arduino-Boards zu kompilieren und hochzuladen.

Es gibt verschiedene Board-Pakete, z.B. avr, samd und megaavr. Jedes dieser Pakete unterstützt unterschiedliche Familien von Arduino-Boards:

- avr wird für ältere Boards wie den Arduino Uno verwendet,
- samd ist für neuere Boards wie den Arduino Zero gedacht.

Durch die Verwendung des Boards Managers wird sichergestellt, dass die richtigen Werkzeuge und Dateien für das gewählte Board installiert sind.

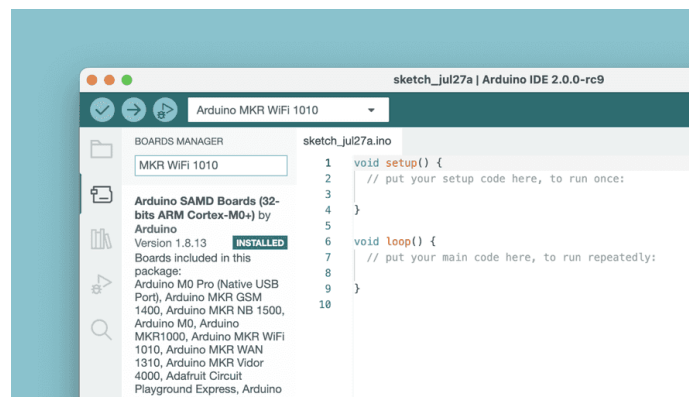


Abbildung 12.9.: Boards Manager

12.3.3. Library Manager

Der Library Manager kann in der Arduino IDE über folgende Schritte aufgerufen werden: **Tools** » **Manage Libraries**.

Der Bibliotheksverwalter ermöglicht es Nutzerinnen und Nutzern, eine große Auswahl an Bibliotheken zu durchsuchen und zu installieren. Bibliotheken erweitern die Grundfunktionen von Arduino und erleichtern Aufgaben wie:

- das Steuern von Servomotoren,
- das Auslesen von Daten bestimmter Sensoren oder
- die Arbeit mit Modulen wie WLAN.

Diese Bibliotheken enthalten vorgefertigten Code, um bestimmte Hardware-Komponenten anzusteuern und komplexe Aufgaben zu vereinfachen. Dadurch wird die Entwicklung von Arduino-Projekten schneller und effizienter wie in 12.10 dargestellt.



Abbildung 12.10.: Library Manager

12.4. Integration und Verwendung einer benutzerdefinierten Bibliothek in der Arduino IDE

Bei der Entwicklung von eingebetteten Systemen auf der Arduino-Plattform sind Bibliotheken unerlässlich, um komplexe Funktionen zu abstrahieren und zu vereinfachen. Eine benutzerdefinierte Bibliothek ist besonders dann sinnvoll, wenn eine bestimmte Funktionalität oder eine spezifische Hardware-Schnittstelle häufig in verschiedenen Projekten verwendet wird.

Dieser Abschnitt beschreibt den Ablauf zum Erstellen, Installieren und Verwenden einer eigenen Bibliothek innerhalb eines Arduino-Projekts - in der Arduino Integrated Development Environment (IDE).

12.4.1. Erstellen der Bibliotheksdateien

Eine gut strukturierte, benutzerdefinierte Arduino-Bibliothek besteht typischerweise aus mindestens zwei zentralen Komponenten:

Header-Datei (.h): Diese Datei enthält die Deklarationen von Klassen, Funktionen und Variablen, die die Schnittstelle (Interface) der Bibliothek definieren. Sie stellt die notwendigen Definitionen bereit, damit externe Programme die Funktionen und Klassen verwenden können.

Quellcode-Datei (.cpp): Diese Datei enthält die tatsächliche Implementierung der in der Header-Datei deklarierten Funktionen und Methoden. Sie definiert das Verhalten der Funktionen und Klassen.

Die Struktur der Bibliothek ist so organisiert, dass eine klare Trennung zwischen Deklaration und Implementierung ihrer Funktionalitäten besteht.

12.4.2. Installation und Integration der Arduino IDE

Sobald die benutzerdefinierte Bibliothek erstellt wurde, muss sie korrekt installiert und in die Arduino IDE integriert werden, um in Projekten verwendet werden zu können. Um eine benutzerdefinierte Bibliothek in der Arduino IDE zu installieren, befolgen Sie diese Schritte:

1. **Bibliotheksordner finden:** Die Arduino IDE sucht nach Bibliotheken im Ordner `libraries`, der sich im Sketchbook-Verzeichnis befindet. Der typische Pfad zu diesem Verzeichnis lautet:

`C:/Users/<Benutzername>/Documents/Arduino/libraries`

- Benutzerdefinierten Bibliotheksordner kopieren: Kopieren Sie den Ordner ihrer Bibliothek, z.B. Nano33BLESenseLED, in das Verzeichnis **libraries**. Der vollständige Pfad sollte dann wie folgt aussehen:

C:/Users/<Benutzername>/Documents/Arduino/libraries/Nano33BLESenseLED

- Arduino IDE neu starten: Nachdem die Bibliothek in das richtige Verzeichnis kopiert wurde, starten sie die Arduino IDE neu, damit sie die neu hinzugefügte Bibliothek erkennt.

12.4.3. Verwendung symbolischer Verknüpfungen zur Bibliotheksintegration

In manchen Fällen kann es bequemer oder notwendig sein, die Bibliotheksdateien außerhalb des Standardbibliotheksverzeichnisses zu speichern. Zum Beispiel, wenn die Bibliotheken in einem GitHub-Repository gespeichert sind oder zur besseren Versionskontrolle, kann der Befehl **mklink** verwendet werden, um eine symbolische Verknüpfung zu erstellen.

Dies ermöglicht es der Arduino IDE, auf die Bibliotheksdateien zuzugreifen, als ob sie sich im Standardverzeichnis befinden, ohne dass die Dateien dupliziert werden müssen.

12.4.4. Windows-Tool **mklink** zum Erstellen symbolischer Verknüpfungen

Symbolische Verknüpfungen ermöglichen es, Bibliotheken an einem anderen Speicherort abzulegen, während sie von der Arduino IDE trotzdem so behandelt werden, als befänden sie sich im Standardverzeichnis **libraries**.

Dies kann besonders nützlich sein in versionierten Umgebungen, z.B. bei der Verwendung von Git, oder um Bibliotheken besser zu organisieren, ohne Dateien zu duplizieren.

Schritte zum Erstellen einer symbolischen Verknüpfung:

- Öffnen der Eingabeaufforderung mit Administratorrechten:** Drücken Sie **Win + S** und suchen Sie nach **cmd**, klicken Sie dann auf **Eingabeaufforderung** und wählen Sie **Als Administrator ausführen** wie in 12.11 gezeigt.

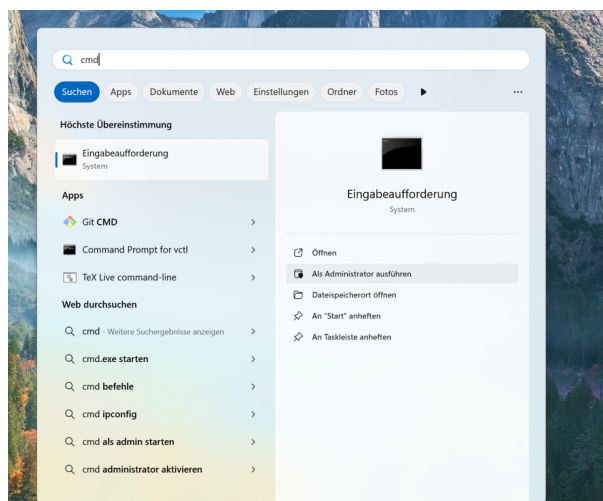


Abbildung 12.11.: Eingabeaufforderung mit Administrator Rechten

2. **Führen Sie den Befehl `mklink` aus:** Der Befehl zum Erstellen einer symbolischen Verknüpfung lautet: `mklink /D TTargetPathSourcePath"`

- **TargetPath:** Der Ort, an dem die Arduino IDE die Bibliothek erwartet also im Ordner `libraries`.
- **SourcePath:** Der tatsächliche Speicherort der Bibliothek.

In diesem Beispiel befindet sich die Bibliothek unter:

`C:/.../.../.../GitHub/241031ArduinoNano33BLESense/report/Code/Nano33BLESense/Nano33BLESenseLED`

Der Zielpfad im libraries-Ordner der Arduino IDE lautet:

`C:/.../.../.../Arduino/libraries/Nano33BLESenseLED`

Der vollständige Command ist in 12.12 zu sehen.

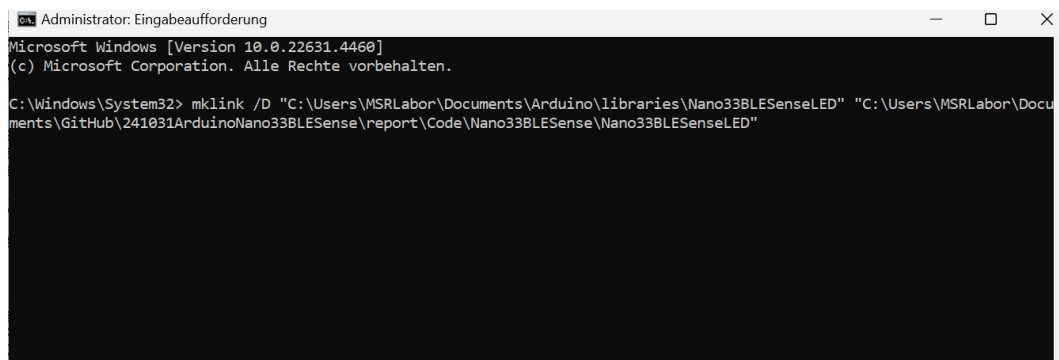


Abbildung 12.12.: Eingabeaufforderung

Überprüfen der symbolischen Verknüpfung: Nach dem Ausführen des Befehls navigieren Sie zum Ordner `libraries`. Dort sollte ein Ordner mit dem Namen `Nano33BLESenseLED` vorhanden sein, der als symbolische Verknüpfung fungiert und auf den tatsächlichen Speicherort der Bibliothek verweist.

12.4.5. Überprüfen der Verfügbarkeit der Bibliothek in der Arduino IDE

Um sicherzustellen, dass die benutzerdefinierte Bibliothek erfolgreich integriert wurde und von der Arduino IDE erkannt wird, befolgen Sie diese Schritte:

1. Arduino IDE neu starten: Falls die Arduino IDE während der Installation der Bibliothek oder nach dem Erstellen der symbolischen Verknüpfung geöffnet war, schließen und öffnen Sie sie erneut. Dadurch wird die Liste der verfügbaren Bibliotheken neu geladen.
2. Bibliotheksmenü überprüfen: Navigieren Sie zu `Sketch` gehen Sie zu `Include Library` und wählen Sie die Bibliothek `Nano33BLESenseLED` aus, wie in 12.13 gezeigt.
3. Überprüfung in der Liste: Wenn die Bibliothek in der Liste erscheint, bedeutet das, dass sie von der IDE erfolgreich erkannt wurde. Falls nicht, überprüfen Sie nochmals die Verzeichnisstruktur und die symbolische Verknüpfung, um sicherzustellen, dass sie korrekt eingerichtet sind.

4. Testprogramm kompilieren: Schreiben Sie ein einfaches Testprogramm, das Funktionen oder Klassen aus der Bibliothek verwendet. Kompilieren Sie den Sketch, um sicherzustellen, dass keine Fehler auftreten – das bestätigt, dass die Bibliothek erfolgreich integriert wurde.

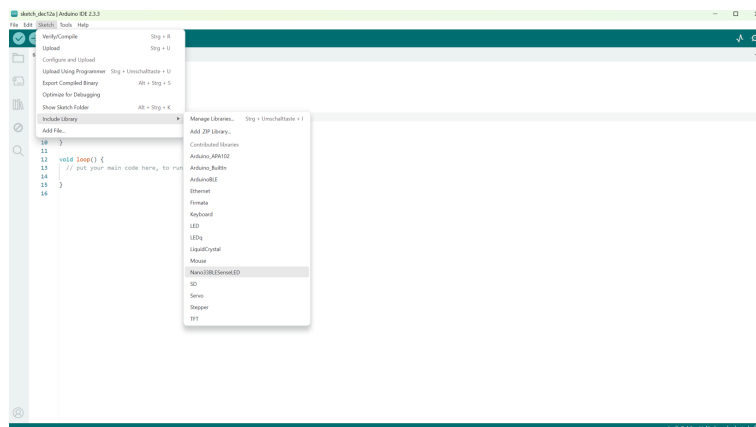


Abbildung 12.13.: Einbinden der Nano33BLESenseLED Bibliothek

Sobald das Programm ohne Fehler kompiliert, ist die Bibliothek einsatzbereit und kann bedenkenlos in Arduino-Projekten verwendet werden.

12.4.6. Serieller Monitor

Der Serieller Monitor kann über folgende Schritte aufgerufen werden: **Tool** » **Serial Monitor**

Der Serieller Monitor ist ein Werkzeug, das es ermöglicht, Daten vom Board anzuzeigen, zum Beispiel mithilfe des Befehls `Serial.print()`.

Früher wurde dieses Tool in einem separaten Fenster geöffnet, ist aber jetzt in den Editor integriert, was es erleichtert, mehrere Instanzen gleichzeitig auf dem Computer auszuführen.

Der Serieller Monitor ist in 12.14 dargestellt.

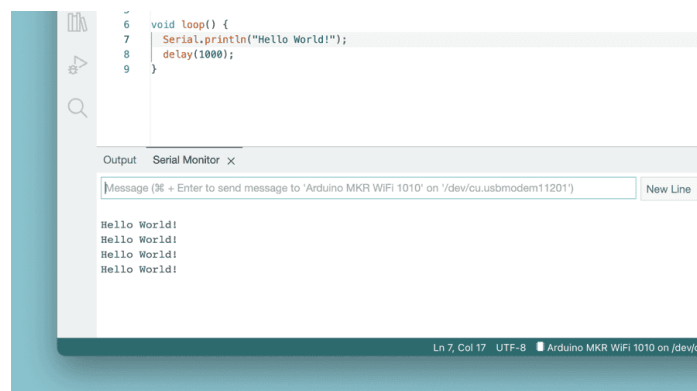


Abbildung 12.14.: Serieller Monitor

12.4.7. Serieller Plotter

Das Werkzeug **Serial Plotter** eignet sich hervorragend zur Visualisierung von Daten in Form von Diagrammen, z.B. zur Überwachung von Spannungsspitzen.

Es ermöglicht das gleichzeitige Beobachten mehrerer Variablen und bietet die Möglichkeit, bestimmte Datentypen gezielt ein- oder auszublenden.

In Abbildung 12.15 wird die Beispielliste angezeigt, wenn ein UNO R4 WiFi-Board mit dem Computer verbunden ist.

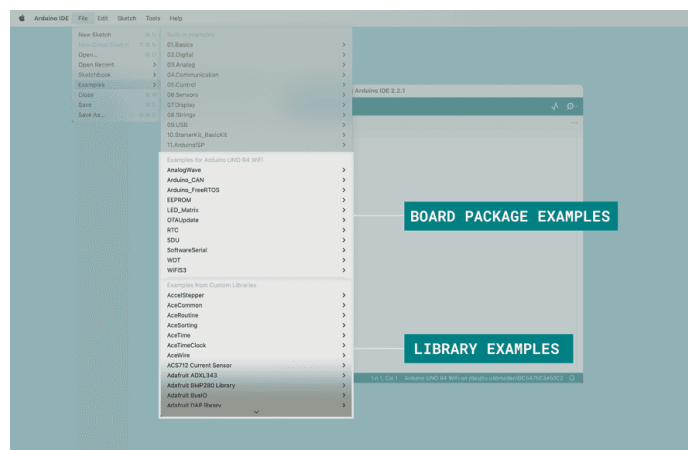


Abbildung 12.15.: Beispiele

12.6. Debuggen

Diese Funktion verbessert den Entwicklungsprozess, indem sie es erleichtert, Fehler zur Laufzeit zu finden und zu beheben, anstatt sich ausschliesslich auf print-Anweisungen oder Versuch-und-Irrtum zu verlassen 12.16.

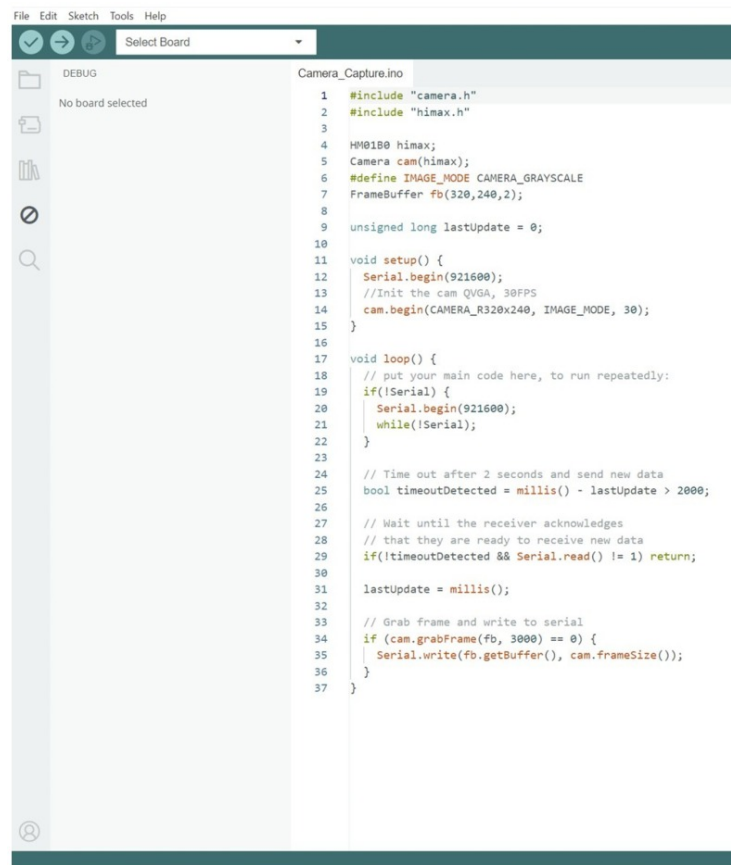


Abbildung 12.16.: Beispiel Debuggen

12.7. Autovervollständigung

Die Autovervollständigung ist eine zentrale Funktion der Arduino IDE Version 2, die das Programmieren erleichtert, indem sie Funktionen und Elemente aus der Arduino-API vorschlägt. Diese Funktion hilft dabei, Code schneller und genauer zu schreiben, was die Effizienz und Genauigkeit verbessert.

Damit die Autovervollständigung funktioniert, muss in der Arduino IDE das verwendete Board ausgewählt sein. Dadurch erkennt der Editor die richtigen Funktionen und Bibliotheken für das jeweilige Board und kann präzise Vorschläge machen 12.17.

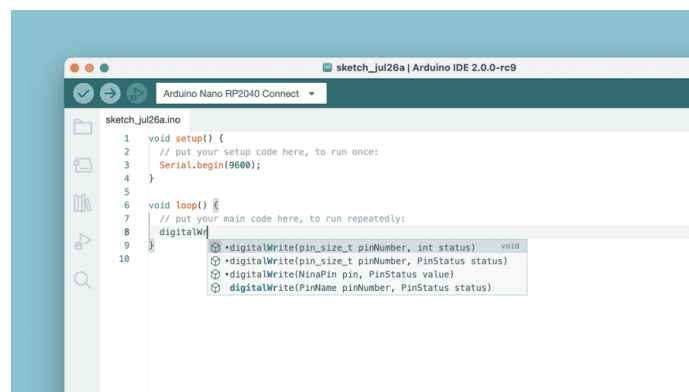


Abbildung 12.17.: Autovervollständigung

12.8. Bibliotheken und Modularität

Die Arduino IDE erlaubt es, Funktionen in Bibliotheken auszulagern und wiederzuverwenden. Viele Sensoren oder Module wie DHT11, LCD oder Motorshields bringen eigene Bibliotheken mit, die über die Bibliotheksverwaltung eingebunden werden können [Smi11]:

```
#include <ArduinoBLE.h>
```

Die Verwendung von Bibliotheksfunktionen ermöglicht es, gröSSere Programme in modulare Teilfunktionen zu gliedern und dadurch Struktur sowie Wartbarkeit deutlich zu verbessern. So kann beispielsweise mit der Funktion `BLE.setLocalName()` ein Bluetooth-Name für die Applikation festgelegt werden, ohne dass die zugrunde liegenden Befehle und Abläufe im Detail bekannt sein müssen.

13. Processing 4.3

Dieses Kapitel liefert eine detaillierte Einführung in die Entwicklungsumgebung Processing, einschließlich ihrer zentralen Funktionen, grafischen Fähigkeiten und der grundlegenden Arbeitsweise. Es erläutert die einzelnen Bestandteile der Entwicklungsumgebung, beschreibt den Installationsprozess anhand klar strukturierter Schritte für verschiedene Plattformen und bietet eine Anleitung zur Konfiguration der Programmierungsumgebung für die serielle Datenverarbeitung.

13.1. Processing Beschreibung

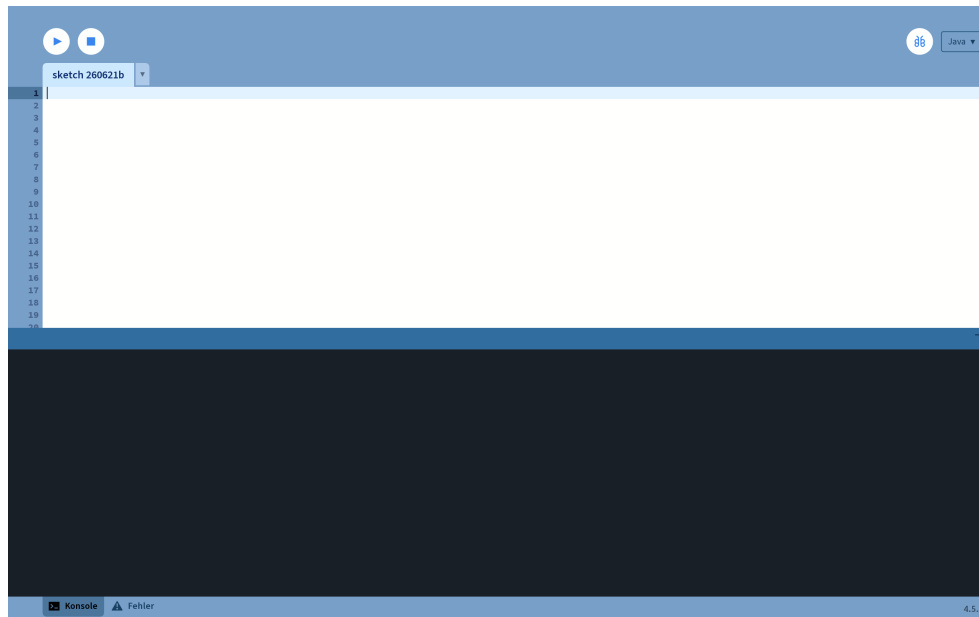
Processing ist eine flexible, quelloffene (Open-Source) Entwicklungsumgebung und eine objektorientierte Programmiersprache, die ursprünglich im Jahr 2001 am MIT Media Lab initiiert wurde. Sie wurde gezielt entwickelt, um Designern, Künstlern und Ingenieuren den Einstieg in die Programmierung visueller, interaktiver Anwendungen und Grafiken zu erleichtern. Basiert auf der Java-Plattform, bietet Processing eine stark vereinfachte Syntax, die komplexe Grafikoperationen wie das Zeichnen von Primitiven, das Verarbeiten von Transformationsmatrizen und das Rendern in Echtzeit mit nur wenigen Zeilen Code ermöglicht. Als plattformübergreifende Anwendung läuft die integrierte Entwicklungsumgebung (IDE) nativ unter Windows, Linux und macOS [Reas:2014].

In Analogie zur Arduino-Umgebung werden Programme in Processing ebenfalls als Sketche bezeichnet. Diese Skripte definieren die Geometrie, Farbwerte und das Verhalten des Anzeigefensters, das sich bei der Ausführung des Codes öffnet. Ein Processing-Sketch wird standardmäßig mit der Dateiendung `.pde` abgespeichert. Nach dem Starten des Programms übersetzt die IDE den Quellcode intern in Java-Bytecode, bindet die erforderlichen Grafikbibliotheken ein und führt die Applikation als eigenständiges Java-Applet innerhalb eines dedizierten Display-Fensters aus. Dies erlaubt eine unmittelbare visuelle Rückmeldung über das programmierte Verhalten, was insbesondere bei der Echtzeit-Visualisierung von Sensordaten von großem Vorteil ist [Reas:2014].

Die integrierte Entwicklungsumgebung von Processing setzt sich im Wesentlichen aus drei funktionalen Ebenen zusammen: dem Code-Editor mit der darüberliegenden Toolbar, dem Nachrichtenbereich für den Systemstatus und der Konsole im unteren Abschnitt. Während der Editor das Schreiben des Quellcodes inklusive Syntax-Hervorhebung übernimmt, dient die Konsole der Ausgabe von Fehlermeldungen (Compiler-Errors) sowie textbasierten Debug-Informationen, die mittels spezifischer Ausgabebefehle generiert werden [Reas:2014].

In der aktuellen Version von Processing kommt der Toolbar am oberen Rand des Editor-Fensters eine zentrale Bedeutung zu. Sie ist im Vergleich zu IDEs minimalistisch gehalten, um den Fokus auf den kreativen und funktionalen Code zu lenken. Über das Menü Skript oder direkt über die Symbole der Toolbar lassen sich Programme ausführen, stoppen oder für die plattformunabhängige Weitergabe als eigenständige Applikation exportieren. Über den Menüpunkt Beispiele bietet Processing zudem eine umfangreiche Sammlung vorgefertigter Skripte aus den Bereichen Grafik, Interaktion, Mathematik und Netzwerkkonstruktion, die als fundierte Grundlage für eigene Entwicklungen herangezogen werden können [Reas:2014].

Die zentrale Benutzeroberfläche zum schreiben des Codes sieht wie folgt aus:



Das Wiedergabe-Symbol (Play) wird verwendet, um den geschriebenen Code zu **Ausführen**. Die IDE kompiliert das Programm, öffnet das grafische Ausgabefenster und startet die Anwendung.

Das Quadrat-Symbol **Stoppen** beendet die Ausführung des aktuell laufenden Sketches sofort und schließt das dazugehörige grafische Ausgabefenster.

13.2. Installation von Processing

13.2.1. Download von Processing

Die Erstellung von grafischen Oberflächen zur Datenvisualisierung erfolgt über die Processing-Entwicklungsumgebung, die sich als mächtiges Werkzeug für die Verknüpfung von Hardware-Rohdaten mit visuellen Elementen etabliert hat. Processing steht als Open-Source-Software kostenlos zur Verfügung und erfordert für die reine Ausführung innerhalb der IDE keine komplexe lokale Vorinstallation von Java-Entwicklungsumgebungen, da eine passende Java-Laufzeitumgebung (JRE) direkt in den Softwarepaketen integriert ist.

Die Distributionen sind für alle gängigen Desktop-Betriebssysteme verfügbar. Der Zugriff auf die offiziell verifizierten Softwarepakete erfolgt über die Webpräsenz der Entwicklergemeinschaft unter [Processing-Software](#). Ein Überblick über das Download-Verzeichnis ist in Abbildung 13.1 dargestellt.

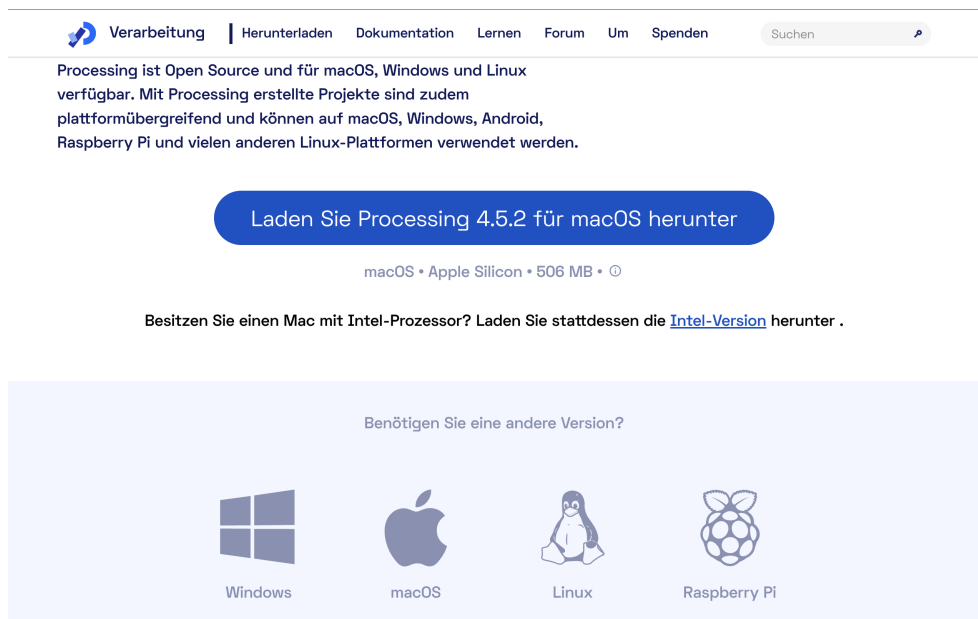


Abbildung 13.1.: Processing Downloadverzeichnis

13.2.2. Anforderungen Betriebssystem

Die Systemanforderungen von Processing sind plattformübergreifend optimiert. Um eine stabile Render-Performance der Grafik-Pipelines (insbesondere bei der Nutzung von OpenGL- oder P3D-Renderern) zu gewährleisten, müssen die Hardwarekomponenten und das Betriebssystem die folgenden Mindestanforderungen erfüllen:

- Windows: Windows 10 oder neuer, 64-Bit Intel- oder AMD-Prozessor
- Linux: 64-Bit (Kernel-Spezifikationen entsprechend der gängigen Distributionen wie Ubuntu)
- macOS: Intel-Architektur, Version 10.15 (Catalina) oder neuer, 64-Bit
- macOS: Apple Silicon (M1/M2/M3), Version 11 (Big Sur) oder neuer, native ARM-Unterstützung

13.2.3. Installation auf Windows

Die Einrichtung unter Windows erfordert keinen klassischen Windows-Installer, sondern wird als portable Anwendung bereitgestellt. Nach dem Download der Archivdatei [processing-4.3-windows-x64.zip](#) wird diese per Rechtsklick entpackt. Der resultierende Ordner kann in ein beliebiges Verzeichnis verschoben werden (z.,B. [C:/Programme/](#)). Die Entwicklungsumgebung wird anschließend durch einen Doppelklick auf die ausführbare Datei [processing.exe](#) gestartet.

13.2.4. Installation auf macOS

Für macOS-Systeme liegt die Software als komprimiertes ZIP-Archiv vor. Nach dem automatischen Entpacken durch den Browser wird die Applikation [Processing.app](#) per Drag-and-Drop in den Systemordner [Programme](#) verschoben.

Beim ersten Start über ein Netzwerklaufwerk oder ein externes Speichermedium (wie einen USB-Stick) greifen die Sicherheitsmechanismen von macOS (Gatekeeper). Sollte

das System den Start verweigern, kann die Quarantäne-Eigenschaft der Datei über das Terminal mittels des Befehls `xattr -cr /Pfad/zu/Processing.app` aufgehoben werden. Nach dem erfolgreichen Bootvorgang präsentiert sich das Startfenster der IDE mit einem leeren Dokumenten-Template, wie in Abbildung 13.2 illustriert.

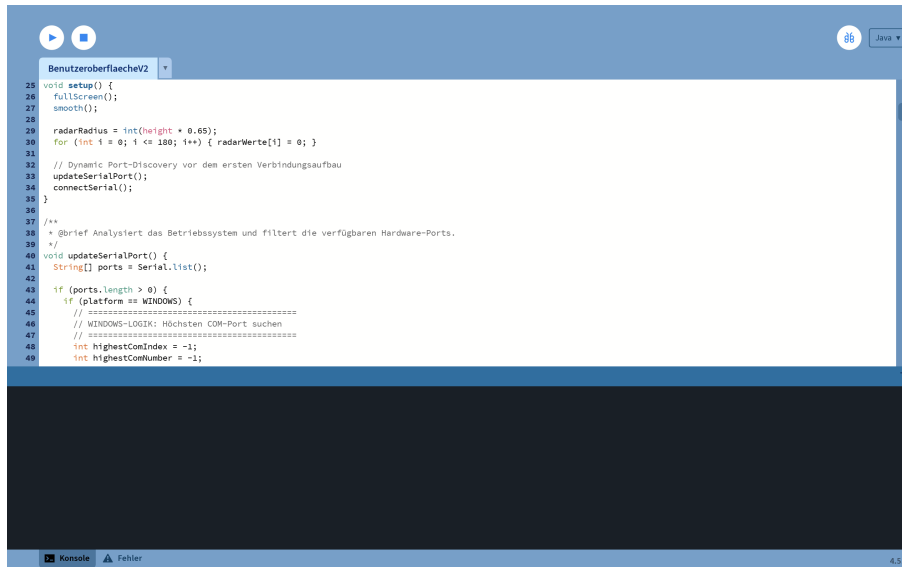


Abbildung 13.2.: Startfenster der Processing-IDE

Das geöffnete Template zeigt die grundlegende Struktur. Obwohl einfache Skripte in Processing linear geschrieben werden können, basiert jede professionelle, interaktive Anwendung auf zwei tragenden Säulenstrukturen:

Die Funktion `void setup()` wird beim Programmstart genau einmal aufgerufen. Hier werden grundlegende Systemeinstellungen vorgenommen, wie die Definition der Fenstergröße mittels `size()` oder `fullScreen()`, die Konfiguration des Antialiasing (`smooth()`) sowie das Öffnen von Kommunikationskanälen zu seriellen Ports.

Die Funktion `void draw()` fungiert als kontinuierliche Zeichenschleife. Sie wird standardmäßig exakt 60-mal pro Sekunde (60 Hz) aufgerufen. Innerhalb dieser Schleife wird das gesamte Radarbild in jedem Frame komplett neu gezeichnet, Sensor-Arrays ausgelesen und Hindernisse mathematisch berechnet. Durch das wiederholte Zeichnen entsteht für den Anwender die fließende Animation des rotierenden Radarstrahls.

13.3. Übersicht der Eigenschaften und Contribution Manager

Über das Menü `Skizze` `Bibliothek hinzufügen` `Bibliothek verwalten...` verfügt Processing über ein mächtiges Werkzeug zur modularen Code-Erweiterung: den Contribution Manager (Abbildung 13.3).

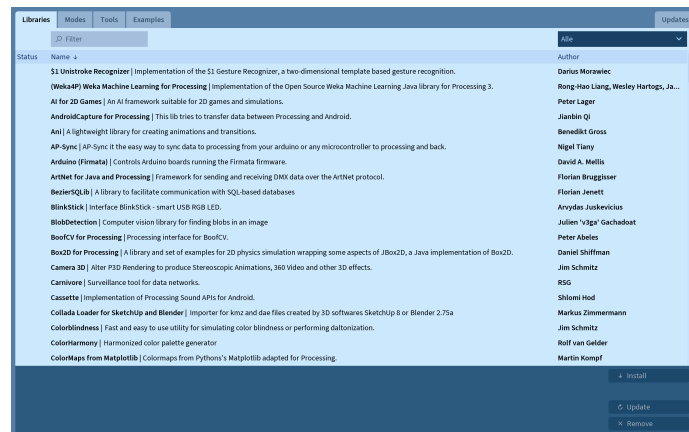


Abbildung 13.3.: Processing Contribution Manager

Der Contribution Manager gliedert sich in vier funktionale Bereiche:

- **Libraries:** Ermöglicht das Durchsuchen und Installieren tausender Community-Bibliotheken zur Integration komplexer Hardwarekomponenten (z.,B. Videokameras, Sound-Engines oder Netzwerkschnittstellen).
- **Modes:** Erlaubt das Umschalten der Programmiersprache innerhalb der IDE (z.,B. von Java auf Python- oder JavaScript-Modus).
- **Tools:** Integriert nützliche Helferwerkzeuge direkt in den Editor, wie beispielsweise Farbwähler oder Font-Generatoren zur Erstellung benutzerdefinierter Pixelfonts.
- **Examples:** Lädt vollständige Beispiel-Projekte zu installierten Bibliotheken herunter.

13.3.1. Die serielle Schnittstelle (Serial-Bibliothek)

Für Hardware-Projekte, bei denen Messdaten von einem externen Mikrocontroller ausgewertet werden sollen, ist die Bibliothek `processing.serial.*` von essenzieller Bedeutung. Sie ist standardmässig im Kern von Processing enthalten, muss jedoch zu Beginn eines jeden Visualisierungs-Sketches explizit importiert werden: `import processing.serial.*`; Diese Bibliothek stellt das `Serial`-Objekt bereit, welches Interrupt-gesteuert im Hintergrund den Hardware-Puffer des gewählten COM- oder USB-Ports überwacht. Mittels Funktionen wie `myPort.readStringUntil n` können Datenpakete des Arduinos verlustfrei abgefangen, segmentiert und unmittelbar zur grafischen Darstellung an die `draw()`-Schleife übergeben werden.

Teil V.

Developmenmt

14. Flow Chart Development XY

Teil VI.

Monitoring

15. Monitoring XY

Teil VII.

**Verification - Evaluation -
Conclusion**

16. Evaluation/Verification XY

17. To DoX Y

18. Conclusion XY

Teil VIII.

Anhang

19. Bill Of Material

Tabelle 19.1.: Hardware Bill of Materials

Bild	Stückzahl	Beschreibung	Zulieferer	Preis()
	1	Arduino Nano 33 BLE Sense	reichelt.de	33.03
	1	Ultraschallsensor HC-SR-04	reichelt.de	2.90
	1	TowerPro Servomotor SG90	az-delivery.de	5.99
	1	Breadbord 25 Kontakte	reichelt.de	2.39
	1	LED grün 8mm	reichelt.de	0.19
	1	LED rot 8mm	reichelt.de	0.19
	1	Entwicklerboards - Steckbrückenkabel 40 Pole	reichelt.de	1.55
	1	Micro-USB Kabel	reichelt.de	5.26
	2	220 Ohm Widerstand	reichelt.de	0.18
	1	2k Ohm Widerstand	reichelt.de	0.10
	1	1k Ohm Widerstand	reichelt.de	0.09
Total				≈ 51,87

Stichwortverzeichnis Keywords

20. **SBOM**

`requirements.txt`

21. Package Example

21.1. Introduction

21.2. Description

21.3. Installation

`pip packageExample`

21.4. Example - Manual

21.5. Example

21.6. Example - Code

21.7. Example - Files

`PackageExample.py`

21.8. Further Reading

Literatur

- [Aba+16] M. Abadi u. a. “TensorFlow: A System for Large-Scale Machine Learning”. In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. Savannah, GA: USENIX Association, Nov. 2016, S. 265–283. ISBN: 978-1-931971-33-1. URL: <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi>.

Teil IX.

**Hints - Examples for LaTeX -
Read, Use and Remove**

22. Hinweise

Bitte verwenden Sie **biber.exe**.

Wenn Sie ein Symbolverzeichnis erstellen möchten, rufen Sie makeindex auf:
makeindex %.nlo -s nomencl.ist

22.1. Allgemeine mathematische Beschreibung Bézier-Kurve

Bézier-Kurven werden mit Hilfe von Bernsteinpolynomen gebildet. Wenn mindestens zwei Punkte sowie zwei Tangenten bekannt sind, können Kontrollpunkte ermittelt werden mit denen ein Kontrollpolygon gebildet wird. An diesem Kontrollpolygon orientiert sich der Verlauf der Kurve. Die Anzahl der Kontrollpunkte ist abhängig vom Grad der Bézier-Kurve. Eine Bézier-Kurve vom Grad n hat $n+1$ Kontrollpunkte. Die Berechnung der Bézier-Kurve erfolgt über den De-Casteljau-Algorithmus.

Definition. Im Intervall $[0; 1]$ ist das **Bernstein-Polynom n -ten Grades** definiert durch:

$$b_{i,n}(\lambda) = \binom{n}{i} (1-\lambda)^{n-i} \lambda^i, \quad \lambda \in [0, 1], \quad i = 0, \dots, n \quad (22.1)$$

Definition. Der Binomialkoeffizient ist definiert durch:

$$\binom{n}{i} = \frac{n!}{i!(n-i)!}, \quad i = 0, \dots, n \quad (22.2)$$

Hierbei bilden die Bernstein-Polynome $b_{i,n}$ eine Basis des Vektorraumes $\mathbb{P}^n(I)$ der Polynome höchstens vom Grad n über I . Somit lässt sich jedes Polynom höchstens n -ten Grades eindeutig als Linearkombination schreiben. [Far02]

Beispiel. Bestimmung von Bernstein-Polynome für $n = 3$ gilt:

$$\begin{aligned} b_{3,0}(\lambda) &= \binom{3}{0} (1-\lambda)^{3-0} \lambda^0 = (1-\lambda)^3 \\ b_{3,1}(\lambda) &= \binom{3}{1} (1-\lambda)^{3-1} \lambda^1 = 3\lambda(1-\lambda)^2 \\ b_{3,2}(\lambda) &= \binom{3}{2} (1-\lambda)^{3-2} \lambda^2 = 3\lambda^2(1-\lambda) \\ b_{3,3}(\lambda) &= \binom{3}{3} (1-\lambda)^{3-3} \lambda^3 = \lambda^3 \end{aligned}$$

$b_{3,i}(\lambda)$ mit $i = 0, 1, 2, 3$ sind die kubischen Bernstein-Polynome von Grad 3.

Definition. Gegeben seien die Punkte

$$Q_i = \begin{pmatrix} x_i \\ y_i \end{pmatrix} \quad \text{mit} \quad Q_i \in \mathbb{R}^2, \quad i = 0, 1, \dots, n$$

Eine **Bézier-Kurve** ist dann definiert durch

$$C(\lambda) = \sum_{i=0}^n Q_i \cdot b_{i,n}(\lambda) \quad (22.3)$$

Die Punkte $Q_i, i = 0, \dots, n$ heißen **Kontrollpunkte**.

Die Kontrollpunkte einer Bézier-Kurve bilden das sogenannte Kontrollpolygon.

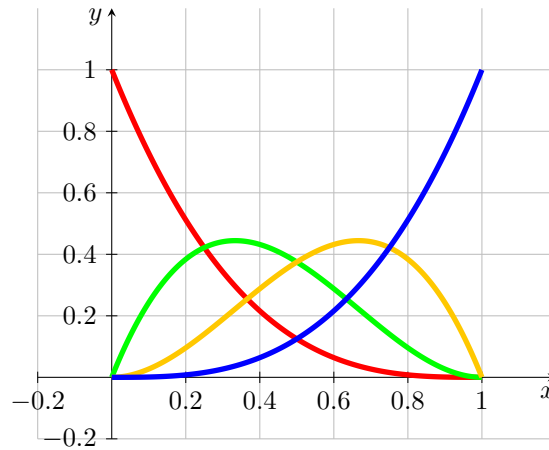


Abbildung 22.1.: Bernstein-Polynom vom Grad 3

Bemerkung. Es sei der Startpunkt P_0 und der Endpunkt P_1 , sowie die Tangenten $\vec{t}_0$ und $\vec{t}_1$ gegeben. Die Tangenten sind nicht notwendigerweise normiert. Die Kontrollpunkte Q_0, Q_1, Q_2 und Q_3 der zugehörigen Bézier-Kurve sind durch folgende Gleichungen gegeben:

$$Q_0 = P_0, \quad Q_1 = P_0 + \lambda_0 \vec{t}_0, \quad Q_2 = P_1 - \lambda_1 \vec{t}_n, \quad Q_3 = P_1 \quad (22.4)$$

[Jak+10]

Beispiel. Gegeben seien

$$P_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad \vec{t}_0 = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad \text{und} \quad P_1 = \begin{pmatrix} 6 \\ 1 \end{pmatrix}, \quad \vec{t}_1 = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

Dann ergibt sich gemäß Satz 22.4 für Kontrollpunkte der zugehörigen Bézier-Kurve:

$$Q_0 = P_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad Q_1 = P_0 + \vec{t}_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 5 \end{pmatrix},$$

$$Q_2 = P_1 - \vec{t}_n = \begin{pmatrix} 6 \\ 1 \end{pmatrix} - \begin{pmatrix} 1 \\ -2 \end{pmatrix} = \begin{pmatrix} 5 \\ 3 \end{pmatrix}, \quad Q_3 = P_1 = \begin{pmatrix} 6 \\ 1 \end{pmatrix}$$

Die Bézier-Kurve und ihre Kontrollpunkte sind in der Abbildung 22.2 dargestellt.

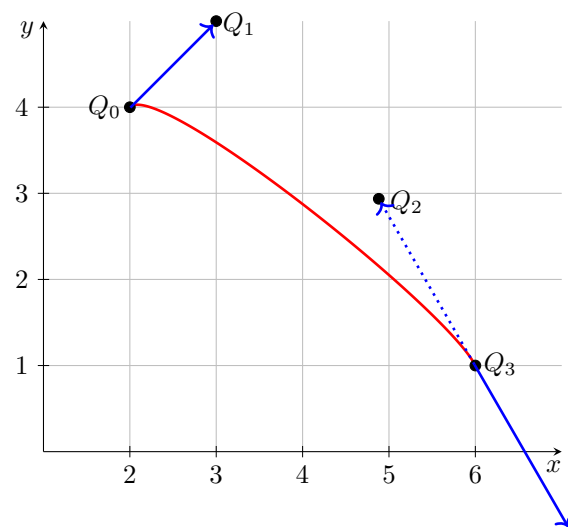


Abbildung 22.2.: Bézier-Kurve zum Beispiel 22.1

23. Verrundung mit einem Kreisbogen

23.1. Gleichungen

Die Verrundung mit einem Kreisbogen ist eine einfache Variante der Glättung von Ecken. Diese Variante ermöglicht die Bearbeitung und die Erzeugung von Dateien gemäss DIN 66025. Zur Glättung werden stets drei Punkte P_0 und S und P_1 betrachtet, damit ergibt sich ein symmetrisches Hermite-Problem. Gemäss Satz ?? wird vorausgesetzt, dass es sich in der Standardform $HP(L, \alpha)$ befindet.

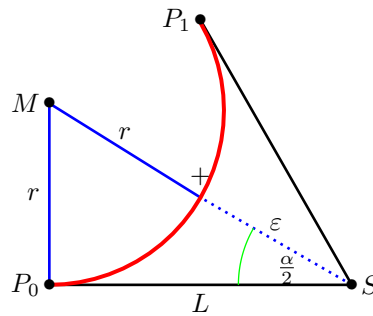


Abbildung 23.1.: Glättung einer Ecke mit Hilfe eines Kreisbogens - Dreieck

Die Abbildung 23.1 stellt die Situation dar. Die Punkte P_0 , S und P_1 sind gegeben. Der eingeschlossene Winkel $\alpha = \angle(P_0; S; P_1)$ sowie der Abstand $L = |||S - P_0||| = |||P_1 - S|||$ sind in der Grafik dargestellt. Der rote Kreisbogen ist das gewünschte Ergebnis. Der Abstand seines Mittelpunkts M zu S beträgt dann $r + \varepsilon$, wobei r der Radius des Kreisbogens und ε die vorgegebene Toleranz ist. Somit erhalten wir ein rechtwinkliges Dreieck P_0SM , dessen Kantenlängen L , $r + \varepsilon$ und r sind.

Gemäss der Definition des Sinus und des Tangens folgt:

$$\sin\left(\frac{\alpha}{2}\right) = \frac{L}{r + \varepsilon} \quad \text{und} \quad \tan\left(\frac{\alpha}{2}\right) = \frac{L}{r}$$

$$\Leftrightarrow \varepsilon = \frac{L}{\sin\left(\frac{\alpha}{2}\right)} - r \quad \text{und} \quad r = \cot\left(\frac{\alpha}{2}\right) L$$

Die beiden Gleichung können zusammengefasst werden, so dass sich folgende Bedingungen ergeben:

$$\varepsilon = L \cdot \frac{1 - \cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} \quad \text{bzw.} \quad L = \varepsilon \cdot \frac{\sin\left(\frac{\alpha}{2}\right)}{1 - \cos\left(\frac{\alpha}{2}\right)}$$

Damit ergibt sich für den Radius die Gleichung:

$$r = L \cdot \cot\left(\frac{\alpha}{2}\right) = L \cdot \sqrt{\frac{1 - \cos(\alpha)}{1 + \cos(\alpha)}} = L \cdot \frac{\sin(\alpha)}{1 + \cos(\alpha)} = L \cdot \frac{P_{1,x}}{P_{1,y}}$$

Der Faktor zum Umrechnung von L und ε kann mit Hilfe trigonometrischer Umformungen vereinfacht werden.

$$\frac{1 - \cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} = \frac{1 - \sqrt{\frac{1 - \cos(\alpha)}{2}}}{\sqrt{\frac{1 + \cos(\alpha)}{2}}} = \frac{\sqrt{2} - \sqrt{1 - \cos(\alpha)}}{\sqrt{1 + \cos(\alpha)}} = \frac{\sqrt{1 + \cos(\alpha)} - \sin(\alpha)}{1 + \cos(\alpha)}$$

Durch den Vergleich mit dem symmetrischen Hermite-Problem in Standardform ergibt sich dann die folgende Notation:

$$\frac{1 - \cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} = \frac{\sqrt{P_{1,x}} - P_{1,y}}{P_{1,x}}$$

Die vorherigen Überlegungen werden nun im nachfolgenden Satz zusammengefasst.

Satz. Es sei ein symmetrisches Hermite-Problem $(P_0, \vec{t}_0, P_1, \vec{t}_1, S, L)$ gegeben. Ohne Einschränkung der Allgemeinheit liegt es in der Standardform

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} L + L \cdot \cos(\alpha) \\ L \cdot \sin(\alpha) \end{pmatrix}, \begin{pmatrix} \cos(\alpha) \\ \sin(\alpha) \end{pmatrix}, \begin{pmatrix} L \\ 0 \end{pmatrix}, L \right\}$$

mit $L \in \mathbb{R}^{>0}$ und $\alpha \in (-\pi; \pi]$ vor.

Dann kann ein Kreisbogen gefunden werden, der die Punkte P_0 und P_1 verbindet, die gleichen Tangentenrichtungen in den beiden Punkten besitzt.

Für die Kreisbogen gilt:

$$r = L \cdot \left| \frac{P_{1,x}}{P_{1,y}} \right|; \quad \phi_0 = -\text{sign}(\alpha) \cdot \frac{\pi}{2}; \quad \phi_1 - \phi_0 = \text{sign}(\alpha) \cdot \pi - \alpha = \beta;$$

$$M = P_0 + \text{sign}(\alpha) \cdot r \cdot \vec{t}_0^\perp = \text{sign}(\alpha) \cdot r \cdot \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

Aus der Vorgabe des Abstand L kann, wie in Satz 23.1 dargestellt, der maximale Fehler berechnet werden. Gemäss der Herleitung ist es auch möglich, den maximalen Fehler ε vorzugeben und daraus den maximalen Abstand L zu bestimmen.

Satz. Es sei ein symmetrisches Hermite-Problem $(P_0, \vec{t}_0, P_1, \vec{t}_1, S, L)$ gegeben. Ohne Einschränkung der Allgemeinheit liegt es in der Standardform

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} L + L \cdot \cos(\alpha) \\ L \cdot \sin(\alpha) \end{pmatrix}, \begin{pmatrix} \cos(\alpha) \\ \sin(\alpha) \end{pmatrix}, \begin{pmatrix} L \\ 0 \end{pmatrix}, L \right\}$$

mit $L \in \mathbb{R}^{>0}$ und $\alpha \in (-\pi; \pi]$ vor.

- a) Es sei der maximale Fehler ε vorgegeben. Der maximale Abstand L , für den ein Kreisbogen gemäss Satz 23.1 existiert, der den Fehler berücksichtigt, ist gegeben durch:

$$L(\varepsilon, \alpha) = \varepsilon \cdot \frac{P_{1,x}}{\sqrt{P_{1,x}} - P_{1,y}} = \varepsilon \cdot \frac{L + L \cdot \cos(\alpha)}{\sqrt{L + L \cdot \cos(\alpha)} - L + L \cdot \cos(\alpha)}$$

- b) Bei der Vorgabe des Abstands L ergibt sich der folgende, maximale Fehler:

$$\varepsilon(L, \alpha) = L \cdot \frac{\sqrt{P_{1,x}} - P_{1,y}}{P_{1,x}} = L \cdot \frac{\sqrt{L + L \cdot \cos(\alpha)} - L + L \cdot \cos(\alpha)}{L + L \cdot \cos(\alpha)}$$

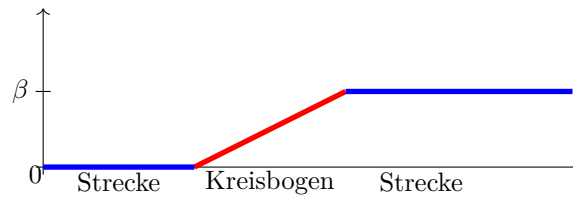


Abbildung 23.2.: Winkeländerung bei einer Verrundung mit einem Kreisbogen

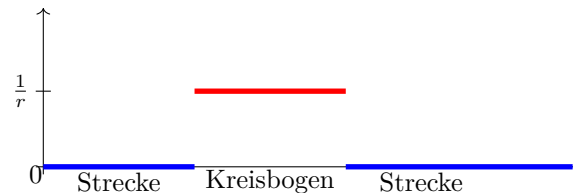


Abbildung 23.3.: Krümmungsverlauf bei der Verrundung mit einem Kreisbogen

Aus diesen Überlegungen ergeben sich Einschränkungen für den Einsatz dieser Strategie, die in der folgenden Bemerkung zusammengefasst sind.

Bemerkung. Folgende Bedingungen zum Anwenden der Strategie eines Kreises müssen erfüllt sein:

a)

$$P_0 \neq P_1$$

b)

$$\vec{t}_0 = -\vec{t}_1 \Leftrightarrow \alpha = 0$$

c)

$$\vec{t}_0 = \vec{t}_1 \Leftrightarrow \alpha = \pi$$

23.2. Bewertung

Die Verrundung mittels eines Kreisbogens führt zu einem stetigen Verlauf der Winkeländerung. Die Abbildung 23.2 stellt dies dar.

Durch die Verrundung mit einem Kreisbogen wird ist der Krümmungsverlauf der Kurve nicht stetig. Die Abbildung 23.3 zeigt, dass die Krümmung im Bereich der Strecken 0 ist und auf dem Kreisbogen konstant mit dem Wert $\frac{1}{r}$, wobei r der Radius des eingesetzten Kreisbogens ist. Aufgrund der Formel $a = \frac{v^2}{r}$ für eine Bewegung auf einem Kreisbogen weist der Beschleunigungsverlauf bei einer konstanten Bahngeschwindigkeit Stetigkeitssprüngen an den Übergängen aus.

In Abhängigkeit der Winkeländerung β an der Ecke S und der Toleranz ε kann die maximale Länge der Verkürzung der Strecken berechnet werden. Die Abbildung 23.4 zeigt das zugehörige Diagramm, wobei die Toleranz ε auf den Wert 1 gesetzt ist.

Der Bereich, der zur Verfügung gestellt wird, kann auch vorgegeben werden. In Abhängigkeit des Abstands L vom Eckpunkt S kann die Abweichung ε berechnet

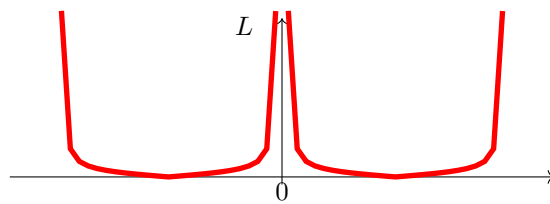


Abbildung 23.4.: Maximaler Bereich für die Verrundung mit einem Kreisbogen - $L(\varepsilon = \text{const}, \alpha)$

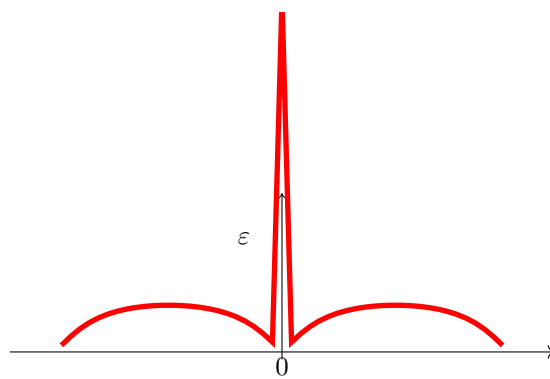


Abbildung 23.5.: Abweichung bei Vorgabe des Abstands L bei der Verrundung mit einem Kreisbogen

werden. Die Abbildung 23.5 stellt die Funktion in Abhängigkeit von L und des Winkels α dar.

Die Abbildungen 23.4 und 23.5 zeigen die Kurven der Länge in Abhängigkeit der Winkeländerung bei einer festen Toleranz und den Fehler in Abhängigkeit der Winkeländerung bei vorgegebenen Länge. Falls die Winkeländerung minimal ist, dann ist der Fehler oder die Länge L extrem. In diesem Fall ist eine Verrundung mittels eines Kreisbogens nicht möglich. Um eine sinnvolle Strategie zu entwickeln, muss eine minimale Winkeländerung vorgegeben werden, ab der eine Verrundung mit einem Kreisbogen erlaubt ist. Ebenso ist eine maximale Winkeländerung zu definieren. Denn bei einer Winkeländerung von $\pm\pi$ handelt sich um eine Kehre. in diesem Fall ist eine Verrundung auch nicht möglich.

23.3. Beispiele

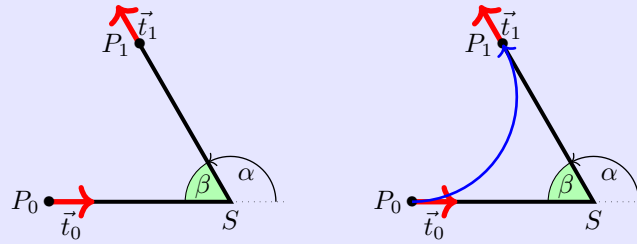
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{2}{3}\pi\right) \\ 6.0 \cdot \sin\left(\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{2}{3}\pi\right) \\ \sin\left(\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = \frac{2}{3}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ 3,4641 \end{pmatrix}; \quad r = 3,46412; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{2}{3}\pi$$



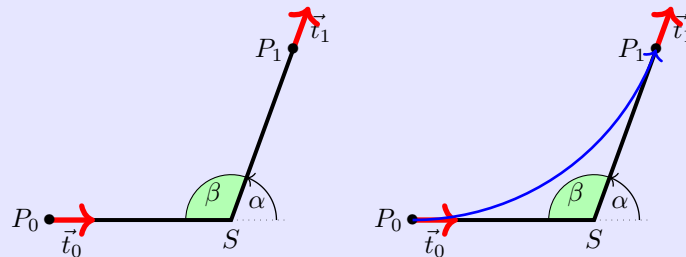
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{7}{18}\pi\right) \\ 6.0 \cdot \sin\left(\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{7}{18}\pi\right) \\ \sin\left(\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = \frac{7}{18}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ 8,5689 \end{pmatrix}; \quad r = 8,5689; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{7}{18}\pi$$



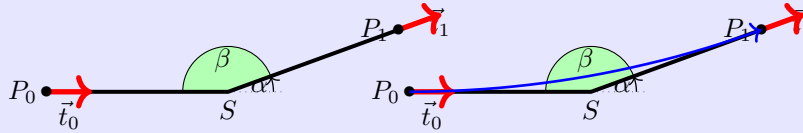
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{1}{9}\pi\right) \\ 6.0 \cdot \sin\left(\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{1}{9}\pi\right) \\ \sin\left(\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = \frac{1}{9}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ 34,0277 \end{pmatrix}; \quad r = 34,0277; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{1}{9}\pi$$

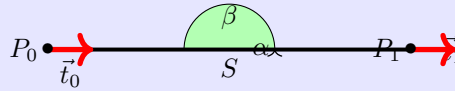


Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 12.0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = 0$ gegeben.

Hier existiert kein Verrundungskreisbogen.



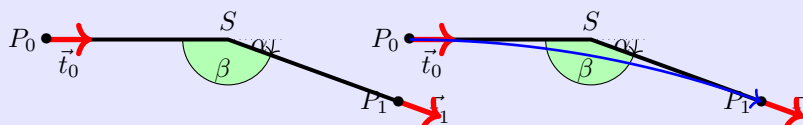
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(-\frac{1}{9}\pi\right) \\ 6.0 \cdot \sin\left(-\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(-\frac{1}{9}\pi\right) \\ \sin\left(-\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = -\frac{1}{9}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ -34,0277 \end{pmatrix}; \quad r = 34,0277; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{1}{9}\pi$$



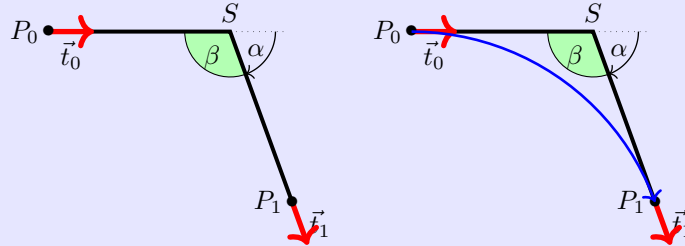
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(-\frac{7}{18}\pi\right) \\ 6.0 \cdot \sin\left(-\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(-\frac{7}{18}\pi\right) \\ \sin\left(-\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = -\frac{7}{18}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ -8,5689 \end{pmatrix}; \quad r = 8,5689; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{7}{18}\pi$$



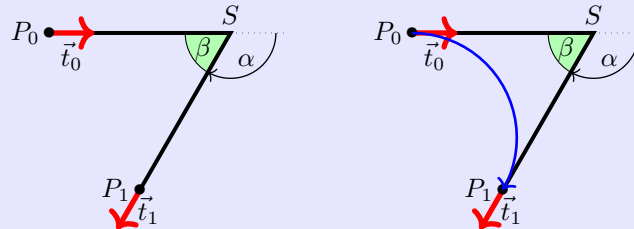
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(-\frac{2}{3}\pi\right) \\ 6.0 \cdot \sin\left(-\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(-\frac{2}{3}\pi\right) \\ \sin\left(-\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = -\frac{2}{3}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ -3,4641 \end{pmatrix}; \quad r = 3,46412; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{2}{3}\pi$$



24. Maple-Dateien

[Wat17c; Wat17d; Wat17b; Wat17e; Wat17a; Wat17f]

24.1. Geometrien mit Maple

Die vorgestellten Algorithmen können gut mit Hilfe von Geometrien dargestellt werden. Dabei können zwei Aspekte untersucht werden. Einerseits ist der Aufwand für eine Umsetzung, die Rechengenauigkeit und die Stabilität eines Algorithmus interessant; andererseits sollen die Verfahren hinsichtlich ihres Nutzens bewertet und verglichen werden. Hierzu wurden Module für das Formelmanipulationssystem Maple [Wat17c] entwickelt.

Maple bietet die Umgebung, Algorithmen einfach und schnell zu implementieren. Darüber hinaus ist die grafische Darstellung mit einfachen Mitteln möglich. Allerdings ist eine einfache Arbeitsmappe von Maple nicht für sehr große Software-Projekte ausgelegt. Hier muss auf die Möglichkeit der Erstellung und Nutzung von Bibliotheken zurückgegriffen werden. Einerseits bietet Maple die Möglichkeit, eigene Bibliotheken, so genannte Module, zu erstellen. Auf diese Weise bleibt man innerhalb der Umgebung und der Syntax von Maple. Dieser Weg wird hier weiter verfolgt. Eine weitere Möglichkeit ist die Verwendung von Bibliotheken, die mittels einer höheren Programmiersprache, z.B. C++, erstellt wurde, den so genannten DLLs.

Im weiteren wird die Erstellung und Verwendung einer Testumgebung mit Hilfe von Module beschrieben. Zunächst werden die Geometrieelemente beschrieben, die in verschiedenen Module abgelegt sind, und deren Verwendung. Damit eigene Erweiterungen und Ergänzungen möglich sind, wird dann der Aufbau und die Verwendung eines Moduls in Maple erläutert.

24.2. Verwendete Geometrieelemente

Da es sich um eine Testumgebung für die vorgestellten Algorithmen, werden auch nur Geometrien verwendet, die beschrieben wurden.

- Punkte (**MPoint**)
- Strecken (**MLine**)
- Kreisbögen (**MArc**)
- Bézier-Kurven (**Bezier**)
- Polygonzüge (**MPolygon**)
- Geometrieliste (**MGeoList**)
- Hermite-Probleme (**MHermiteProblem**)
- Symmetrische Hermite-Probleme (**MHermiteProblemSym**)

Für jedes Geometrieelement wurde ein entsprechendes Modul angelegt, dessen Name in der obigen Liste angegeben ist. Die Auflistung, welche Funktionen zur Verfügung stehen, wird in einem weiteren Abschnitt dargestellt.

Bei der Umsetzung eines solchen Projekts wird schnell deutlich, dass bei einer Verwendung von mehreren Geometrieelementen eine klare Datenstruktur und ein wohldefinierter Zugriff auf die Daten unerlässlich ist. Zum Beispiel bei der Darstellung von Geraden ist die Entscheidung zu treffen, ob die Darstellung Punkt-Richtung oder mittels zweier Punkte erfolgen soll. Die Auswahl erfolgt im Allgemeinen in Abhängigkeit des Anwendungsfalls. Hier wird der Weg beschritten, dass aufgrund eines wohldefinierten Zugriffs auf die Daten die Verwendung beider Darstellungen möglich ist.

24.3. Aufbau der Datenstruktur

Die Idee ist, eine möglichst einheitliche und einfache Datenstruktur zu verwenden. Maple bietet zwar die Möglichkeit, Objekte zu definieren. Allerdings ist die Verwendung innerhalb einer prozeduralen Umgebung schwierig. Daher werden alle Daten grundsätzlich als Listen dargestellt. Das erste Listenelement enthält dabei stets eine Kennung des Elements `??`. Dann folgen die Daten, die wiederum Geometrieelemente sein können. Es ist zu beachten, dass der direkte Zugriff auf die Daten nicht unterbunden werden kann; hier ist der Anwender in der Pflicht.

Der Zugriff auf die Daten soll ausschließlich über Prozeduren eines Moduls erfolgen. Im folgenden ist beispielhaft die Datenstruktur für Punkte zu sehen:

```
[MVPOINT, [x, y]]
```

Die Erstellung eines Punkts erfolgt dann über eine Prozedur `New`:

```
NeuerPunkt := MPoint:-New(10,15);
```

Beim Aufruf in der Testdatei wird dann folgendes ausgegeben:

```
TestP0 := [PPoint", [10,15]]
```

Diese Datenstrukturen werden für die Berechnung der Geometrien verwendet. Mit ihnen wird in Funktionen bzw. Prozeduren gearbeitet. Anders als Variablen werden die Datenstrukturen und Prozeduren hier global und nicht lokal definiert. Unter dem Befehl `export` werden die Prozeduren zu Beginn des Moduls deklariert und sind so auch außerhalb des Moduls verwendbar. Soll beispielsweise eine Datenstruktur aus `MPoint` in einem anderen Modul oder außerhalb der Module verwendet werden, wird sie wie folgt aufgerufen:

```
P0 := MPoint:-New(x,y);
```

Zusätzlich zu den Modulen für die Geometrien gibt es ein Modul (`MConstant`) zum Speichern von Konstanten und Namen. Dort ist für `MVPOINT` z.B. „Point“ gespeichert oder z.B. eine Konstante für den Vergleich auf Null für reelle Zahlen.

Zuletzt gibt es die Testdatei, welche kein Modul ist, in der die Module geladen, aufgerufen und in ihrer Funktion getestet werden. Zu dieser Datei später im Punkt „1.4 Maple Testdatei“ mehr.

24.4. Struktur eines Moduls

In dem folgenden Abschnitt geht es um den Zweck von Modulen und deren Struktur eingegangen.

Es geht in dem Projekt darum die oben genannten Geometrien in einer Datenstruktur zu erfassen und sie in Maple darzustellen. Für die letztendliche Darstellung sind die Berechnungen und Formeln, die verwendet werden müssen, allerdings hinderlich. Module eignen sich sehr gut dafür die Berechnungen, die in Funktionen erfolgen,

zusammenzufassen und zu verstecken. So kann in Maple auf die wichtigen Funktionen zugegriffen werden, ohne dass man sieht, was in diesen steht.

Beispiel:

Die Funktion **Angle** aus dem Modul **MPoint**: Funktion zur Berechnung eines Winkels zwischen Ortsvektor und x-Achse:

```
Angle := proc(P)
    local alpha, x, y;
    x := GetX(P);
    y := GetY(P);
    alpha := 0;
    if abs(x) < 0.00001
    then
        alpha := 3*Pi/2;
    else
        alpha := Pi/2;
    end if;
else
    alpha := arctan(y/x);
    if x < 0
    then
        alpha := alpha + Pi;
    else
        if y < 0
        then
            alpha := alpha + 2*Pi;
        end if;
    end if;
    end if;
    return factor(alpha);
end proc;
```

Diese Funktion ist lang, stellt aber nur einen kleinen Teil des Programms für die Darstellung dar, um die es geht. Deshalb steht sie in dem Modul und kann so extern mit einem einzigen Befehl aufgerufen werden:

```
Winkel := MPoint:-Angle(P)
```

Im Folgenden Abschnitt wird die Struktur eines Moduls beschrieben. Da Maple hier sehr vielfältige Möglichkeiten bietet, findet eine Beschränkung statt. Es wird nur die Struktur der verwendeten Module, hier anhand des Moduls **MPoint**, beschrieben. Für tiefergehende Möglichkeiten wird hier auf das Handbuch von Maple [Wat17c] verwiesen.

Struktur:

Das Modul muss zunächst gestartet werden. Dies erfolgt mit dem Namen (hier immer ein groSSes M und die Geometrie) des Moduls und dem folgenden Befehl:

```
MPoint := module()
```

Danach müssen, ähnlich wie in Prozeduren, die Variablen und Funktionen definiert werden. Dabei kann man entweder lokal oder global deklarieren. Werden die Variablen

oder Prozeduren nur in dem Modul gebraucht und verändert, erfolgt die Deklaration mit dem Befehl `local` wie folgt:

```
local Variablenamen, mit, Komma, getrennt;
```

Sollen die Variablen oder Prozeduren auch auSSerhalb des Moduls verwendbar sein, wird mit `export` dies definiert:

```
export Funktionsnamen, mit, Komma, getrennt;
```

Daraufhin folgt die Angabe der Optionen:

```
option package;
```

die Beschreibung des Moduls:

```
description "Selbstgewählte Modulbeschreibung z.B. Modul für Punkte ";
```

und die Initialisierung des Moduls:

```
ModuleLoad := proc
    MVPPOINT:=MConstant:-GetPoint();
    print("Modul MPoint ist geladen");
end proc;
```

Ab hier wird programmiert wie sonst in Maple auch. Man verwendet die vorher definierten Prozedur- und Variablenamen und programmiert mit Befehlen die man auSSerhalb eines Moduls in Maple auch verwendet. Wichtig zu wissen ist, dass bei Modulen jede Funktion ständig abgerufen werden kann, die Reihenfolge der Funktionen also keine Rolle spielt.

Um zuletzt das Modul zu beenden, nutzt man den folgenden Befehl:

```
end module;
```

24.4.1. Genereller Aufbau eines Moduls

Zusammenfassend haben die Module also folgendes Gerüst:

```
Modulname := module()

    local Namen, mit, Komma, getrennt;

    export Namen, mit, Komma, getrennt;

    global Namen, mit, Komma, getrennt;1

    option package;

    description "Selbstgewählte Modulbeschreibung";

    ModuleLoad := proc()2
        MVPPOINT:=MConstant:-GetPoint();
        print("Modul MPoint ist geladen");
    end proc;

    Procedure1 := proc (Übergabeparameter)
```

¹möglich, aber in den Modulen nicht verwendet

²Beispiel `MPoint`

```
        inhalt;  
    end proc;  
  
    Procedure2 := proc (Übergabeparameter)  
        inhalt;  
    end proc;  
  
    ...  
end module;
```

24.4.2. Sicherung eines Moduls

Die Verwaltung von Module wird von Maple automatisch durchgeführt. Da aber die Module weitergegeben werden und einzeln zu bearbeiten sind, müssen einige Einstellungen vorgenommen werden. Daher wird bei jeder Maple-Datei des Projekts folgender Präfix verwendet:

```
1 restart;
2 with(LibraryTools);
3 lib := C:/FH/Tools/Maple/MyLibs/Blending.mla";
4 march('open', lib);
5 ThisModule := 'MArc';
```

Die 1. Zeile initialisiert das System. Die nächsten 3 Zeile ermöglichen das Arbeiten mit Archiven. In der zweiten Zeile werden die Werkzeuge geladen, so dass die Variable **lib** belegt werden kann. Alle Module werden in einem Archiv abgelegt; in diesem Beispiel ist es die Datei **Blending.mla** im Verzeichnis **C:/FH/Tools/Maple/MyLibs/**. Das Kommando **ThisModule := 'MArc';** enthält den Namen des aktuellen Moduls.

Ein Modul wird dann mittels des Befehls **savelib('ModuleName')** gespeichert. Es wird dann eine Datei **ModuleName.mla** angelegt. Je nach Konfiguration von Maple ist der eingestellte Pfad, wo die Datei automatisch angelegt wird, nicht beschreibbar. Dann kann man das System so konfigurieren, dass die mla-Datei im aktuellen Verzeichnis oder in einem Verzeichnis der eigenen Wahl abgelegt wird.

Falls der obige Vorspann für eine Maple-Datei verwendet wird, genügt zum Speichern des Moduls

```
savelib(ThisModule, lib);
```

24.4.3. Verwendung eines Moduls

Ein Modul, das abgespeichert wurde, kann nun in anderen Maple-Worksheets verwendet werden. Dazu wird der Befehl

```
with(ModuleName)
```

verwendet. Falls man einen speziellen Pfad verwendet hat, so kann Maple die Datei **ModuleName.mla** nicht finden. Dann muss der Pfad bekannt gemacht werden, z.B.:

```
savelibname := c:/Maple/MyLibs";", savelibname;
```

Nun kann man auf die Prozeduren des Moduls zugegriffen werden. Eine Übersicht über alle exportierten Prozeduren wird durch den Befehl

```
Describe(ModuleName)
```

angezeigt. Dabei wird eine List der Namen inklusiver der Namen der Übergabeparameter dargestellt. Falls eine Prozedur mit der Feld **description** ausgestattet ist, so wird dieser Text zusätzlich wiedergegeben.

24.4.4. Erstellung eines Maple-Moduls

Die Erstellung eines eigenen Moduls ist schnellst erledigt, falls man den obigen Rahmen verwendet. Allerdings sollte man vorher einige Überlegungen anstellen und einen Standard pflegen.

Bevor ein eigenes Modul erstellt wird, sollte das Datenmodell und die zugehörigen Prozeduren erarbeitet sein. Ein Programmablaufplan leistet hier gute Dienste. Die zentrale Aufgabe des Moduls ist in der Regel schnell ermittelt. Zusätzlich sollten aber folgende Regeln eingehalten werden.

Regel 1 Grundsätzlich erhält sowohl das Modul als auch jede Prozedur eine Beschreibung. Diese beschränkt sich nicht auf das optionale Argument **description**,

sonder wird jeder Prozedur vorangestellt. Die Beschreibung enthält grundsätzlich die Beschreibung der Aufgabe. Dabei werden auch die Voraussetzung genannt bzw. eine Fehlerbehandlung beschrieben. Dann folgt die Beschreibung aller Eingabeparameter, deren Funktion und deren Datenstruktur. Der Rückgabewert wird anschließend beschrieben.

Regel 2 Jedes Modul erhält eine Prozedur `Version()`, die die aktuelle Versionsnummer zurückliefert.

Regel 3 Daten sind nicht global. Um auf Daten zugreifen zu können, werden Prozeduren `Set*` und `Get*` zur Verfügung gestellt. Auch innerhalb der Prozeduren eines Moduls werden diese Prozeduren verwendet.

Regel 4 Für jede Prozedur wird mindestens eine Testfunktion geschrieben. Anhand der Testfunktion kann die Verwendung und ggfs. Besonderheiten dargestellt werden.

24.5. Funktionen der Module

Im folgendem werden die einzelnen Module aufgeführt. Die Datenstruktur jedes Moduls und alle Funktionen mit zugehöriger Aufgabe werden genannt.

24.5.1. Modul `MPoint`

`MPoint` ist ein Modul für Punkte und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur `New` für einen Punkt ist eine Liste, die wie folgt aufgebaut ist:

```
[MVPOINT, [x,y]]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name `PPoint` und die Elemente sind die x- und die y-Koordinate für einen Punkt. Es wird hier kein Unterschied zwischen Punkt und Vektor gemacht. Ein Punkt kann auch als Vektor vom Koordinatenursprung zum Punkt gesehen werden.

Über die Get-Funktionen `GetX` und `GetY` kann man die Koordinaten des Punktes erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Punkten/Vektoren dann gerechnet werden.

MPoint

E	New	Datenstruktur für einen Punkt
E	GetX	Lesen der x-Koordinate
E	GetY	Lesen der y-Koordinate
E	Angle	Berechnung des Winkels zwischen x-Achse und dem Punkt
E	Add	Errechnet eine Linearkombination zweier Punkte/Vektoren
E	Sub	Errechnet die Differenz zweier Punkte/Vektoren
E	Cos	Errechnet den Cosinus zwischen zwei Vektoren
E	Sin	Errechnet den Sinus zwischen zwei Vektoren
E	Scale	Skaliert einen Vektor mit einem Faktor
E	Perp	Errechnet einen Vektor, der orthogonal zum angegebenen Vektor ist
L	IllustrateXY	Plot Funktion zur Darstellung eines blauen Punktes
E	Illustrate	Darstellung eines blauen Punktes
E	Plot	Darstellung eines grünen Punktes
E	Plot2D	Darstellung eines Punktes mit eigenen Optionen
E	Length	Errechnet den Abstand des Punktes zum Koordinatenursprung
E	Uniform	Normiert den Vektor auf die Länge 1
E	LinetoVector	Errechnet Vektor aus einer Strecke
E	Distance	Errechnet den Abstand zwischen zwei Punkten

24.5.2. Modul MLine

MLine ist ein Modul für Strecken und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für eine Strecke ist eine Liste, die wie folgt aufgebaut ist:

[MVLIN, [P0,P1]]

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „Line“ und die Elemente sind der Start- und der Endpunkt für eine Strecke. Die Datenstruktur **NewPointVector** ist ebenfalls eine Datenstruktur für eine Strecke, allerdings wird die Strecke hier über einen Startpunkt und einen Richtungsvektor definiert.

Über die Get-Funktionen **StartPoint** und **EndPoint** kann man Start- und Endpunkt der Strecke erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Strecken dann gerechnet werden.

MLine

E	New	Datenstruktur für eine Strecke (Zwei-Punkt-Form)
E	NewPointVector	Schaffung einer Strecke (Punkt-Richtungs-Form)
E	StartPoint	Lesen des Startpunktes
E	EndPoint	Lesen des Endpunktes
E	Position	Errechnen eines Punktes der auf der Strecke liegt
E	Plot2D	Darstellung eines Teils der Strecke ausgehend vom Startpunkt
E	Plot2DTangent	Darstellung eines Punktes mit Tangente
E	Plot2DTangentArrow	Darstellung eines Punktes mit Tangente (als Pfeil)
E	LineLine	Errechnung des Schnittpunktes zweier Geraden
E	AngleLineLine	Errechnung des Winkels zwischen zwei Geraden

24.5.3. Modul **MArc**

MArc ist ein Modul für Kreisbögen und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für einen Kreisbogen ist eine Liste, die wie folgt aufgebaut ist:

```
[MVARC, [mx,my,r,phi,alpha]]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name **MArc** und die Elemente sind die x- und y-Koordinate für den Mittelpunkt, der Radius, der Startwinkel und die Winkeländerung für einen Kreisbogen.

Über die Get-Funktionen **GetM**, **GetMX**, **GetMY**, **GetR**, **GetPhi**, und **GetAlpha** kann man die Daten für einen Kreisbogen erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Kreisbögen dann gerechnet werden.

MArc

E	New	Datenstruktur für einen Kreisbogen
E	GetMX	Lesen der x-Koordinate des Mittelpunktes
E	GetMY	Lesen der y-Koordinate des Mittelpunktes
E	GetR	Lesen des Radius
E	GetPhi	Lesen des Startwinkels
E	GetAlpha	Lesen der Winkeländerung
E	GetM	Lesen des Mittelpunktes
E	Position	Errechnung eines Punktes auf dem Kreisbogen
E	Plot2D	Darstellung eines Teils des Kreisbogens ausgehend vom Startwinkel
E	Blend	Errechnung eines Kreisbogens aus einem symmetrischen Hermite-Problem

24.5.4. Modul **MBezier**

Die Datenstruktur **New** für ein Polygon ist eine Liste, die wie folgt aufgebaut ist:

```
[MVBEZIER, [PointList]]
```

Innerhalb des Moduls **MBezier** existieren die in folgender Tabelle aufgeführten Prozeduren.

MBezier

E	New	Manuelle Eingabe von Kontrollpunkten für eine Bézier-Kurve
E	Version	Ausgabe der Version
E	BlendCurvature	Bestimmung von Kontrollpunkten aus symmetrischen Hermite-Problem
E	BlendCurvatureEpsilon	Bestimmung von Kontrollpunkten aus symmetrischen Hermite-Problem mit vorgegebenen Fehler
E	Position	Position auf der Bézier-Kurve
E	GetTheta	Lesen des Winkels
E	GetEpsilon	Lesen der Toleranz
E	GetControlPoint	Lesen der Kontrollpunkte
E	Plot2D	Darstellung der Bézier-Kurve
E	PlotControlPoints	Darstellung aller Kontrollpunkte

24.5.5. Modul **MPolygon**

MPolygon ist ein Modul für Polygonzüge und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für ein Polygon ist eine Liste, die wie folgt aufgebaut ist:

```
[MVPOLYGON, [PointList]]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name **PPolygon** und die Elemente sind beliebig viele Punkte.

Über die Get-Funktionen **GetPoint** und **GetN** kann man die Anzahl der Punkte und die Punkte selbst erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Punkten bzw. dem Polygonzug dann gerechnet werden.

MPolygon

E	New	Datenstruktur für eine Punkteliste
E	GetPoint	Lesen des i-ten Punktes aus der Punkteliste
E	GetN	Bestimmung der Anzahl der Punkte in der Punkteliste
E	Length	Bestimmung der euklidischen Länge des Polygonzuges
E	Position	Berechnung eines Punktes auf dem Polygonzug
E	Tangents	Berechnung der Tangenten
L	Plot2DAll	Plotliste aller Punkte
E	Plot2D	Darstellung aller Punkte als Polygonzug
E	Plot2DTangent	Darstellung des Polygonzuges mit Tangenten
E	PlotPoints	Darstellung aller Punkte

24.5.6. Modul **MGeoList**

MGeoList ist ein Modul für eine Geometrieliste und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für eine Geometrieliste ist eine Liste, die wie folgt aufgebaut ist:

```
[MVGEOLIST, []]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „GeoList“ und die Elemente sind beliebig viele einzelne Geometrien.

Über die Get-Funktionen **GeoGeo** und **GetN** kann man das i-te Element der Liste und die Anzahl der Elemente in der Liste erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Geometrien bzw. der Liste dann gerechnet werden. In den Funktionen **Length**, **Plot2DAll**, **Plot2D** und **Position** werden die Funktionen in sich selber aufgerufen. Dies ist möglich da die Funktionen unabhängig voneinander funktionieren.

MGeoList

E	New	Datenstruktur für eine Geometrieliste
E	Append	Anfügen eines Geometrieelements in die Datenstruktur
E	Prepend	Einfügen eines Geometrieelements als erstes Element der Liste
E	Replace	Ersetzen eines Geometrieelements durch ein Anderes
E	GetN	Bestimmung der Anzahl der Geometrieelemente in der Liste
E	GeoGeo	Lesen des i-ten Geometrieelements
E	Length	Errechnet die euklidische Länge der Geometrieelemente
E	Position	Berechnung eines Punktes auf der Geometrie
L	Plot2DAll	Plot Funktion für die Darstellung der Geometrieelemente
E	Plot2D	Darstellung eines Teils der Geometrieliste ausgehend vom ersten Element

24.5.7. Modul MHermiteProblem

MHermiteProblem ist ein Modul für Hermite-Probleme und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für eine Strecke ist eine Liste, die wie folgt aufgebaut ist:

[MVHERMITEPROBLEM, [P0,T0n,P1,T1n]]

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „HermiteProb“ und die Elemente sind zwei Punkte mit zugehörigen Tangenten (Strecken).

Über die Get-Funktionen **StartPoint**, **EndPoint**, **StartTangent** und **EndTangent** kann man die Punkte und ihre Tangenten erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Daten für das Hermite-Problem dann gerechnet werden.

MHermiteProblem

E	New	Datenstruktur für ein Hermite-Problem
E	StartPoint	Lesen des Startpunktes
E	EndPoint	Lesen des Endpunktes
E	StartTangent	Lesen der Starttangente
E	EndTangent	Lesen der Endtangente
E	Plot2D	Darstellung des Hermite Problems

24.5.8. Modul MHermiteProblemSym

MHermiteProblemSym ist ein Modul für symmetrische Hermite-Probleme und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für ein symmetrisches Hermite-Problem ist eine Liste, die wie folgt aufgebaut ist:

[MVHERMITEPROBLEMSYM, [P0,T0n,P1,T1n,S,L]]

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „SymHermiteProb“ und die Elemente sind zwei Punkte mit zugehörigen Tangenten, ihr Schnittpunkt, und der Abstand der Punkte zum Schnittpunkt.

Über die Get-Funktionen **StartPoint**, **EndPoint**, **StartTangent**, **EndTangent**, **CrossPoint** und **ParameterL** kann man die Daten erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Daten für das symmetrische Hermite-Problem dann gerechnet werden.

MHermiteProblemSym

E	New	Datenstruktur für ein Symmetrisches Hermite-Problem
E	StartPoint	Lesen des Startpunktes
E	EndPoint	Lesen des Endpunktes
E	StartTangent	Lesen der Starttangente
E	EndTangent	Lesen der Endtangente
E	ParameterL	Lesen des Abstandes
E	CrossPoint	Lesen des Schnittpunktes
E	Plot2D	Darstellung des symmetrischen Hermite Problems
E	Create	Erzeugung eines Sym. Hermite-Problems durch 3 Punkte
E	BlendArc	Verrundung des Eckpunktes durch einen Kreisbogen

24.5.9. Modul **MConstant**

MConstant ist ein Modul zum Speichern von festen Konstanten und Namen zur Kennung der Datenstrukturen. In der lokal deklarierten Funktionen, beginnend mit CV, werden die Konstanten zur Deklaration der verschiedenen Geometrien definiert. Dies sind Namen zur Erkennung der Geometrieelemente in der Testdatei. In den global deklarierten Funktionen, beginnend mit Get, erfolgt die Rückgabe der Namen zur Kennung der Geometrieelemente.

MConstant

L	NULLEPS	Konstante zum Vergleich auf Null
L	CVPOINT	Konstante für Geometrieelemente: Point
L	CVLINE	Konstante für Geometrieelemente: Line
L	CVARC	Konstante für Geometrieelemente: Arc
L	CVPOLYGON	Konstante für Geometrieelemente: Polygon
L	CVGEOLIST	Konstante für Geometrieelemente: GeoList
L	CVHERMITEPROBLEM	Konstante für Geometrieelemente: HermiteProb
L	CVHERMITEPROBLEMSYMMETRIC	Konst. für Geometrieelemente: SymHermiteProb
L	CVBIARC	Konst. für Geometrieelemente: Biarc
E	GetNullEps	Rückgabe des Nullvergleichs
E	GetPoint	Kennung für Punkte
E	GetLine	Kennung für Strecken
E	GetArc	Kennung für Kreisbögen
E	GetPolygon	Kennung für Polygone
E	GetGeoList	Kennung für Geometrielisten
E	GetHermiteProblemSymmetric	Kennung für symmetrische Hermite-Probleme
E	GetHermiteProblem	Kennung für Hermite-Probleme
E	GetBiarc	Kennung für Biarcs

24.5.10. Modul **MGeneralMath**

MGeneralMath ist ein Modul für allgemeine mathematische Funktionen und arbeitet mit den Datenstrukturen und Prozeduren.

MGeneralMath

E	MPoint	Datenstruktur für einen Punkt
E	MPointX	Lesen der x-Koordinate für einen Punkt
E	MPointY	Lesen der y-Koordinate für einen Punkt
L	MPointIllustrateXY	Plotstruktur für einen blauen Punkt
E	MPointIllustrate	Darstellung eines blauen Punktes
E	MPointPlot	Darstellung eines grünen Punktes
E	MLine	Datenstruktur für eine Strecke
E	MLineStartPoint	Lesen des Startpunktes für eine Strecke
E	MLineEndPoint	Lesen des Endpunktes für eine Strecke
E	MPointOnLine	Berechnung eines Punktes auf der Strecke
E	MLinePlot2D	Darstellung des Teils einer Strecke beginnend beim Startpunkt
E	MLineLine	Errechnen des Schnittpunktes zweier Strecken

24.5.11. Modul Biarc**Datenstruktur**

Voraussetzungen und Festlegungen:

Der Biarc soll ein dafür verwendet werden ein Hermite-Problem zu lösen. Ein Hermite Problem ist wie folgt definiert:

Gegeben seien zwei Punkte P_0 und P_1 mit zugehörigen normierten Tangenten $\vec{t}_0$ und $\vec{t}_1$. Der Biarc muss die Punkte so verbinden, dass die Tangenten des Start- und des Endpunktes des Biarcs mit den Tangenten des Hermites-Problems übereinstimmen.

Datenstruktur:

Die Datenstruktur **New** für einen Biarc muss zwei Kreisbögen enthalten.

Benötigt wird also die Datenstruktur für einen Kreisbogen: **MArc:-New**

Diese enthält wiederum die Koordinaten für den Mittelpunkt, den Radius, den Startwinkel und die Winkeländerung.

24.5.12. Modul MBiarc

MBiarc ist ein Modul für Biarcs und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für einen Biarc ist eine Liste, die wie folgt aufgebaut ist:

[MVBIArc, [Arc0, Arc1]]

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „Biarc“ und die Elemente sind zwei Kreisbögen.

Über die Get-Funktionen **GetArc0** und **GetArc1** kann man die beiden Kreisbögen für den Biarc erfassen. Mit den unten aufgeführten Funktionen kann man die Daten für den Biarc errechnen.

MBiarc

E	New	Datenstruktur für einen Biarc
E	GetArc0	Lesen des ersten Kreisbogens
E	GetArc1	Lesen des zweiten Kreisbogens
E	Circle	Berechnung des Kreises K_J aus den Daten des Hermite-Problems
E	Plot2DCircle	Darstellung des Kreises K_J
E	angle	Errechnet den Winkel vom Kreismittelpunkt zu Punkten auf dem Kreis
E	Plot2D	Darstellung des Biarcs
E	ConnectionPoint	Errechnung des Verbindungspunktes J (Equal Chord)
E	TangentTj	Errechnen der Tangente an J
E	Tangent Biarc	Drehung von Tj für den Biarc
E	BiarcCenter	Errechnen der Mittelpunkte der Kreisbögen des Biarcs
E	Biarc	Errechnen des Biarcs
E	Position	Ermitteln eines Punktes auf dem Biarc
E	Blend	Errechnen des Biarcs nur aus dem Hermite-Problem

24.6. Programmablaufplan**24.6.1. Gesamauf****24.6.2. Verrundung der Kurve**

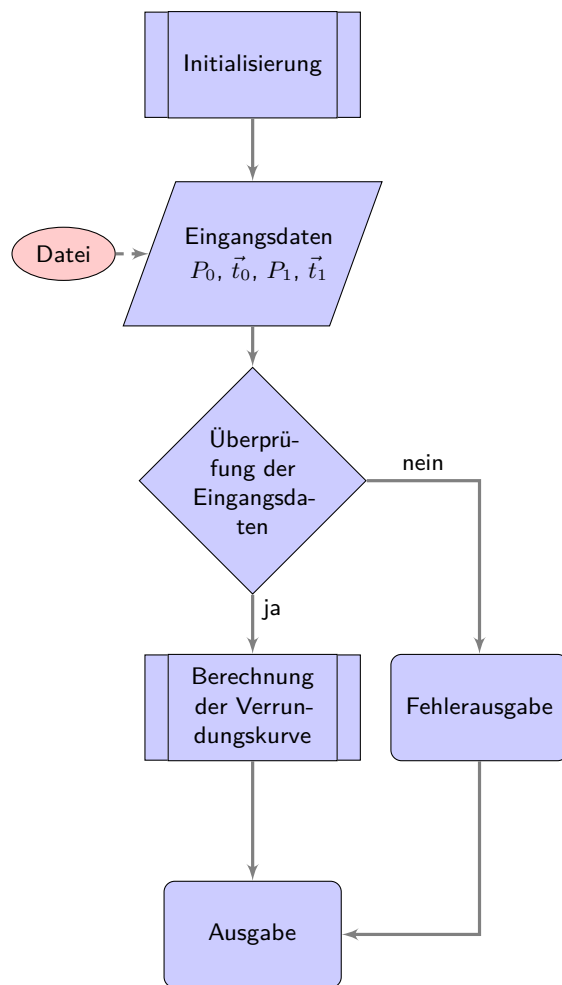


Abbildung 24.1.: Programmablaufplan „Eckenverrundung“

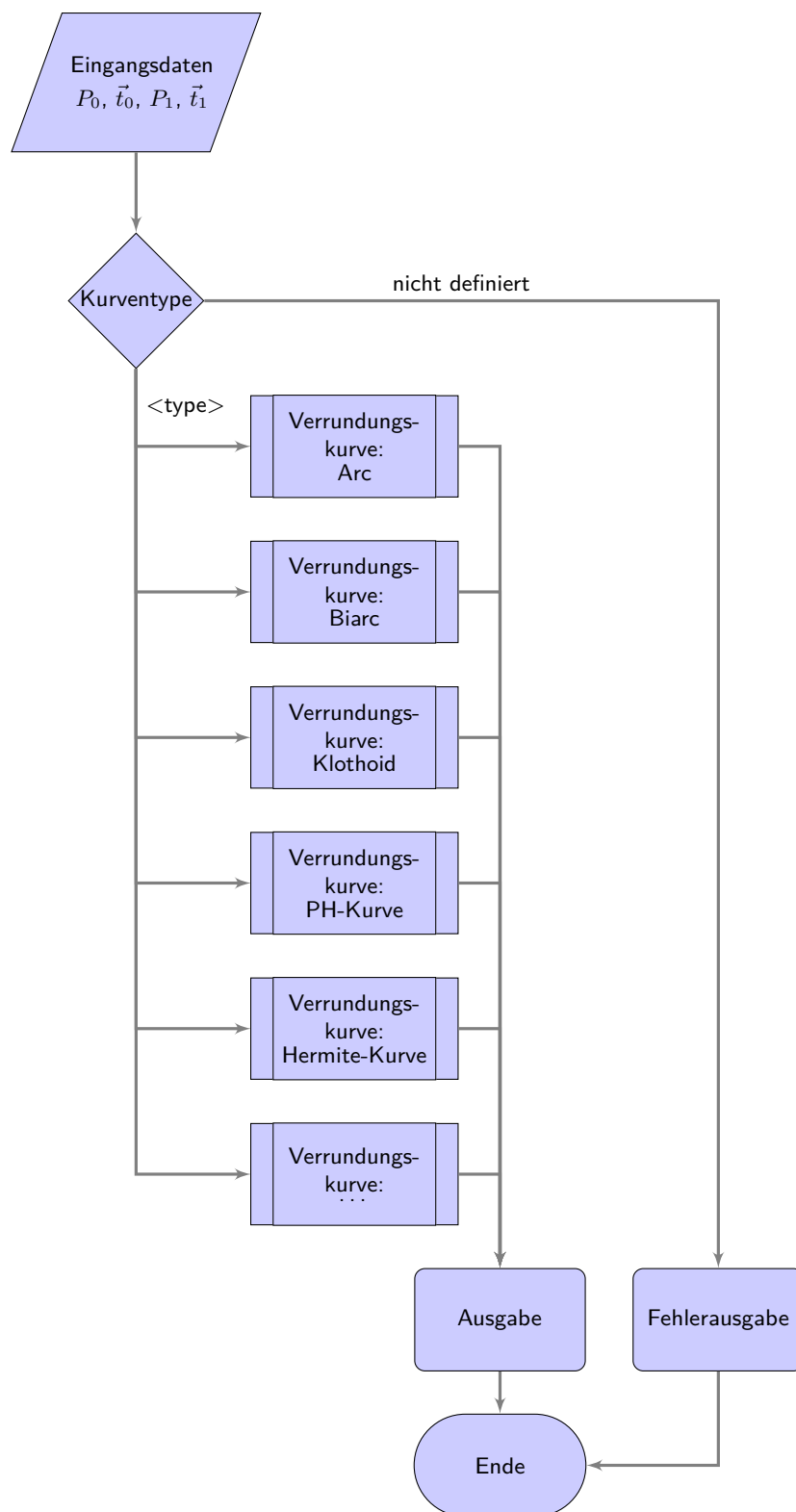


Abbildung 24.2.: Programmablaufplan „Auswahl der Verrundungsstrategien“

25. Representation of Python Programs

It is possible to integrate a part into the document, see 25.1. This is the most elegant method and is also preferable. Individual lines and line ranges can also be selected in the list of options. If a file is integrated, it is always as up-to-date as the document. It may also be useful to integrate program lines: `redLED = pyb.LED(1)`.

Attention: The integration of programs with the help of images is pointless.

Listing 25.1.: The program “Hello World” in Python for microcontroller boards is inserted from the file `Blink.py`.

```

1000 %%
1001 %% This is file 'rename-to-empty-base.tex',
1002 %% generated with the docstrip utility.
1003 %%
1004 %% The original source files were:
1005 %%
1006 %% fileerr.dtx (with options: 'return')
1007 %%
1008 %% This is a generated file.
1009 %%
1010 %% The source is maintained by the LaTeX Project team and bug
1011 %% reports for it can be opened at https://latex-project.org/bugs/
1012 %% (but please observe conditions on bug reports sent to that address!)
1013 %%
1014 %%
1015 %% Copyright (C) 1993–2025
1016 %% The LaTeX Project and any individual authors listed elsewhere
1017 %% in this file.
1018 %%
1019 %% This file was generated from file(s) of the Standard LaTeX ‘Tools
1020 %% Bundle’.
1021 %%
1022 %%
1023 %% It may be distributed and/or modified under the
1024 %% conditions of the LaTeX Project Public License, either version 1.3c
1025 %% of this license or (at your option) any later version.
1026 %% The latest version of this license is in
1027 %% https://www.latex-project.org/lppl.txt
1028 %% and version 1.3c or later is part of all distributions of LaTeX
1029 %% version 2005/12/01 or later.
1030 %%
1031 %% This file may only be distributed together with a copy of the LaTeX
1032 %% ‘Tools Bundle’. You may however distribute the LaTeX ‘Tools Bundle’
1033 %% without such generated files.
1034 %%
1035 %% The list of all files belonging to the LaTeX ‘Tools Bundle’ is
1036 %% given in the file ‘manifest.txt’.
1037 %%
1038 %% \message{File ignored}
1039 %% \endinput
1040 %%
1041 %% End of file 'rename-to-empty-base.tex'.

```

`../Code/HelloWorld/Blink.py`

26. Representation of Programs written for Arduino Boards

It is possible to integrate a part into the document, see 26.1. This is the most elegant method and is also preferable. Individual lines and line ranges can also be selected in the list of options. If a file is integrated, it is always as up-to-date as the document.

Attention: The integration of programs with the help of images is pointless.

Listing 26.1.: The program “Hello World” for an Arduino microcontroller board is inserted from the file `Blink.ino`.

```

1000 /**
1001  *
1002  *   @file Blink.ino
1003  *
1004  *   @brief Simple program for testing the configuration
1005  *
1006  *   Turns an LED on for one second, then off for one second, repeatedly.
1007  *
1008  *   Most Arduinos have an on-board LED you can control. On the UNO, MEGA
1009  *   and ZERO
1010  *   it is attached to digital pin 13, on MKR1000 on pin 6. LED_BUILTIN is
1011  *   set to
1012  *   the correct LED pin independent of which board is used.
1013  *   If you want to know what pin the on-board LED is connected to on your
1014  *   Arduino
1015  *   model, check the Technical Specs of your board at:
1016  *
1017  *   https://www.arduino.cc/en/Main/Products
1018  *
1019  *   modified 8 May 2014
1020  *   by Scott Fitzgerald
1021  *   modified 2 Sep 2016
1022  *   by Arturo Guadalupi
1023  *   modified 8 Sep 2016
1024  *   by Colby Newman
1025  *
1026  *   This example code is in the public domain.
1027  *
1028  *   https://www.arduino.cc/en/Tutorial/BuiltInExamples/Blink
1029  */
1030
1031
1032 /**
1033  *   @brief the setup function runs once when you press reset or power the
1034  *   board
1035  *
1036  *   standard function of Arduino sketches
1037  *
1038  *   Initialization of the pin LED_BUILTIN as output
1039  *
1040  *   @param —
1041  *
1042  *   @return void
1043  */
1044 void setup() {
1045     // initialize digital pin LED_BUILTIN as an output.
1046     pinMode(LED_BUILTIN, OUTPUT);
1047 }
1048
1049 /**
1050  *   @brief the loop function runs over and over again forever
1051  *
1052  *   standard function of Arduino sketches
1053  *
1054  *   swichting the led on / off for 1sec.
1055  *
1056  *   @param —
1057  *
1058  *   @return void
1059  */
1060 void loop() {
1061     digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the
1062     voltage level)
1063     delay(1000); // wait for a second
1064     digitalWrite(LED_BUILTIN, LOW); // turn the LED off by making the
1065     voltage LOW
1066     delay(1000); // wait for a second
1067 }

```

../Code/Arduino/Blink/Blink.ino

27. Erstes Kapitel

...

Eine Speicherprogrammierbare Steuerung (SPS) ist ...

Eine Computerized Numerical Control (CNC) benötigt eine SPS (SPS) zur ...

28. CAGD

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln. Das Buch CAGD von Gerald Farin ist ein Klassiker über Splines. [Far02]

Die Norm 66025 zur Programmierung von CNC-Maschinen ist ebenfalls ein Klassiker; sie behandelt allerdings keine Splines. [DIN66025-2]

Herr F. Farouki hat sich sowohl mit der Programmierung von CNC-Maschinen als auch mit Splines beschäftigt. Sein Artikel¹ über einen Echtzeitinterpolator zeigt dies auch. [FS17]

Ein neue Dimension der Werkzeugmaschinen sind durch die Erfindung von 3D-Drucker entstanden. [Rus+07]

Ein anderer Aspekt der Automatisierungstechnik ist die Kommunikation. Durch die 5G-Technik ist hier ein weiterer Meilenstein erreicht worden.²

¹Mitautor ist J. Srinathu

²Zaf+20.

A. Zeichnungen mit tikz

Das Paket tikz ist ein mächtiges Werkzeug zu erstellen von Grafiken. Es existieren vielen Einführungen. Hier werden nur die ersten Schritte gezeigt, so dass man leicht Flussdiagramme erzeugen kann.

Zeichnen einer Linie und Pfeile

B. Kriterien für eine gutes L^AT_EX-Projekt

Ein guter Bericht zeichnet sich nicht nur durch den guten Inhalt aus, sondern erfüllt auch formale Aspekte. Die folgende Liste soll helfen, grundlegende Regeln einzuhalten. Vor der Abgabe sollten alle Punkte geprüft und abgehakt werden.

- ☐ Wird eine geeignete Verzeichnisstruktur verwendet?
- ☐ Wird eine geeignete Aufteilung in Dateien verwendet?
- ☐ Werden aussagekräftige Verzeichnis- und Dateinamen verwendet?
 - ☐ Werden Verzeichnis- und Dateinamen wie z.B. report oder Hausarbeit vermieden?
 - ☐ Werden für Bilder aussagekräftige Namen und nicht „image1“ verwendet?
 - ☐ Sind die Verzeichnis- und Dateinamen nicht zu lang?
 - ☐ Werden Sonderzeichen und Freizeichen vermieden?
- ☐ Werden Befehle wie `\newline` und `\\` vermieden?
- ☐ Werden die L^AT_EX-Dateien übersichtlich gestaltet?
 - ☐ Werden in den L^AT_EX-Dateien Einrückungen verwendet?
 - ☐ Werden Freizeilen eingefügt?
 - ☐ Wird im Header der Dateien das Projekt erwähnt?
 - ☐ Wird im Header der Dateien die Hauptquellen erwähnt?
 - ☐ Wird im Header der Dateien der Autor erwähnt?
- ☐ Werden alle notwendigen Dateien abgegeben?
- ☐ Werden die temporären Dateien gelöscht?
- ☐ Werden Grafiken selber erstellt?
- ☐ Werden die Quellen der Bilder angegeben?
- ☐ Wird mit dem Kommando `\cite` zitiert?
- ☐ Wird eine bib-Datei verwendet?
- ☐ Sind die Einträge der bib-Datei übersichtlich?
- ☐ Sind die Einträge der bib-Datei so editiert, dass sie korrekt dargestellt werden?
- ☐ Werden die korrekten Typen für die bib-Einträge verwendet?
- ☐ Werden aussagekräftige Schlüssel in den bib-Dateien verwendet?

Leider kann die Liste nicht vollständig sein, aber sie liefert einige Anhaltspunkte.

Index

- 3D-Drucker, 147
- 5G, 147
- Speicherprogrammierbare Steuerung, 145
- Datei
 - Blink.ino, 144
 - Blink.py, 142
 - C:/Programme/, 81
 - libraries, 73, 74
 - Nano33BLESenseLED, 74
 - PackageExample.py, 109
 - processing-4.3-windows-x64.zip, 81
 - Processing.app, 81
 - processing.exe, 81
 - Programme, 81
 - requirements.txt, 107
- Abschluss und Fazit der Applikation, 105
- Applikation auf dem Arduino, 105
- Arduino Mbed OS, 105
- Arduino Nano 33 BLE Sense, 105
 - Arduino IDE Konfiguration, 105
 - Bibliotheken Arduino Nano 33 BLE Sense, 105
 - BLE Sense Board, 105
 - Board Manager Eintrag, 105
 - Nano 33 BLE, 105
 - Stromversorgung Nano 33 BLE Sense, 105
- Barometer, 105
- Bluetooth Low Energy, 105
- CAGD, 147
 - Splines, 147
- CNN, 13, 145
- Computerized Numerical Control
 - siehe* CNN, 13, 145
- Datenverarbeitung auf dem Arduino, 105
- Distanzmessung mit Servo, 105
- Domain Knowledge Application, 105
- eingebaute Sensoren, 105
- Example, 109
 - Description, 109
 - Installation, 109
 - Introduction, 109
- HC-SR04, 105
- IMU Sensor, 105
- Interrupts und Timer, 105
- Messzyklus Arduino, 105
- Monitoring der Messdaten, 105
- nRF52840 Mikrocontroller, 105
- Onboard Mikrofon, 105
- Radar-Applikation, 105
- scannender Ultraschall-Radar, 105
- Serial Plotter Ausgabe, 105
- Servomotor
 - Drehbereich SG90, 105
 - Getriebe Servo, 105
 - mechanische Befestigung Servo, 105
 - Positionsregelung Servo, 105
 - PWM-Signal Servo, 105
 - Servoansteuerung mit Arduino, 105
 - Servohorn, 105
 - Servostromaufnahme, 105
 - Stellwinkel Servo, 105
- Servomotor TowerPro SG90, 105
- SG90 Servomotor, 105
- Softwarearchitektur Radar, 105
- Speicherprogrammierbare Steuerung
 - siehe* SPS, 13, 145
- SPS, 13, 145, *siehe* Speicherprogrammierbare Steuerung
- State Machine Applikation, 105
- Temperatur- und Feuchtigkeitssensor, 105
- TowerPro SG90, 105
- Ultraschallsensor, 105
 - Ansteuerung HC-SR04 mit Arduino, 105
 - Echo Pin, 105
 - Entfernungsmessung Ultraschall, 105
 - Messbereich HC-SR04, 105
 - Messfehler Ultraschallsensor, 105
 - Montageposition Ultraschallsensor, 105
 - Störquellen Ultraschall, 105
 - Trigger Pin, 105

Versorgungsspannung HC-SR04, 105

Zeit-zu-Distanz-Berechnung, 105

Ultraschallsensor HC-SR04, 105

Verification und Evaluation, 105

Visualisierung Radardaten, 105